

MATHÉMATIQUES • TLED

ORGANISATION ET GESTION DES DONNÉES

LEÇON 12

THÉORIE DES GRAPHEs

DURÉE CONSEILLÉE : 6 H

DANS CETTE LEÇON

Cette leçon introduit la théorie des graphes : modélisation de réseaux (routes, télécoms, réseaux sociaux), notions de connexité et d'arbres, puis trois algorithmes fondamentaux — parcours en largeur, arbre couvrant de poids minimal (Kruskal, Prim) et plus court chemin (Dijkstra). Elle fournit des compétences directement exigibles au Baccalauréat C, D, E et TI.

- À la fin de cette leçon, je sais modéliser une situation concrète (réseau routier, télécom, social) par un graphe orienté ou non, pondéré ou non.
- À la fin de cette leçon, je sais identifier sommets, arêtes, chaînes, chemins, cycles et calculer la longueur d'une chaîne.
- À la fin de cette leçon, je sais déterminer si un graphe est connexe et en extraire un sous-graphe ou un arbre couvrant.
- À la fin de cette leçon, je sais exécuter un parcours en largeur (BFS) depuis un sommet donné et interpréter l'ordre de visite.
- À la fin de cette leçon, je sais appliquer l'algorithme de Kruskal pour déterminer un arbre couvrant de poids minimal.
- À la fin de cette leçon, je sais appliquer l'algorithme de Prim pour déterminer un arbre couvrant de poids minimal.



TÉLÉCHARGE ONBUCH

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats



PASSE À ONBUCH+

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir



Signé • OnBuch+

ÉQUIPE PÉDAGOGIQUE ONBUCH+
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

- | | |
|---|---|
| 1. Graphes : définitions et modélisation | Dijkstra |
| 2. Chaînes, chemins, cycles et connexité | 7. Synthèse et modélisation de situations |
| 3. Arbres et arbres couvrants | 8. Activité d'intégration |
| 4. Parcours en largeur (BFS) | 9. Méthodes et exercices résolus |
| 5. Arbre couvrant de poids minimal :
Kruskal et Prim | 10. Exercices |
| 6. Plus court chemin : algorithme de | 11. Corrigés des exercices |
| | 12. Fiche bilan |

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais modéliser une situation concrète (réseau routier, télécom, social) par un graphe orienté ou non, pondéré ou non.
- À la fin de cette leçon, je sais identifier sommets, arêtes, chaînes, chemins, cycles et calculer la longueur d'une chaîne.
- À la fin de cette leçon, je sais déterminer si un graphe est connexe et en extraire un sous-graphe ou un arbre couvrant.
- À la fin de cette leçon, je sais exécuter un parcours en largeur (BFS) depuis un sommet donné et interpréter l'ordre de visite.
- À la fin de cette leçon, je sais appliquer l'algorithme de Kruskal pour déterminer un arbre couvrant de poids minimal.
- À la fin de cette leçon, je sais appliquer l'algorithme de Prim pour déterminer un arbre couvrant de poids minimal.
- À la fin de cette leçon, je sais appliquer l'algorithme de Dijkstra pour déterminer le plus court chemin entre deux sommets et donner sa longueur.
- À la fin de cette leçon, je sais interpréter les résultats obtenus (coût minimal, distance minimale, connexité) dans le contexte de la situation modélisée.

PRÉREQUIS

- Notions de dénombrement et de raisonnement logique (disjonction de cas, récurrence simple) vues en Première

- Lecture et exploitation de tableaux de données (tableaux de distances, matrices simples)
- Notion de fonction et de minimum d'une quantité (optimisation élémentaire)
- Représentation de relations entre objets (diagrammes, tableaux à double entrée)

1 GRAPHES : DÉFINITIONS ET MODÉLISATION

1.1 Une situation bien concrète : trois problèmes, un seul objet

Imagine les trois scènes suivantes, toutes bien réelles au Cameroun.

- La SODECOTON veut relier par fibre optique 6 villes du Nord-Cameroun (Garoua, Maroua, Ngaoundéré, Guider, Figuil, Touboro) au moindre coût : chaque tronçon possible a un prix connu. Quels tronçons construire pour que toutes les villes soient reliées, sans gaspiller un seul milliard de FCFA ?
- Un transporteur de Yaoundé cherche l'itinéraire le plus court entre deux quartiers en tenant compte des embouteillages. Quelle route choisir ?
- Un chef de réseau MTN veut vérifier que son réseau reste connecté après la panne d'une antenne. Le réseau tient-il encore debout ?

Ces trois problèmes, en apparence différents, se ramènent à un même objet mathématique : le **graphe**. Un graphe, c'est simplement un ensemble de points (les villes, les quartiers, les antennes) reliés par des lignes (les tronçons de fibre, les rues, les liaisons radio). Toute la richesse de la théorie des graphes vient de cette simplicité : en oubliant la géographie réelle pour ne garder que « qui est relié à qui », on obtient un langage commun capable de traiter des milliers de situations.

Problématique de la leçon. Comment modéliser une situation concrète par un graphe, et quels outils mathématiques permettent ensuite de répondre aux questions de coût minimal, de plus court chemin et de connexité ?

LESAVAIS-TU ? Un peu d'histoire

La théorie des graphes est née en 1736 à Königsberg (aujourd'hui Kaliningrad). Les habitants se demandaient s'il était possible de traverser chacun des sept ponts de la ville une seule fois et de revenir à son point de départ. Le grand mathématicien suisse Leonhard Euler a répondu par la négative, en ramenant la ville à un schéma de points et de lignes : le premier graphe de l'histoire ! Aujourd'hui, les graphes sont partout : réseaux sociaux, GPS, internet, planification des examens du baccalauréat.

1.2 Graphe non orienté : définition et représentation

Partons de l'intuition. Prends une carte du réseau routier entre quelques villes. Gomme les rivières, les reliefs, les distances exactes : ne garde que les villes (des points) et les routes directes entre elles (des lignes). Tu viens de construire un graphe.

DÉFINITION Graphe non orienté

Un **graphe non orienté** G est la donnée d'un couple (S, A) où :

- S est un ensemble fini non vide dont les éléments sont appelés les **sommets** du graphe ;
- A est un ensemble de paires $\{u, v\}$ de sommets distincts, appelées les **arêtes** du graphe.

On note $G = (S, A)$. L'arête $\{u, v\}$ se note aussi simplement uv ou vu : l'ordre n'a pas d'importance.

On représente un graphe par un schéma sagittal : chaque sommet est un point (ou un petit cercle étiqueté), et chaque arête est un trait joignant les deux sommets concernés. Attention : la position des points sur le dessin n'a aucune importance ; seules comptent les liaisons.

EXEMPLE Le réseau routier du Grand Nord

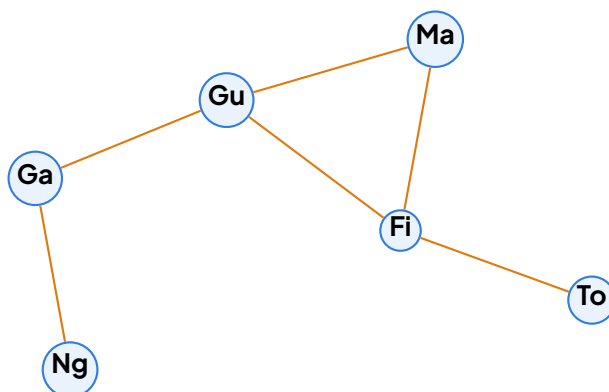
Considérons le graphe $G = (S, A)$ avec :

$$S = \{\text{Ga}, \text{Ma}, \text{Ng}, \text{Gu}, \text{Fi}, \text{To}\}$$

(Garoua, Maroua, Ngaoundéré, Guider, Figuil, Touboro) et

$$A = \{\{\text{Ga}, \text{Gu}\}, \{\text{Ga}, \text{Ng}\}, \{\text{Gu}, \text{Ma}\}, \{\text{Gu}, \text{Fi}\}, \{\text{Ma}, \text{Fi}\}, \{\text{Fi}, \text{To}\}\}.$$

Ce graphe a 6 sommets et 6 arêtes. Par exemple, l'arête $\{\text{Ga}, \text{Gu}\}$ traduit l'existence d'une route directe entre Garoua et Guider.



Le réseau routier modélisé par un graphe non orienté à 6 sommets et 6 arêtes.
La position des villes sur le dessin est arbitraire : seules les liaisons comptent.

1.3 Vocabulaire fondamental : adjacence, incidence, degré, ordre

DÉFINITION Adjacence, incidence, degré, ordre

Soit $G = (S, A)$ un graphe non orienté.

- Deux sommets u et v sont dits **adjacents** (ou voisins) s'il existe une arête $\{u, v\}$ dans A .
- Une arête $a = \{u, v\}$ est dite **incidente** aux sommets u et v , qui sont ses extrémités.
- Le **degré** d'un sommet v , noté $d(v)$, est le nombre d'arêtes incidentes à v (c'est-à-dire le nombre de ses voisins).
- L'**ordre** du graphe G est le nombre de ses sommets, c'est-à-dire le cardinal de S .

EXEMPLE Calcul des degrés sur le réseau du Grand Nord

Reprenons le graphe de l'exemple précédent. Comptons, pour chaque ville, le nombre de routes directes qui en partent :

Sommet	Ga	Ma	Ng	Gu	Fi	To
Degré	2	2	1	3	3	1

Ainsi Guider et Figuil sont des carrefours (degré 3), tandis que Ngaoundéré et Touboro sont des « bouts » de réseau (degré 1). Le graphe est d'ordre 6.

ATTENTION Boucles et arêtes multiples

Une **boucle** est une arête reliant un sommet à lui-même ; des arêtes multiples relient plusieurs fois la même paire de sommets. Dans cette leçon, sauf mention contraire, nous travaillons sur des graphes simples : sans boucle et sans arête multiple. Dans un graphe simple d'ordre n , le degré de tout sommet vérifie donc $0 \leq d(v) \leq n - 1$.

1.4 Le lemme des poignées de main

Voici un résultat aussi simple qu'utile. Dans une réunion, si chaque participant compte le nombre de poignées de main qu'il a échangées, la somme de tous ces nombres est égale au double du nombre total de poignées de main : chaque poignée de main a été comptée deux fois, une fois par chacun des deux participants. Transposé aux graphes, cela donne :

PROPRIÉTÉ Lemme des poignées de main

Dans tout graphe non orienté $G = (S, A)$, la somme des degrés des sommets est égale au double du nombre d'arêtes :

$$\sum_{v \in S} d(v) = 2 |A|.$$

En particulier, la somme des degrés est toujours un nombre pair.

EXPÉRIENCE**Démonstration guidée du lemme des poignées de main**

Raisonnons en comptant de deux façons différentes le nombre de couples (v, a) où v est un sommet et a une arête incidente à v .

1. Comptage par les sommets. Pour chaque sommet v , il y a exactement $d(v)$ arêtes incidentes à v . Le nombre de couples (v, a) est donc $\sum_{v \in S} d(v)$.
2. Comptage par les arêtes. Chaque arête $a = \{u, v\}$ possède exactement deux extrémités, donc apparaît dans exactement deux couples : (u, a) et (v, a) . Le nombre de couples est donc $2 |A|$.

Les deux comptages portent sur le même ensemble de couples, ils donnent le même résultat :

$$\sum_{v \in S} d(v) = 2 |A|. \quad \blacksquare$$

EXERCICE RÉSOLU 1**Vérification du lemme sur le réseau du Grand Nord**

Énoncé. On reprend le graphe des 6 villes du Nord dont les degrés sont : $d(\text{Ga}) = 2$, $d(\text{Ma}) = 2$, $d(\text{Ng}) = 1$, $d(\text{Gu}) = 3$, $d(\text{Fi}) = 3$, $d(\text{To}) = 1$. Vérifier le lemme des poignées de main.

SOLUTION

Solution. La somme des degrés vaut :

$$2 + 2 + 1 + 3 + 3 + 1 = 12.$$

Le graphe possède $|A| = 6$ arêtes, donc $2 |A| = 12$. On a bien $\sum d(v) = 2 |A|$: le lemme est vérifié. Ce contrôle rapide est un excellent réflexe au baccalauréat pour détecter une erreur de comptage des degrés.

ATTENTION**Conséquence à connaître**

Dans tout graphe, le nombre de sommets de degré impair est toujours pair. En effet, si ce

nombre était impair, la somme des degrés serait impaire, ce qui contredit le lemme. On ne peut donc jamais avoir, par exemple, exactement 3 sommets de degré impair dans un graphe.

1.5 Graphes orientés : quand le sens compte

Jusqu'ici, nos liaisons étaient « à double sens » : une route relie Garoua à Guider dans les deux sens. Mais certaines situations sont asymétriques : une rue en sens unique à Yaoundé, la relation « suit » sur un réseau social (tu peux suivre quelqu'un sans être suivi en retour), un transfert d'argent Mobile Money. Il faut alors des flèches.

DÉFINITION Graphe orienté

Un **graphe orienté** $G = (S, A)$ est la donnée d'un ensemble fini S de sommets et d'un ensemble A de couples ordonnés (u, v) de sommets distincts, appelés **arcs**. L'arc (u, v) va de u vers v : u est son origine (ou extrémité initiale) et v son extrémité finale.

DÉFINITION Successeurs, prédécesseurs, degrés entrant et sortant

Soit $G = (S, A)$ un graphe orienté et v un sommet.

- Les **successeurs** de v sont les sommets w tels que $(v, w) \in A$ (les arcs qui partent de v).
- Les **prédécesseurs** de v sont les sommets u tels que $(u, v) \in A$ (les arcs qui arrivent à v).
- Le **degré sortant** $d^+(v)$ est le nombre de successeurs de v ; le **degré entrant** $d^-(v)$ est le nombre de prédécesseurs de v .

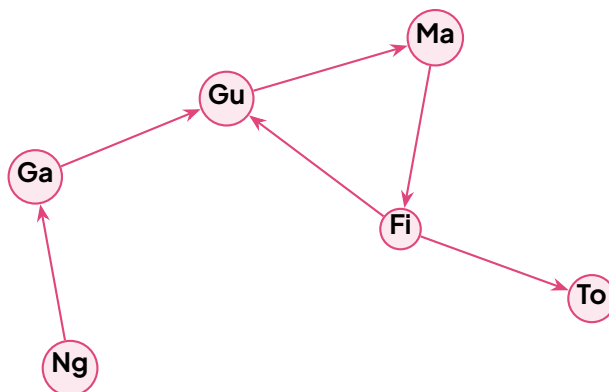
On a toujours : $\sum_{v \in S} d^+(v) = \sum_{v \in S} d^-(v) = |A|$, car chaque arc possède exactement une origine et une extrémité finale.

EXEMPLE Sens uniques dans le réseau

Imaginons que certains tronçons deviennent à sens unique (travaux, pont à voie unique). On obtient le graphe orienté ci-dessous, avec les arcs :

$$(Ga, Gu), (Ng, Ga), (Gu, Ma), (Fi, Gu), (Ma, Fi), (Fi, To).$$

Par exemple, Guider a pour successeur Maroua ($d^+(Gu) = 1$) et pour prédécesseurs Garoua et Figuil ($d^-(Gu) = 2$).



Version orientée du réseau : chaque arc est un sens unique. Par exemple, on peut aller de Garoua à Guider, mais pas l'inverse directement.

ATTENTION Arête ou arc : ne pas confondre !

C'est l'erreur la plus fréquente. Une arête $\{u, v\}$ est non ordonnée : $\{u, v\} = \{v, u\}$, elle se dessine par un simple trait. Un arc (u, v) est ordonné : $(u, v) \neq (v, u)$, il se dessine par une flèche de u vers v . Avant tout calcul, demande-toi : la situation est-elle symétrique (arêtes) ou asymétrique (arcs) ?

1.6 Sous-graphes

Lorsque le chef de réseau MTN étudie la panne d'une antenne, il « retire » un sommet et ses liaisons du réseau : il considère un sous-graphe.

DÉFINITION Sous-graphe, sous-graphe couvrant

Soit $G = (S, A)$ un graphe. Un **sous-graphe** de G est un graphe $G' = (S', A')$ tel que :

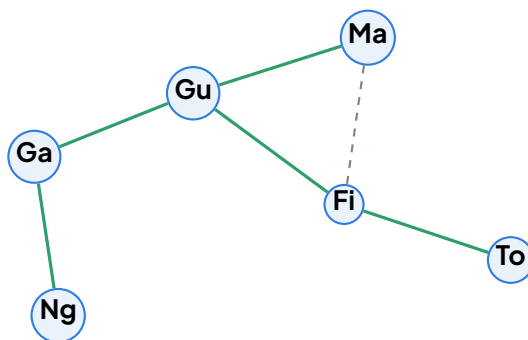
$$S' \subseteq S \quad \text{et} \quad A' \subseteq A,$$

chaque arête de A' ayant bien sûr ses deux extrémités dans S' . Si de plus $S' = S$ (on garde tous les sommets, en ne supprimant que des arêtes), on dit que G' est un **sous-graphe couvrant** de G .

EXEMPLE Un sous-réseau du Grand Nord

Dans le graphe des 6 villes, le graphe G' de sommets $S' = \{Ga, Gu, Ma\}$ et d'arêtes $A' = \{\{Ga, Gu\}, \{Gu, Ma\}\}$ est un sous-graphe de G . Le graphe G'' obtenu en gardant les 6 villes

mais seulement les 5 arêtes $\{Ga,Gu\}$, $\{Ga,Ng\}$, $\{Gu,Ma\}$, $\{Gu,Fi\}$, $\{Fi,To\}$ est un sous-graphe couvrant : toutes les villes y figurent encore. Cette notion sera centrale pour les arbres couvrants de la section 3.



Un sous-graphe couvrant (en vert, 5 arêtes) du graphe complet : tous les sommets sont conservés, seule l'arête $\{Ma,Fi\}$ (en pointillés) a été retirée.

1.7 Modéliser une situation par un graphe

C'est la compétence clé du programme : traduire un énoncé concret en un objet mathématique. Voici la démarche à adopter systématiquement.

MÉTHODE Modéliser une situation par un graphe en 3 étapes

1. Identifier les objets de la situation (villes, personnes, équipes, antennes, quartiers...) : ce seront les **sommets**.
2. Identifier les liaisons entre ces objets (routes, relations, matchs, câbles...) : ce seront les **arêtes** si la relation est symétrique, les **arcs** si elle a un sens.
3. Reporter les données chiffrées (distances, coûts, durées, capacités...) sous forme d'**étiquettes** sur les arêtes ou les arcs : on obtient alors un graphe pondéré, étudié en section 5.

Termine toujours en reformulant la question posée en langage de graphes : « trouver le chemin le plus court entre deux sommets », « vérifier que le graphe reste connexe », etc.

EXEMPLE Trois modélisations guidées

(a) Lignes de bus. Sommets : les arrêts de bus d'une ville. Arêtes : deux arrêts consécutifs

sur une même ligne. Étiquettes : durée du trajet en minutes. Question traduite : quel est le chemin de durée totale minimale entre deux arrêts ?

(b) Tournoi sportif. Lors d'un championnat où chaque équipe rencontre chaque autre une fois, les sommets sont les équipes et on place un arc (X, Y) si l'équipe X a battu l'équipe Y . Le graphe est orienté car « battre » n'est pas symétrique. Le degré sortant d'une équipe est son nombre de victoires.

(c) Réseau d'antennes. Sommets : les antennes relais MTN d'une région. Arêtes : deux antennes à portée radio l'une de l'autre. Question traduite : le graphe est-il connexe, c'est-à-dire toute antenne peut-elle communiquer avec toute autre, éventuellement par relais intermédiaires ?

2 CHAÎNES, CHEMINS, CYCLES ET CONNEXITÉ

Dans la section précédente, nous avons appris à modéliser une situation par un graphe : les sommets représentent des lieux, des personnes ou des états, et les arêtes matérialisent les liaisons. Mais un graphe n'est pas une fin en soi : c'est un outil pour répondre à des questions. Or la plupart de ces questions se ramènent à une interrogation fondamentale : **peut-on aller d'un sommet à un autre, et comment ?** Peut-on relier Bonabéri à Akwa par la route ? Le message d'un abonné MTN peut-il atteindre n'importe quel autre abonné du réseau ? Existe-t-il un itinéraire qui revient à son point de départ ? Pour y répondre rigoureusement, il nous faut un vocabulaire précis : celui des **chaînes**, des **chemins**, des **cycles** et de la **connexité**. C'est l'objet de cette section, qui pose les fondations de tous les algorithmes de la leçon (BFS, Kruskal, Prim, Dijkstra).

2.1 Chaînes et longueur d'une chaîne

Intuition. Imagine que tu te déplaces dans Douala en suivant les rues. À chaque intersection, tu choisis une rue qui part de là où tu es. La liste des quartiers que tu traverses successivement forme ce que les mathématiciens appellent une chaîne. Rien ne t'interdit, dans cette définition de base, de repasser deux fois par le même carrefour : tu as le droit de « faire des détours inutiles ».

DÉFINITION Chaîne et longueur

Soit $G = (S, A)$ un graphe non orienté.

- Une **chaîne** est une suite de sommets $(s_0, s_1, \dots, s_k)$ telle que deux sommets consécutifs quelconques s_i et s_{i+1} sont reliés par une arête de G .
- Les sommets s_0 et s_k sont appelés les **extrémités** de la chaîne ; on dit que la chaîne relie s_0 à s_k .
- La **longueur** d'une chaîne est le nombre d'arêtes qu'elle emprunte, c'est-à-dire l'entier k dans l'écriture ci-dessus.

ATTENTION Longueur = nombre d'arêtes, pas de sommets

C'est l'erreur la plus fréquente au Baccalauréat. La chaîne (A, B, C, D) contient 4 sommets mais sa longueur est 3, car elle emprunte les arêtes $\{A, B\}$, $\{B, C\}$ et $\{C, D\}$. Retiens l'image du randonneur : la longueur, c'est le nombre de pas (les arêtes), pas le nombre de positions occupées (les sommets). Une chaîne réduite à un seul sommet (A) est de longueur 0 : c'est la chaîne triviale.

Toutes les chaînes ne se valent pas. Certaines « se promènent » en repassant plusieurs fois par les mêmes endroits ; d'autres sont économes. On distingue deux cas particuliers essentiels.

DÉFINITION Chaîne simple, chaîne élémentaire

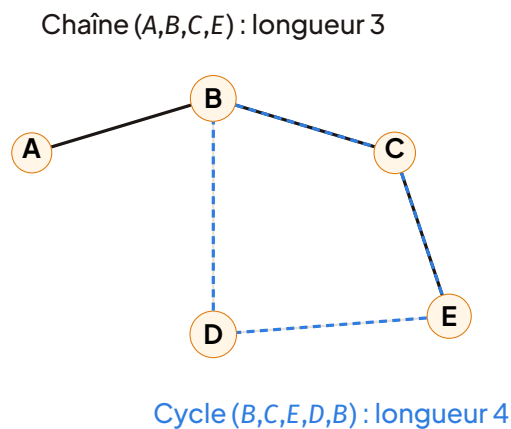
- Une chaîne est dite **simple** si elle n'emprunte jamais deux fois la même arête.
- Une chaîne est dite **élémentaire** si elle ne passe jamais deux fois par le même sommet.

Toute chaîne élémentaire est simple, mais la réciproque est fausse : une chaîne simple peut repasser par un sommet déjà visité (en empruntant des arêtes différentes).

EXEMPLE Lecture sur un graphe

Considère le graphe ci-dessous dont les sommets sont A, B, C, D, E et les arêtes : $\{A, B\}$, $\{B, C\}$, $\{C, E\}$, $\{B, D\}$, $\{D, E\}$.

- (A, B, C, E) est une chaîne élémentaire de longueur 3 (tracée en rose) ;
- (A, B, D, B, C) est une chaîne de longueur 4 qui n'est ni simple ni élémentaire : elle repasse par B et réutilise l'arête $\{B, D\}$;
- (B, C, E, D, B) est une chaîne fermée de longueur 4 : elle revient à son point de départ.



En rose plein, une chaîne élémentaire de longueur 3 ; en bleu pointillé, un cycle de longueur 4. La longueur se compte en arêtes. Les arêtes communes (B-C, C-E) appartiennent aux deux parcours.

2.2 Cycles : les chaînes qui bouclent

Intuition. Un agent commercial part de son agence le matin, visite ses clients et rentre le soir à l'agence. Son parcours « boucle ». En théorie des graphes, ces parcours fermés jouent un rôle capital : leur présence ou leur absence caractérisera les arbres de la section suivante.

DÉFINITION Cycle

Un **cycle** est une chaîne simple dont les deux extrémités coïncident : $s_0 = s_k$, avec $k \geq 1$. Un cycle est dit **élémentaire** si, hormis les extrémités, il ne passe jamais deux fois par le même sommet.

ATTENTION Cycle et graphe simple

Dans un graphe simple (sans boucle ni arêtes multiples), un cycle a nécessairement une longueur $k \geq 3$. La suite (B, D, B), qui aller-retourne sur la même arête, n'est pas un cycle car la chaîne n'est pas simple (l'arête {B,D} est empruntée deux fois).

Sur la figure précédente, (B, C, E, D, B) est un cycle élémentaire de longueur 4.

PROPRIÉTÉ Élagage d'une chaîne

Si deux sommets distincts x et y sont reliés par une chaîne, alors ils sont reliés par une chaîne élémentaire.

EXPÉRIENCE Démonstration guidée (formatrice)

On part d'une chaîne quelconque $C = (s_0, s_1, \dots, s_k)$ reliant x à y , et on va « élaguer » ses détours.

1. Supposons que C passe deux fois par un même sommet : il existe $i < j$ avec $s_i = s_j$.
2. Le morceau $(s_i, s_{i+1}, \dots, s_j)$ est alors un aller-retour inutile : supprimons-le. La suite $(s_0, \dots, s_i, s_{j+1}, \dots, s_k)$ est encore une chaîne de x à y (car $s_i = s_j$ est adjacent à s_{j+1}), strictement plus courte.
3. Tant qu'un sommet est répété, on recommence. Le nombre d'étapes est fini car la longueur diminue strictement à chaque fois.
4. À la fin, aucun sommet n'est répété : la chaîne obtenue est élémentaire. ■

Ce résultat, très intuitif, justifie que pour étudier la connexité ou les distances, on puisse toujours se ramener à des chaînes élémentaires.

2.3 Cas des graphes orientés : chemins et circuits

Quand le graphe est orienté (rues à sens unique, liens « suivre » sur un réseau social, étapes d'un protocole), les arêtes deviennent des arcs et le vocabulaire change.

DÉFINITION Chemin et circuit

Soit $G = (S, A)$ un graphe orienté.

- Un **chemin** est une suite de sommets $(s_0, s_1, \dots, s_k)$ telle que, pour tout i , l'arc (s_i, s_{i+1}) existe dans ce sens. Sa **longueur** est le nombre k d'arcs.
- Un **circuit** est un chemin fermé ($s_0 = s_k$) n'empruntant jamais deux fois le même arc.

ATTENTION Respecter le sens des flèches

Dans un graphe orienté, l'existence d'un arc (A, B) ne garantit pas qu'on puisse aller de B vers A . Avant d'affirmer qu'un chemin existe, vérifie le sens de chaque arc de la suite. Une

rue à sens unique ne se remonte pas, même en mathématiques !

2.4 Distance entre deux sommets

Intuition. Entre deux quartiers, il existe en général plusieurs itinéraires. Le plus court (en nombre de rues traversées, ou plus tard en kilomètres) définit naturellement une distance. C'est exactement cette notion que l'algorithme de Dijkstra (section 6) calculera efficacement sur des graphes pondérés.

DÉFINITION Distance dans un graphe non pondéré

Soient x et y deux sommets d'un graphe. La **distance** de x à y , notée $d(x, y)$, est la longueur de la plus courte chaîne reliant x à y (c'est-à-dire le nombre minimal d'arêtes à emprunter). Par convention, $d(x, x) = 0$. Si x et y ne sont pas reliés (graphe non connexe), on pose $d(x, y) = +\infty$.

EXERCICE RÉSOLU 2 Distances entre quartiers de Douala

On modélise six quartiers de Douala par le graphe suivant : les sommets sont A (Akwa), B (Bonanjo), C (Bonabéri), D (Deïdo), E (Bali) et F (Ndokoti). Les arêtes sont : $\{A, B\}$, $\{A, D\}$, $\{A, E\}$, $\{B, C\}$, $\{D, E\}$, $\{D, F\}$, $\{E, F\}$. Déterminer $d(B, F)$, $d(C, E)$ et $d(C, F)$.

SOLUTION

Recherche de $d(B, F)$. La chaîne (B, A, D, F) relie B à F en 3 arêtes. Existe-t-il mieux ? Une chaîne de longueur 2 passerait par un voisin commun de B et F ; or les voisins de B sont $\{A, C\}$ et ceux de F sont $\{D, E\}$: aucun sommet commun. Donc $d(B, F) = 3$.

Recherche de $d(C, E)$. La chaîne (C, B, A, E) est de longueur 3. Les voisins de C se réduisent à $\{B\}$; tout chemin partant de C commence par B , et $d(B, E) = 2$ via (B, A, E) . Donc $d(C, E) = 1 + 2 = 3$.

Recherche de $d(C, F)$. De même, $d(C, F) = 1 + d(B, F) = 4$, réalisée par (C, B, A, D, F) ou (C, B, A, E, F) .

Interprétation. Un moto-taxi partant de Bonabéri doit traverser au minimum 4 « tronçons » du réseau modélisé pour atteindre Ndokoti : c'est le quartier le plus excentré vu de Bonabéri.

2.5 Graphes connexes et composantes connexes

Intuition. En saison des pluies, il arrive qu'une piste soit coupée entre deux villages : le réseau routier de la région se « brise » alors en morceaux étanches. Certains couples de sommets ne sont plus reliés par aucune chaîne. C'est exactement le phénomène que capture la notion de connexité.

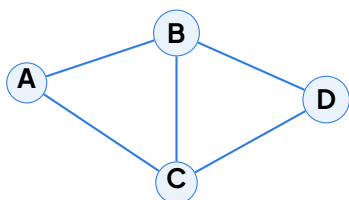
DÉFINITION Graphe connexe, composante connexe

- Un graphe non orienté G est dit **connexe** si, pour tout couple de sommets (x, y) , il existe une chaîne reliant x à y .
- Une **composante connexe** de G est un sous-graphe connexe maximal : on ne peut lui ajouter aucun sommet de G sans perdre la connexité.

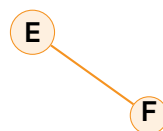
Un graphe est donc connexe si et seulement s'il possède une seule composante connexe. Un sommet sans aucune arête forme à lui seul une composante connexe (on parle de sommet **isolé**).

EXEMPLE Le réseau routier et l'île isolée

Considère le graphe suivant : les sommets A, B, C, D représentent quatre villages du continent reliés par des pistes, et les sommets E, F représentent deux villages d'une île reliés entre eux par une piste locale, mais sans pont ni bac vers le continent. Le graphe possède deux composantes connexes : $\{A, B, C, D\}$ et $\{E, F\}$. Il n'est pas connexe : aucune chaîne ne relie, par exemple, A à E . Construire un pont (ajouter une arête entre les deux blocs) rendrait le graphe connexe : c'est l'enjeu concret de nombreux projets de désenclavement rural au Cameroun.



Composante connexe 1



Composante connexe 2

Un graphe non connexe à deux composantes connexes : aucune arête ne relie le bloc bleu au bloc orange (réseau coupé, île sans pont).

2.6 Vérifier la connexité en pratique

Vérifier la définition « à la main » exigerait de tester tous les couples de sommets : pour n sommets, cela fait $\frac{n(n-1)}{2}$ couples, vite ingérable. Heureusement, il suffit de partir d'un **seul** sommet et de marquer progressivement tout ce qu'on peut atteindre.

MÉTHODE Tester la connexité par marquage progressif

1. Choisir un sommet de départ s quelconque et le marquer.
2. Marquer tous les voisins des sommets déjà marqués.
3. Répéter l'étape 2 tant que de nouveaux sommets sont marqués.
4. À la fin : si tous les sommets sont marqués, le graphe est connexe ; sinon, il ne l'est pas, et les sommets marqués forment exactement la composante connexe de s .

Ce marquage « en taches d'encre » est précisément l'idée du **parcours en largeur** (BFS), que nous formaliserons en section 4 : on explore le graphe couche par couche, en s'éloignant progressivement de s .

EXEMPLE Application au réseau coupé

Sur le graphe de la figure précédente, partons de A : on marque A , puis ses voisins B et C , puis le voisin D de B et C . Plus aucun sommet nouveau n'est atteignable : les sommets marqués sont $\{A, B, C, D\}$. Comme E et F restent non marqués, le graphe n'est pas connexe, et $\{A, B, C, D\}$ est la composante connexe de A . En repartant de E , on obtiendrait la seconde composante $\{E, F\}$.

À RETENIR L'essentiel de la section

- Une **chaîne** est une suite de sommets consécutifs adjacents ; sa **longueur** est son nombre d'arêtes.
- Une chaîne est **simple** (pas deux fois la même arête) ou **élémentaire** (pas deux fois le même sommet) ; un **cycle** est une chaîne simple fermée (longueur ≥ 3 dans un graphe simple).
- En orienté, on parle de **chemin** et de **circuit**, en respectant le sens des arcs.
- La **distance** $d(x, y)$ est la longueur de la plus courte chaîne reliant x à y ($+\infty$ si non reliés).
- Un graphe est **connexe** si tout couple de sommets est relié ; sinon il se décompose en **composantes connexes**, que l'on détermine par marquage progressif (futur BFS).

LE SAVAIS-TU ? Le « petit monde » : six poignées de main

En 1967, le psychologue américain Stanley Milgram mena une expérience célèbre : il demanda à des habitants du Nebraska d'envoyer un courrier à un inconnu de Boston, en le

transmettant uniquement à des connaissances personnelles. Les lettres arrivées à destination avaient traversé en moyenne six intermédiaires. Autrement dit, dans le graphe gigantesque des connaissances humaines (les sommets sont les individus, les arêtes les relations « se connaissent »), la distance moyenne entre deux personnes serait d'environ 6 ! Avec les réseaux sociaux, cette distance a encore fondu : Facebook mesurait une moyenne d'environ 3,5 en 2016. La prochaine fois que tu découvres qu'un ami connaît un ami d'un ami qui connaît une célébrité, souviens-toi : ce n'est pas un hasard, c'est un théorème de graphes en action — et la distance que tu as calculée entre Bonabéri et Ndokoti relève exactement de la même mathématique.

Nous disposons désormais du vocabulaire des parcours et de la connexité. Dans la section suivante, nous étudierons les graphes sans cycle qui restent connexes : les **arbres**, squelettes économiques des réseaux, et leurs arbres couvrants.

3 ARBRES ET ARBRES COUVRANTS

Dans la section précédente, nous avons appris à nous déplacer dans un graphe : chaînes, chemins, cycles, et nous avons dégagé la notion fondamentale de **connexité** — un graphe est connexe lorsqu'on peut toujours relier deux sommets quelconques par une chaîne. Une question naturelle se pose alors : un graphe connexe contient-il des arêtes « inutiles » ? Autrement dit, peut-on supprimer des arêtes tout en conservant la connexité, jusqu'à obtenir le « squelette minimal » du réseau ? C'est exactement l'objet de cette section : ce squelette minimal s'appelle un **arbre**, et sa version incluse dans un graphe donné s'appelle un **arbre couvrant**. Ces notions sont au cœur de problèmes très concrets : câbler un quartier à moindre coût, interconnecter des agences bancaires, déployer la fibre optique de CAMTEL entre des villes.

3.1 La notion d'arbre : connexe et sans cycle

Intuition. Imagine un réseau de pistes rurales reliant des villages. Si le réseau contient un cycle (une boucle), alors l'une des pistes de cette boucle est redondante : on peut la fermer, les villages restent reliés entre eux. Un réseau sans aucune boucle mais où tous les villages restent accessibles est un réseau « économe » : c'est un arbre. Le nom vient de la forme de ces graphes, qui se ramifient comme les branches d'un arbre, sans jamais se reboucler.

DÉFINITION Arbre

Un **arbre** est un graphe **connexe** et **sans cycle**.

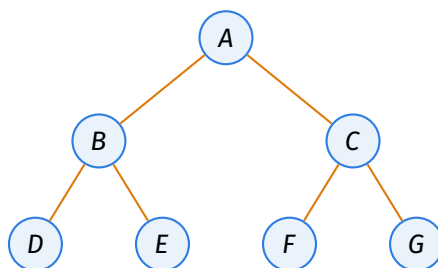
EXEMPLE Reconnaître un arbre

Le graphe formé des sommets A, B, C, D et des arêtes $\{A, B\}, \{B, C\}, \{B, D\}$ est un arbre : il est

connexe (tout sommet est relié à tout autre via B) et ne contient aucun cycle. En revanche, si l'on ajoute l'arête $\{C, D\}$, on crée le cycle $B - C - D - B$: le graphe n'est plus un arbre. De même, le graphe formé des arêtes $\{A, B\}$ et $\{C, D\}$ seules n'est pas un arbre, car il n'est pas connexe.

ATTENTION Les deux conditions sont indispensables

Un graphe sans cycle n'est pas forcément un arbre : il peut être non connexe (on parle alors de forêt, c'est-à-dire une réunion d'arbres). Un graphe connexe n'est pas forcément un arbre : il peut contenir des cycles. Il faut retenir : arbre = connexe + sans cycle, les deux à la fois.



Un arbre à 7 sommets. Comptons ses arêtes : AB, AC, BD, BE, CF, CG , soit exactement $6 = 7 - 1$ arêtes. Il est connexe et ne contient aucun cycle.

3.2 Propriétés caractéristiques des arbres

Intuition. Sur la figure ci-dessus, une remarque saute aux yeux : avec 7 sommets, l'arbre possède exactement 6 arêtes. Ce n'est pas un hasard. Chaque arête d'un arbre « accroche » un nouveau sommet au réseau ; une fois le premier sommet posé, il faut exactement une arête par sommet supplémentaire, ni plus (sinon on crée un cycle), ni moins (sinon un sommet reste isolé).

PROPRIÉTÉ Nombre d'arêtes d'un arbre

Un arbre à n sommets ($n \geq 1$) possède exactement $n - 1$ arêtes.

EXPÉRIENCE Démonstration guidée par récurrence sur n

Démontrons par récurrence la propriété $P(n)$: « tout arbre à n sommets possède $n - 1$

arêtes ».

1. Initialisation. Pour $n = 1$, l'arbre est réduit à un sommet isolé : il possède $0 = 1 - 1$ arête. $P(1)$ est vraie.
2. Hérédité. Supposons $P(n)$ vraie pour un entier $n \geq 1$, et considérons un arbre T à $n + 1$ sommets.
 - Étape 1 : T possède un sommet de degré 1 (une « feuille »). En effet, partons d'un sommet quelconque et avançons d'arête en arête sans jamais revenir sur nos pas. Comme T est sans cycle, on ne peut jamais repasser par un sommet déjà visité ; comme T est fini, la promenade doit s'arrêter : elle s'arrête en un sommet s de degré 1.
 - Étape 2 : on retire la feuille. Supprimons s et l'unique arête qui le touche. Le graphe T' obtenu est encore connexe (les autres sommets restent reliés entre eux comme avant) et sans cycle (on n'a rien ajouté) : c'est un arbre à n sommets.
 - Étape 3 : on applique l'hypothèse de récurrence. Par $P(n)$, T' possède $n - 1$ arêtes. Or T possède exactement une arête de plus que T' (celle qu'on a retirée). Donc T possède $(n - 1) + 1 = n = (n + 1) - 1$ arêtes : $P(n + 1)$ est vraie.
3. Conclusion. Par le principe de récurrence, tout arbre à n sommets possède exactement $n - 1$ arêtes.

PROPRIÉTÉ Unicité de la chaîne dans un arbre

Dans un arbre, deux sommets quelconques sont reliés par une unique chaîne (élémentaire).

EXPÉRIENCE Démonstration guidée par l'absurde

Soit T un arbre et u, v deux de ses sommets.

1. Existence. T est connexe, donc il existe au moins une chaîne reliant u à v .
2. Unicité, par l'absurde. Supposons qu'il existe deux chaînes élémentaires différentes C_1 et C_2 reliant u à v . En suivant C_1 de u vers v , il existe un premier endroit où les deux chaînes se séparent, puis un premier sommet où elles se rejoignent à nouveau. La portion de C_1 entre ces deux sommets, suivie de la portion de C_2 parcourue en sens inverse, forme alors un cycle dans T .
3. Contradiction. Or T est un arbre, donc sans cycle : contradiction. L'hypothèse de départ est fausse : la chaîne reliant u à v est unique.

À RETENIR Caractérisations d'un arbre à n sommets

Pour un graphe G à n sommets, les affirmations suivantes sont équivalentes :

- G est un arbre (connexe sans cycle) ;
- G est connexe et possède $n - 1$ arêtes ;
- G est sans cycle et possède $n - 1$ arêtes ;
- deux sommets quelconques de G sont reliés par une unique chaîne.

En pratique, pour vérifier qu'un graphe à n sommets est un arbre, il suffit de vérifier deux des trois ingrédients : connexité, absence de cycle, $n - 1$ arêtes.

3.3 Arbre couvrant d'un graphe connexe

Intuition. Revenons à notre réseau de pistes. Le réseau complet contient des boucles, donc des pistes redondantes. L'ingénieur veut déclasser certaines pistes pour économiser l'entretien, tout en garantissant que chaque village reste accessible depuis n'importe quel autre. Le réseau déclassé doit : garder tous les villages (on dit qu'il est couvrant), rester connexe, et ne plus contenir de boucle. C'est précisément un arbre couvrant.

DÉFINITION Arbre couvrant

Soit G un graphe connexe. Un **arbre couvrant** de G est un **sous-graphe couvrant** de G (c'est-à-dire un sous-graphe qui contient tous les sommets de G) qui est un **arbre**.

Autrement dit, un arbre couvrant s'obtient en supprimant des arêtes de G (jamais de sommets) jusqu'à éliminer tous les cycles sans jamais détruire la connexité. Si G a n sommets, tout arbre couvrant de G possède exactement $n - 1$ arêtes : c'est un excellent moyen de vérification de vos réponses au Bac.

PROPRIÉTÉ Existence d'un arbre couvrant — démonstration exigible

Tout graphe connexe admet (au moins) un arbre couvrant.

EXPÉRIENCE Démonstration constructive guidée

Soit G un graphe connexe. On construit un arbre couvrant par le procédé suivant :

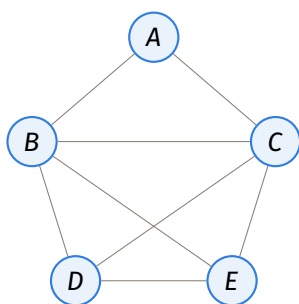
1. Initialisation. Posons $H = G$: H est connexe et contient tous les sommets de G .
2. Tant que H contient un cycle : choisissons un cycle de H et supprimons une arête de ce cycle.

3. Justification de la conservation de la connexité. Soit $\{x, y\}$ l'arête supprimée, appartenant à un cycle. Il existe alors dans le cycle un autre chemin reliant x à y (le « reste » du cycle). Toute chaîne de H qui utilisait l'arête $\{x, y\}$ peut être réécrite en empruntant ce détour : deux sommets reliés avant la suppression le sont encore après. Donc H reste connexe, et il contient toujours tous les sommets de G .
4. Terminaison. Chaque suppression enlève une arête ; le nombre d'arêtes est fini, donc le procédé s'arrête. À la fin, H ne contient plus de cycle.
5. Conclusion. Le graphe final H est couvrant, connexe et sans cycle : c'est un arbre couvrant de G .

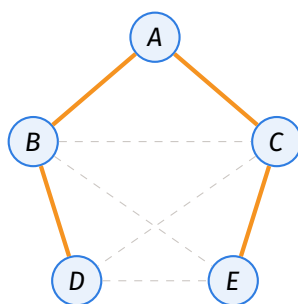
Cette démonstration est constructive : elle fournit directement une méthode pratique d'extraction d'un arbre couvrant, que nous utiliserons dans les exemples.

ATTENTION Deux pièges classiques

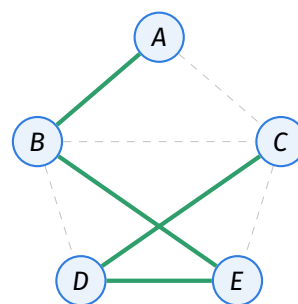
- Un arbre couvrant n'est pas unique en général. Dès que le graphe contient un cycle, il existe plusieurs arbres couvrants distincts (on peut choisir quelle arête du cycle supprimer). Ne dis jamais « l'arbre couvrant » mais « un arbre couvrant ».
- Ne pas confondre « sous-graphe couvrant » et « arbre couvrant ». Un sous-graphe couvrant contient tous les sommets, mais peut encore contenir des cycles : ce n'est pas forcément un arbre. Pour être un arbre couvrant, il faut en plus la connexité et l'absence de cycle — soit, en pratique, $n - 1$ arêtes et la connexité.



Graphe connexe G



Arbre couvrant T_1



Arbre couvrant T_2

Un graphe connexe G à 5 sommets et 8 arêtes, et deux arbres couvrants différents T_1 et T_2 (arêtes retenues en gras, arêtes supprimées en pointillés). Chacun possède exactement $5 - 1 = 4$ arêtes.

3.4 Graphes pondérés et poids d'un sous-graphe

Intuition. Dans la réalité, toutes les arêtes ne se valent pas : poser un câble entre Douala et Yaoundé ne coûte pas le même prix qu'entre deux quartiers voisins. Pour traiter ces problèmes d'optimisation, on attache à chaque arête un nombre : un coût, une distance, une durée. Le graphe devient pondéré.

DÉFINITION Graphe pondéré, poids d'un sous-graphe

Un **graphe pondéré** est un graphe dont chaque arête e est munie d'un nombre réel positif $w(e)$, appelé **poids** de l'arête (coût, longueur, durée...).

Le **poids d'un sous-graphe** H est la somme des poids de ses arêtes :

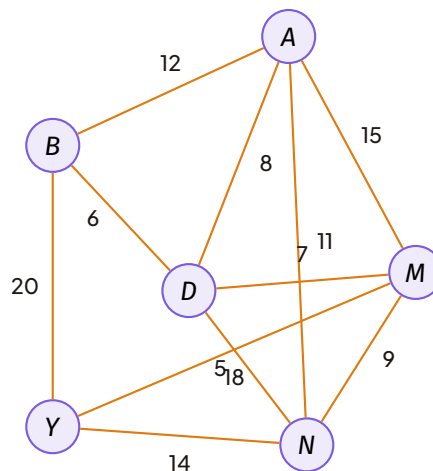
$$w(H) = \sum_{e \in H} w(e).$$

EXEMPLE Fil rouge : le réseau de câblage des six villes

Une entreprise de télécommunications veut interconnecter six villes : Abong-Mbang (A), Bertoua (B), Doumé (D), Mbandjock (M), Nanga-Eboko (N) et Yokadouma (Y). Le tableau ci-dessous donne le coût estimé de chaque liaison directe possible, en millions de FCFA.

Liaison	AB	AD	AM	BD	BY	DM
Coût (millions FCFA)	12	8	15	6	20	7
Liaison	DN	MN	NY	AN	MY	
Coût (millions FCFA)	5	9	14	11	18	

Ce graphe pondéré nous servira de fil rouge : nous y chercherons un arbre couvrant, puis, dans les sections suivantes, un arbre couvrant de poids minimal.



Le graphe pondéré fil rouge : six villes, coûts des liaisons en millions de FCFA.

3.5 Le problème de l'arbre couvrant de poids minimal

L'entreprise de notre fil rouge ne veut pas câbler toutes les liaisons possibles : ce serait ruineux. Elle cherche un réseau qui :

- relie toutes les villes (couvrant) ;
- ne contient aucune boucle inutile (arbre) ;
- coûte le moins cher possible (poids minimal).

DÉFINITION Arbre couvrant de poids minimal

Soit G un graphe connexe pondéré. Un **arbre couvrant de poids minimal** (en abrégé **ACM**, en anglais Minimum Spanning Tree) est un arbre couvrant de G dont le poids total est minimal parmi tous les arbres couvrants de G .

ATTENTION Minimal ne veut pas dire unique

Comme les arbres couvrants, un ACM n'est pas forcément unique : deux arbres couvrants différents peuvent avoir le même poids minimal (par exemple si deux arêtes ont des poids égaux). En revanche, la valeur du poids minimal, elle, est unique.

Avant d'apprendre les algorithmes de Kruskal et de Prim (section 5), entraînons-nous à extraire manuellement un arbre couvrant et à calculer son poids, en appliquant la méthode constructive de la démonstration : repérer un cycle, supprimer une de ses arêtes, recommencer.

EXERCICE RÉSOLU 3 Extraction manuelle d'un arbre couvrant sur le réseau des six villes

Sur le graphe fil rouge (six villes, 11 arêtes), extraire un arbre couvrant par suppression d'arêtes de cycles, vérifier le nombre d'arêtes, puis calculer le poids total de l'arbre obtenu.

SOLUTION

Solution détaillée.

Étape 1 : nombre d'arêtes attendu. Le graphe a $n = 6$ sommets. Tout arbre couvrant possédera donc exactement $6 - 1 = 5$ arêtes. Il faudra supprimer $11 - 5 = 6$ arêtes.

Étape 2 : suppression de cycles. Appliquons la méthode constructive en supprimant à chaque fois une arête d'un cycle (on choisit ici, par stratégie économique, la plus coûteuse du cycle repéré) :

1. Cycle $B - Y - N - D - B$ (arêtes BY, YN, ND, DB) : on supprime BY (20).
2. Cycle $A - M - Y - N - A$: on supprime MY (18).
3. Cycle $A - B - D - A$: on supprime AB (12).
4. Cycle $A - M - N - A$: on supprime AM (15).
5. Cycle $A - D - N - A$: on supprime AN (11).
6. Cycle $D - M - N - D$: on supprime MN (9).

Étape 3 : arêtes restantes. Il reste les 5 arêtes AD (8), BD (6), DM (7), DN (5) et NY (14). Vérification : $5 = n - 1$ arêtes, et le graphe obtenu est connexe (toutes les villes sont reliées via D) et sans cycle : c'est bien un arbre couvrant.

Étape 4 : poids total.

$$w(T) = 8 + 6 + 7 + 5 + 14 = 40.$$

Cet arbre couvrant coûte 40 millions de FCFA. Nous verrons en section 5, avec les algorithmes de Kruskal et de Prim, que ce poids de 40 millions est optimal : il s'agit bien d'un arbre couvrant de poids minimal (ACM).

MÉTHODE Extraire et vérifier un arbre couvrant

1. Compter les sommets : l'arbre couvrant devra avoir exactement $n - 1$ arêtes.
2. Tant qu'il reste un cycle, supprimer une arête de ce cycle (la connexité est préservée).
3. S'arrêter dès qu'il n'y a plus de cycle — ou dès qu'on atteint $n - 1$ arêtes.
4. Vérifier : tous les sommets sont présents, le graphe est connexe, il y a $n - 1$ arêtes.
5. Si le graphe est pondéré, calculer le poids total en sommant les poids des arêtes retenues.

EXERCICE 1 À toi de jouer

Sur le graphe fil rouge des six villes :

1. Extraire un arbre couvrant différent de celui de l'exercice résolu, en commençant par supprimer d'autres arêtes.
2. Calculer son poids total et comparer avec les 40 millions obtenus précédemment.
3. Peut-on trouver un arbre couvrant de poids inférieur à 40 millions ? (Tu pourras vérifier ta réponse avec l'algorithme de Kruskal dans la section 5.)

CORRIGÉ Exercice : extraction d'un second arbre couvrant

1. Par exemple, supprimons successivement : *BY* (cycle *BYNDB*), *AM* (cycle *AMDA*), *MY* (cycle *MYNM* via *MN*), *AN* (cycle *ANDA*), *MN* (cycle *MNDM*), *DM* (cycle restant éventuel *DM*... — vérifions). Arêtes restantes : *AB* (12), *AD* (8), *BD* (6), *DN* (5), *NY* (14). C'est bien 5 arêtes, le graphe est connexe (chaque ville est reliée à *D* ou via *D* : *A* et *B* sont reliés à *D*, *N* à *D*, *Y* à *N*) et sans cycle : c'est un arbre couvrant, différent du précédent puisqu'il contient *AB* et non *DM*.
2. Son poids : $w = 12 + 8 + 6 + 5 + 14 = 45$ millions de FCFA, soit 5 millions de plus que l'arbre de l'exercice résolu. Le choix des arêtes supprimées a donc une importance économique réelle.
3. Non. Le sommet *Y* (Yokadouma) n'est adjacent qu'aux arêtes *BY* (20), *MY* (18) et *NY* (14). Tout arbre couvrant doit inclure au moins une de ces trois arêtes pour connecter *Y* ; la moins coûteuse est *NY* = 14. Les cinq autres sommets $\{A, B, D, M, N\}$ forment un sous-graphe dont l'ACM (par Kruskal : *DN* = 5, *BD* = 6, *DM* = 7, *AD* = 8) pèse 26. Tout arbre couvrant du graphe complet pèse donc au minimum $26 + 14 = 40$ millions. L'arbre de l'exercice résolu atteint cette borne : c'est un ACM.

LE SAVAIS-TU ? Des arbres partout autour de nous

La théorie des arbres couvrants est née de problèmes très concrets : en 1926, le mathématicien tchèque Otakar Borůvka a inventé le premier algorithme d'arbre couvrant minimal pour optimiser... l'électrification rurale de la Moravie, un problème presque identique à celui de l'électrification des villages de nos régions ! Aujourd'hui, ces algorithmes tournent en permanence : routage des paquets sur Internet, conception des réseaux électriques de la SONATREL, planification des fibres optiques, et même analyse des similarités d'ADN en biologie. Un arbre à n sommets et ses $n-1$ arêtes, c'est la manière la plus économe du monde de connecter n points.

À RETENIR L'essentiel de la section

- Un **arbre** est un graphe connexe sans cycle ; à n sommets, il possède exactement $n - 1$ arêtes, et deux sommets y sont reliés par une unique chaîne.
- Un **arbre couvrant** d'un graphe connexe G est un sous-graphe contenant tous les sommets de G et qui est un arbre ; il n'est pas unique en général.
- Tout graphe connexe admet un arbre couvrant : on l'obtient en supprimant une arête d'un cycle tant qu'il reste des cycles (démonstration constructive exigible).
- Dans un **graphe pondéré**, le poids d'un sous-graphe est la somme des poids de ses arêtes ; un **ACM** est un arbre couvrant de poids total minimal.
- Réflexe de vérification : un arbre couvrant d'un graphe à n sommets a toujours $n - 1$ arêtes.

4 PARCOURS EN LARGEUR (BFS)

Dans la section précédente, nous avons découvert les arbres et les arbres couvrants : dans un graphe connexe, on peut toujours extraire un « squelette » minimal qui relie tous les sommets. Mais une question pratique demeure : comment explorer méthodiquement un graphe, sans oublier aucun sommet et sans tourner en rond ? Comment, par exemple, un opérateur comme CAMTEL peut-il vérifier que tous les relais d'un réseau sont bien accessibles depuis le central de Douala ? La réponse tient en trois lettres : **BFS**, pour Breadth-First Search, c'est-à-dire **parcours en largeur**. C'est l'algorithme d'exploration le plus naturel qui soit, et il va nous offrir gratuitement deux cadeaux : un test de connexité et un arbre couvrant.

4.1 L'idée : explorer par couches, comme une tache d'encre

Imagine qu'une goutte d'encre tombe sur une carte routière au niveau de Yaoundé. La tache s'étale d'abord sur les villes directement reliées à Yaoundé (distance 1), puis sur les villes reliées à celles-ci (distance 2), et ainsi de suite. Le parcours en largeur reproduit exactement ce phénomène : on visite les sommets **par couches de distance croissante** à partir d'un sommet **source**.

DÉFINITION Parcours en largeur

Soit G un graphe (orienté ou non) et s un sommet de G , appelé **source**. Le **parcours en largeur** depuis s est une visite des sommets de G qui respecte la règle suivante : un sommet situé à distance k de s (en nombre d'arêtes) est toujours visité avant tout sommet situé à distance $k + 1$. On dit que l'on explore le graphe couche par couche.

Pour mettre en œuvre cette idée, l'outil mathématique adapté est la **file** (structure FIFO : First

In, First Out, « premier entré, premier sorti »). C'est exactement la file d'attente devant un guichet d'agence MTN : le premier client arrivé est le premier servi, et les nouveaux arrivants se placent à la fin.

ATTENTION File et non pile

Dans le BFS, on utilise une file (FIFO), jamais une pile. Si l'on remplaçait la file par une pile (dernier entré, premier sorti), on obtiendrait un tout autre parcours, le parcours en profondeur, qui ne respecte pas les distances. Retiens : largeur = file ; profondeur = pile.

4.2 Description de l'algorithme

MÉTHODE Algorithme du parcours en largeur (BFS)

Entrée : un graphe G et un sommet source s .

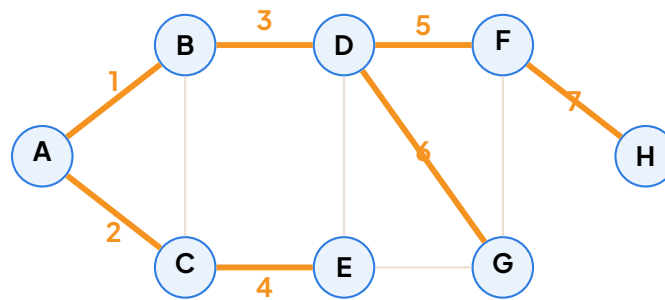
1. Initialisation. Marquer s et l'enfiler dans une file F initialement vide. Tous les autres sommets sont non marqués.
2. Boucle. Tant que la file F n'est pas vide :
 - (a) défiler le sommet u en tête de file (c'est le sommet que l'on traite) ;
 - (b) pour chaque voisin v de u non marqué : marquer v et l'enfiler (et, si l'on construit l'arbre BFS, retenir l'arête $\{u, v\}$).
3. Fin. L'ordre dans lequel les sommets ont été défilés est l'**ordre de visite** du BFS.

ATTENTION Le marquage se fait à l'enfilement

C'est l'erreur classique du Bac : un sommet doit être marqué au moment où on l'enfile, et non au moment où on le défile. Sinon, un même sommet peut être enfilé plusieurs fois par deux voisins différents, ce qui fausse tout le parcours. Avant d'enfiler un voisin, vérifie toujours : « est-il déjà marqué ? » Si oui, on l'ignore.

4.3 Exécution pas à pas sur un exemple

Considérons le graphe à 8 sommets ci-dessous, et lançons le BFS depuis la source A. Les voisins sont toujours examinés par ordre alphabétique.



Graphe à 8 sommets. En gras orange : l'arbre BFS depuis A (arêtes de première visite); les numéros indiquent l'ordre de découverte des sommets (A est l'étape 0).

MÉTHODE Rédiger le tableau de suivi BFS exigé au Bac

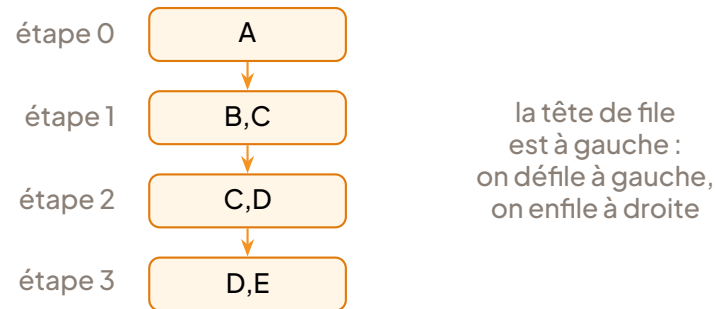
Pour obtenir tous les points, présente un tableau à quatre colonnes : étape, sommet traité (défilé), contenu de la file (après enfilements, la tête à gauche), sommets marqués. À chaque étape : on défile la tête, on enfile ses voisins non marqués dans l'ordre alphabétique, en les marquant immédiatement.

EXEMPLE BFS depuis A, tableau de suivi

Étape	Traité	File (tête à gauche)	Marqués
0	—	A	A
1	A	B, C	A, B, C
2	B	C, D	A, B, C, D
3	C	D, E	A, B, C, D, E
4	D	E, F, G	A, B, C, D, E, F, G
5	E	F, G	A, B, C, D, E, F, G
6	F	G, H	tous
7	G	H	tous
8	H	(vide)	tous

Ordre de visite : A, B, C, D, E, F, G, H. Remarque à l'étape 4 : en traitant D, on enfile F et G ; E est voisin de D mais déjà marqué, donc ignoré. À l'étape 5, en traitant E, son voisin G est déjà marqué : on n'enfile rien.

Le schéma suivant visualise l'évolution de la file FIFO sur les quatre premières étapes.



Évolution de la file FIFO : à chaque étape, la tête sort et les nouveaux voisins entrent par la droite.

4.4 Propriété fondamentale : le BFS donne les distances

PROPRIÉTÉ Distances et arbre BFS (admise)

Soit G un graphe et s un sommet source.

1. Le BFS depuis s visite les sommets par distances croissantes à s : la couche k contient exactement les sommets dont la distance à s (nombre minimal d'arêtes) vaut k .
2. L'ensemble des arêtes de première visite (celles par lesquelles chaque sommet a été marqué) forme un arbre, appelé **arbre BFS**, qui est un arbre couvrant de la composante connexe de s .
3. Les sommets visités sont exactement les sommets atteignables depuis s .

Cette propriété est admise ; son utilisation (lecture des distances sur le tableau de suivi) est exigible.

À RETENIR Lire les distances sur le BFS

La distance de la source s à un sommet v est égale au nombre d'arêtes du chemin de s à v dans l'arbre BFS. Sur notre exemple : $d(A,B) = d(A,C) = 1$; $d(A,D) = d(A,E) = 2$; $d(A,F) = d(A,G) = 3$; $d(A,H) = 4$.

4.5 Application 1 : tester la connexité

PROPRIÉTÉ Test de connexité par BFS

Un graphe est **connexe** si et seulement si le BFS lancé depuis un sommet quelconque vi-

site tous les sommets. Si certains sommets restent non marqués à la fin, le graphe n'est pas connexe, et les sommets visités forment la composante connexe de la source.

C'est l'outil de diagnostic du gestionnaire de réseau : si, après exploration depuis le central, certains relais restent non marqués, c'est qu'ils sont coupés du réseau.

4.6 Application 2 : construire un arbre couvrant

Chaque sommet (sauf la source) est marqué une unique fois, grâce à une unique arête de « première visite ». Ces $n - 1$ arêtes (pour une composante à n sommets) forment donc un arbre couvrant : c'est l'**arbre BFS**, tracé en gras sur notre figure. C'est une méthode constructive qui illustre la propriété du cours : tout graphe connexe contient un arbre couvrant.

4.7 Cas des graphes orientés

Sur un graphe **orienté**, l'algorithme est identique, à une nuance près : les « voisins » de u sont les **successeurs** de u , c'est-à-dire les sommets v tels qu'il existe un arc (u, v) . Le BFS visite alors exactement les sommets **atteignables depuis s par un chemin orienté**, et les distances lues sont des distances orientées (en nombre d'arcs). Un sommet peut très bien ne jamais être visité, même si le graphe sous-jacent est connexe, simplement parce qu'aucun chemin orienté ne mène de s à lui.

4.8 Exemple chiffré complet : le réseau des six villes

EXERCICE RÉSOLU 4 BFS depuis Garoua

Six villes du Nord sont reliées par des pistes : Garoua (G), Figuil (F), Guider (U), Pitoa (P), Touboro (T) et Poli (L). Les liaisons sont : $G-F$, $G-U$, $G-P$, $F-T$, $U-P$, $P-L$, $T-L$. Lancer le BFS depuis Garoua, donner les distances et un arbre couvrant.

SOLUTION

On suit l'ordre alphabétique des voisins.

Étape	Traité	File	Marqués
0	—	G	G
1	G	F, P, U	G, F, P, U
2	F	P, U, T	G, F, P, U, T
3	P	U, T, L	tous
4	U	T, L	tous
5	T	L	tous
6	L	(vide)	tous

Ordre de visite : G, F, P, U, T, L . Les six villes sont visitées : le réseau est connexe. Arêtes de

première visite (arbre BFS) : $G-F$, $G-P$, $G-U$, $F-T$, $P-L$, soit $6 - 1 = 5$ arêtes. Distances depuis Garoua : $d(G,F) = d(G,P) = d(G,U) = 1$ et $d(G,T) = d(G,L) = 2$. Interprétation : depuis Garoua, Figuil, Pitoa et Guider sont accessibles en une seule piste ; Touboro et Poli en deux au minimum.

EXERCICE 2 À toi de jouer

Sur le graphe de la figure à 8 sommets (premier exemple), exécuter le BFS depuis la source H . Donner le tableau de suivi, l'ordre de visite, les distances depuis H et l'arbre BFS obtenu. Le graphe est-il connexe ?

CORRIGÉ Exercice 1

Voisins examinés par ordre alphabétique.

Étape	Traité	File (tête à gauche)	Marqués
0	—	H	H
1	H	F, G	H, F, G
2	F	G, D	H, F, G, D
3	G	D, E	H, F, G, D, E
4	D	E, B	H, F, G, D, E, B
5	E	B, C	H, F, G, D, E, B, C
6	B	C, A	tous
7	C	A	tous
8	A	(vide)	tous

Ordre de visite : H, F, G, D, E, B, C, A . Tous les sommets sont visités : le graphe est connexe. Distances depuis H : $d(H,F) = d(H,G) = 1$; $d(H,D) = d(H,E) = 2$; $d(H,B) = d(H,C) = 3$; $d(H,A) = 4$. Arbre BFS (arêtes de première visite) : $H-F$, $H-G$, $F-D$, $G-E$, $D-B$, $E-C$, $B-A$.

LESAVAIS-TU ? Le BFS dans ta poche

Quand un réseau social te suggère des « amis d'amis », ou qu'un moteur de recherche explore le web page par page, c'est très souvent un parcours en largeur qui travaille en coulisses. L'algorithme a été formalisé dans les années 1950–1960 (par Moore pour le routage dans les réseaux de commutation, puis par Lee pour le câblage de circuits imprimés), en même temps que naissaient les premiers réseaux informatiques — les ancêtres des ré-

seaux MTN et Orange qui couvrent aujourd'hui le Cameroun. Sa complexité en temps est $O(|V| + |E|)$ avec des listes d'adjacence, ce qui le rend utilisable sur des graphes massifs.

Le BFS répond à la question « quels sommets sont atteignables, et en combien d'arêtes ? ». Mais dans la réalité, toutes les arêtes ne se valent pas : une piste de 120 km ne coûte pas comme une route de 40 km. Il nous faut donc des algorithmes qui tiennent compte des poids : ce sera l'objet des sections suivantes, avec Kruskal, Prim et Dijkstra.

5 ARBRE COUVRANT DE POIDS MINIMAL : KRUSKAL ET PRIM

Dans la section précédente, le parcours en largeur (BFS) nous a appris à explorer un graphe connexe depuis un sommet, et à en extraire un arbre couvrant. Mais cet arbre, nous l'avons construit sans nous soucier d'aucun coût : toutes les arêtes étaient « égales ». Or, dans la réalité, relier deux villes par une fibre optique, un oléoduc ou une ligne haute tension ne coûte pas le même prix selon la distance ou le relief. La question naturelle devient alors : parmi tous les arbres couvrants possibles, lequel coûte le moins cher ? C'est tout l'objet de cette section, l'une des plus concrètes et des plus fréquentes au Baccalauréat.

5.1 Position du problème

EXPÉRIENCE

Une situation concrète : la fibre optique dans le Grand Nord

La société CAMTEL souhaite interconnecter par la fibre optique six villes du Grand Nord camerounais : Garoua (G), Guider (U), Maroua (M), Kousséri (K), Ngaoundéré (N) et Poli (P). Les études de terrain ont donné le coût de pose (en millions de FCFA) de chaque tronçon directement réalisable :

Tronçon	G-U	G-N	G-P	U-M	U-P
Coût (millions FCFA)	4	7	5	3	6
Tronçon	M-K	M-P	N-P	K-N	
Coût (millions FCFA)	2	8	5	9	

L'objectif : relier les six villes (chacune doit pouvoir communiquer avec toutes les autres) au moindre coût total, sans payer de tronçon inutile.

Analysons la situation avec nos outils. Relier les six villes sans redondance, c'est exactement construire un **arbre couvrant** du graphe : connexe (toutes les villes communiquent) et sans cycle (un cycle signifierait un tronçon payé pour rien, puisque ses villes seraient déjà reliées par ailleurs). Un arbre couvrant d'un graphe à 6 sommets possède $6 - 1 = 5$ arêtes. Le problème se reformule ainsi :

DÉFINITION Arbre couvrant de poids minimal

Soit G un graphe connexe **pondéré**. Le **poids** d'un sous-graphe est la somme des poids de ses arêtes. Un **arbre couvrant de poids minimal** (on dit aussi arbre couvrant minimum, ou ACM) est un arbre couvrant de G dont le poids est le plus petit possible parmi tous les arbres couvrants de G .

ATTENTION Pourquoi ne pas énumérer tous les arbres couvrants ?

Une idée naïve serait de lister tous les arbres couvrants et de comparer leurs poids. C'est impossible en pratique : un graphe complet à n sommets possède n^{n-2} arbres couvrants (formule de Cayley). Pour seulement $n = 10$ sommets, cela fait déjà $10^8 = 100$ millions d'arbres ! Il nous faut donc une méthode intelligente : les algorithmes de Kruskal et de Prim.

5.2 L'algorithme de Kruskal

L'intuition de Kruskal est d'une simplicité désarmante : prenez d'abord les tronçons les moins chers, tant qu'ils servent vraiment à relier de nouvelles villes. Un tronçon « sert » s'il relie deux villes qui n'étaient pas déjà connectées entre elles ; sinon, il créerait un cycle, donc une redondance payée pour rien.

MÉTHODE Algorithme de Kruskal

Soit G un graphe pondéré connexe à n sommets.

1. Trier toutes les arêtes de G par poids **croissant**.
2. Initialiser : l'arbre T est vide (aucune arête retenue).
3. Parcourir les arêtes dans l'ordre croissant : pour chaque arête, l'ajouter à T si et seulement si elle ne crée pas de cycle avec les arêtes déjà retenues ; sinon, la rejeter.
4. S'arrêter dès que T contient $n-1$ arêtes. T est alors un arbre couvrant de poids minimal.

La difficulté pratique est l'étape 3 : comment vérifier, sur une copie d'examen, qu'une arête ne crée pas de cycle ? Voici la méthode opérationnelle recommandée.

MÉTHODE Détection des cycles : la numérotation des composantes

Attribue à chaque sommet un **numéro de composante** (au départ, chaque sommet est seul : numéros 1, 2, 3, ...). Pour chaque arête examinée xy :

- si x et y ont des numéros différents, l'arête est retenue : on unifie alors les deux com-

posantes en donnant à tous les sommets du plus grand numéro le plus petit numéro ;

- si x et y ont le même numéro, ils sont déjà reliés : l'arête créerait un cycle, elle est rejetée.

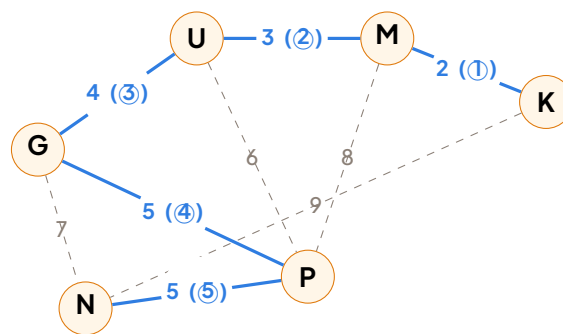
EXEMPLE Exécution de Kruskal sur le graphe fil rouge

Arêtes triées par coût croissant : MK (2), UM (3), GU (4), GP (5), NP (5), UP (6), GN (7), MP (8), KN (9).

Arête	Poids	Décision (composantes)	Issue
MK	2	M et K de composantes différentes	Retenue
UM	3	U et M différentes	Retenue
GU	4	G et U différentes	Retenue
GP	5	G et P différentes	Retenue
NP	5	N et P différentes	Retenue
Arrêt : on a $5 = 6 - 1$ arêtes retenues.			

Poids total : $2 + 3 + 4 + 5 + 5 = 19$ millions de FCFA.

Remarque : Les arêtes restantes (UP , GN , MP , KN) n'ont pas été examinées car l'algorithme s'est arrêté. Si on les examinait, elles seraient toutes rejetées car leurs extrémités appartiennent déjà à la même composante (elles créeraient des cycles).



Arbre couvrant de poids minimal obtenu par Kruskal (arêtes bleues épaisses, numérotées dans l'ordre de sélection). Les arêtes grisées en pointillés n'ont pas été examinées (arrêt à $n - 1$ arêtes); elles auraient été rejetées car créant des cycles. Poids total : 19 millions de FCFA.

ATTENTION Erreur fréquente n°1 : oublier le test de cycle

Dans Kruskal, on ne prend pas systématiquement les $n - 1$ arêtes les moins chères ! Ici par exemple, si l'arête UP de poids 6 avait été nécessaire avant la fin, il aurait fallu vérifier qu'elle ne crée pas de cycle. Une arête dont les deux extrémités sont déjà reliées par les arêtes retenues doit être rejetée, même si son poids est faible. Sur ta copie, justifie chaque rejet par la mention explicite « créerait le cycle ... ».

5.3 L'algorithme de Prim

L'algorithme de Prim adopte une stratégie différente : au lieu de regarder toutes les arêtes globalement, on fait grandir un arbre depuis un sommet de départ, comme une tache d'huile qui s'étend de ville en ville en choisissant toujours le raccordement le moins cher.

MÉTHODE Algorithme de Prim

Soit G un graphe pondéré connexe à n sommets.

1. Choisir un sommet de départ quelconque s . L'ensemble A des sommets atteints est $\{s\}$.
2. Répéter tant que A ne contient pas tous les sommets : parmi toutes les arêtes reliant un sommet de A à un sommet hors de A , choisir celle de **poids minimal** ; l'ajouter à l'arbre et ajouter son extrémité à A .
3. S'arrêter après $n - 1$ arêtes ajoutées : l'arbre obtenu est couvrant et de poids minimal.

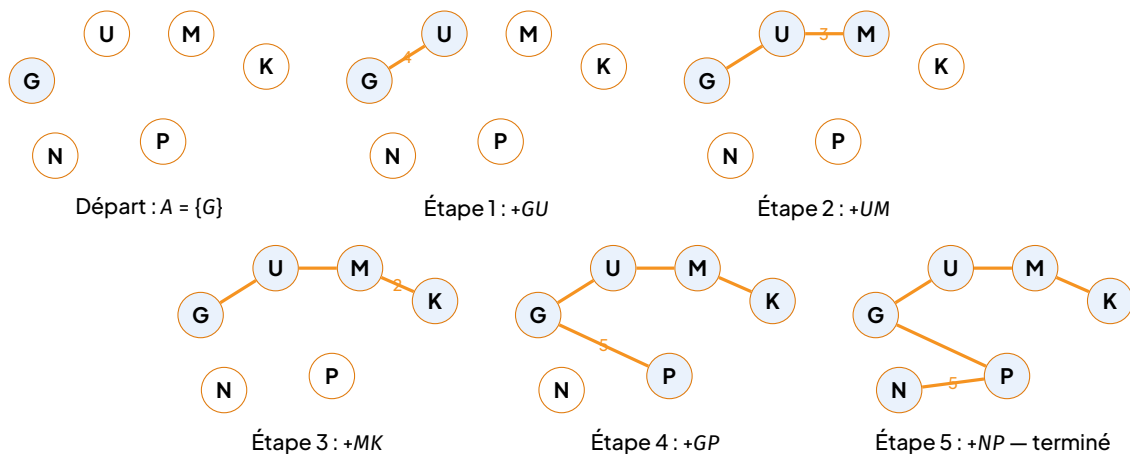
En cas d'égalité entre plusieurs arêtes de poids minimal, on choisit librement (cela peut donner des arbres différents, mais de même poids).

EXEMPLE Exécution de Prim depuis Garoua (G)

Partons de $A = \{G\}$.

- Étape 1 : arêtes sortant de $\{G\}$: $GU(4)$, $GN(7)$, $GP(5)$. Minimum : $GU(4)$. $A = \{G, U\}$.
- Étape 2 : arêtes de A vers l'extérieur : $GN(7)$, $GP(5)$, $UM(3)$, $UP(6)$. Minimum : $UM(3)$. $A = \{G, U, M\}$.
- Étape 3 : candidates : $GN(7)$, $GP(5)$, $UP(6)$, $MK(2)$, $MP(8)$. Minimum : $MK(2)$. $A = \{G, U, M, K\}$.
- Étape 4 : candidates : $GN(7)$, $GP(5)$, $UP(6)$, $MP(8)$, $KN(9)$. Minimum : $GP(5)$. $A = \{G, U, M, K, P\}$.
- Étape 5 : candidates vers N : $GN(7)$, $NP(5)$, $KN(9)$. Minimum : $NP(5)$. $A = \{G, U, M, K, P, N\}$: tous les sommets sont atteints.

Arêtes retenues : GU, UM, MK, GP, NP — poids total : $4+3+2+5+5 = 19$ millions de FCFA. C'est le même arbre que celui de Kruskal (ce n'est pas toujours le cas, mais le poids minimal, lui, est toujours le même).



Croissance de l'arbre de Prim depuis Garoua. Les sommets ombrés (bleus) sont déjà atteints ; à chaque étape, on ajoute l'arête la moins chère reliant un sommet atteint à un sommet non atteint.

ATTENTION Erreur fréquente n°2 : dans Prim, relier deux sommets déjà atteints

À chaque étape de Prim, l'arête choisie doit relier un sommet atteint à un sommet non atteint. Il est interdit de choisir une arête entre deux sommets déjà dans A , même si elle est très bon marché : elle créerait un cycle. Par exemple, après l'étape 3, l'arête UP (6) est candidate, mais une fois P atteint, UP devient définitivement inutilisable. Autre piège : croire que le sommet de départ influence le coût final — il n'influence que l'ordre de construction, pas le poids minimal.

5.4 Optimalité des algorithmes : idée de la preuve

Pourquoi des choix aussi « myopes » (toujours prendre le moins cher disponible) donnent-ils l'optimum global ? Voici l'idée, dite par échange.

EXPÉRIENCE Idée de la preuve par échange (non exigible)

Supposons que Kruskal retienne une arête e de poids p , et qu'il existe un arbre couvrant T^* de poids minimal ne contenant pas e . Ajoutons e à T^* : comme T^* est un arbre, on crée exactement un cycle. Ce cycle contient une autre arête f qui, au moment où Kruskal

a choisi e , était encore disponible et reliait les mêmes composantes : donc $\text{poids}(f) \geq \text{poids}(e)$ (sinon Kruskal aurait examiné f avant e et l'aurait retenue). Remplaçons alors f par e dans T^* : on obtient un nouvel arbre couvrant de poids inférieur ou égal. En répétant ces échanges, on transforme T^* en l'arbre de Kruskal sans jamais augmenter le poids : l'arbre de Kruskal est donc bien de poids minimal. Le raisonnement est analogue pour Prim.

PROPRIÉTÉ Admise — Kruskal et Prim donnent l'optimum

Soit G un graphe pondéré connexe. Les algorithmes de Kruskal et de Prim fournissent chacun un **arbre couvrant de poids minimal** de G . De plus :

- le poids minimal est unique : tous les arbres couvrants minimaux ont le même poids ;
- en revanche, l'arbre minimal n'est pas nécessairement unique : en cas d'égalités de poids, Kruskal et Prim peuvent produire des arbres différents, mais toujours de même poids total.

Ce résultat est admis au niveau Terminale ; la démonstration n'est pas exigible au Baccalauréat.

LES AVAIS-TU ? Des algorithmes « gloutons » qui gagnent toujours... cette fois

Kruskal et Prim sont des **algorithmes gloutons** (greedy en anglais) : à chaque étape, ils font le choix localement optimal (l'arête la moins chère possible) sans jamais remettre en cause les choix passés. En général, une stratégie gloutonne ne garantit pas l'optimum global — par exemple, pour rendre la monnaie avec un nombre minimal de pièces, le choix glouton peut échouer sur certains systèmes de pièces. Le problème de l'arbre couvrant minimal fait partie des cas remarquables où l'avidité paie : la structure mathématique sous-jacente (un matroïde) garantit que l'optimum local conduit à l'optimum global. Kruskal publia son algorithme en 1956, Prim en 1957 (Prim redécouvrit en fait une méthode due au mathématicien tchèque Jarník dès 1930). Ces algorithmes tournent aujourd'hui dans la planification des réseaux d'électrification rurale, de câblage télécom et de conception de circuits imprimés.

5.5 Exercice résolu et comparaison des deux méthodes

EXERCICE RÉSOLU 5 Câblage minimal dans le Grand Nord

CAMTEL dispose d'un budget de 20 millions de FCFA pour interconnecter les six villes G, U, M, K, N, P avec les coûts du tableau du fil rouge. Le projet est-il bouclable dans l'enveloppe ? Quels tronçons faut-il réaliser ?

SOLUTION

Solution. On cherche un arbre couvrant de poids minimal. Appliquons Kruskal (ou Prim : même résultat). Les arêtes triées sont MK (2), UM (3), GU (4), GP (5), NP (5), UP (6), GN (7), MP (8), KN (9). On retient successivement MK , UM , GU , GP , NP (aucune ne crée de cycle, vérifié par numérotation des composantes), soit $5 = 6 - 1$ arêtes : arrêt.

$$\text{Coût total} = 2 + 3 + 4 + 5 + 5 = 19 \text{ millions de FCFA.}$$

Interprétation. Le coût minimal de câblage est de 19 millions de FCFA, inférieur au budget de 20 millions : le projet est bouclable, avec même 1 million de marge. Les tronçons à réaliser sont Maroua–Kousséri, Guider–Maroua, Garoua–Guider, Garoua–Poli et Ngaoundéré–Poli. Toute autre solution complète coûterait au moins autant, d'après la propriété d'optimalité.

MÉTHODE**Kruskal ou Prim : lequel choisir en pratique ? (complément)**

Les deux algorithmes donnent le même poids minimal ; le choix est une question de commodité :

- Kruskal est souvent plus rapide à la main quand le graphe a peu d'arêtes et que le tri est facile ; il est aussi très lisible sous forme de tableau (arête / poids / retenue-rejetée).
- Prim est plus commode quand le graphe est donné par un tableau de distances (matrice) ou quand il est dense (beaucoup d'arêtes) : on travaille localement depuis un sommet, sans trier toutes les arêtes. Le point de départ est arbitraire : choisis celui qui simplifie les comparaisons.

Au Baccalauréat, si l'énoncé n'impose pas la méthode, choisis celle avec laquelle tu es le plus sûr, et présente tes étapes dans un tableau : c'est la clarté qui est évaluée.

EXERCICE 3**À toi de jouer**

On reprend le graphe fil rouge, mais le coût du tronçon NP passe de 5 à 6 millions de FCFA (relief accidenté découvert après étude géotechnique).

1. Appliquer l'algorithme de Kruskal et donner le nouvel arbre couvrant minimal et son poids.
2. Reprendre Prim depuis Garoua : obtient-on le même arbre ? le même poids ?
3. Le budget de 20 millions est-il toujours suffisant ?

CORRIGÉ Exercice

1. Arêtes triées : $MK (2)$, $UM (3)$, $GU (4)$, $GP (5)$, $NP (6)$, $UP (6)$, $GN (7)$, $MP (8)$, $KN (9)$. On retient MK , UM , GU , GP . Composante commune : $\{G, U, M, K, P\}$; N est seul. L'arête $NP (6)$ relie deux composantes différentes : retenue. On a 5 arêtes : arrêt. Poids : $2 + 3 + 4 + 5 + 6 = 20$ millions. (L'arête $UP (6)$ relie deux sommets déjà dans la même composante, elle crée un cycle; NP est la seule arête retenue à cette étape car c'est la moins chère touchant N ... $GN (7)$ et $KN (9)$ relient aussi N , mais sont plus chères.)
2. Prim depuis G : $+GU (4)$, $+UM (3)$, $+MK (2)$, $+GP (5)$, puis vers N : $NP (6)$ contre $GN (7)$ et $KN (9)$: $+NP (6)$. Même arbre, même poids : 20 millions.
3. Le coût minimal est exactement 20 millions : le budget est juste suffisant, sans aucune marge.

À RETENIR L'essentiel de la section

- Un **arbre couvrant de poids minimal** d'un graphe pondéré connexe est un arbre couvrant dont la somme des poids des arêtes est la plus petite possible; il possède $n - 1$ arêtes pour n sommets.
- Kruskal : trier les arêtes par poids croissant, retenir chaque arête qui ne crée pas de cycle (vérifier par numérotation des composantes), s'arrêter à $n - 1$ arêtes.
- Prim : partir d'un sommet, ajouter à chaque étape l'arête de poids minimal reliant un sommet atteint à un sommet non atteint.
- Les deux algorithmes (résultat admis) donnent un arbre couvrant de poids minimal; le poids minimal est unique, l'arbre ne l'est pas forcément.
- Toujours conclure en interprétant le résultat dans le contexte : coût en FCFA, tronçons à construire, faisabilité budgétaire.

6**PLUS COURT CHEMIN : ALGORITHME DE DIJKSTRA**

Dans la section précédente, nous avons appris à relier tous les sommets d'un graphe pondéré au moindre coût grâce aux algorithmes de Kruskal et de Prim : l'arbre couvrant de poids minimal répond à la question « comment connecter tout le réseau le moins cher possible ? ». Mais un voyageur, lui, ne veut pas connecter tout le réseau : il veut aller d'un point à un autre le plus vite possible. Un chauffeur de transport en commun qui quitte Garoua pour Touboro se fiche de savoir comment électrifier toute la région : il veut son itinéraire optimal. C'est exactement l'objet de cette section, couronnée par l'un des algorithmes les plus célèbres des mathématiques appliquées : l'**algorithme de Dijkstra**.

6.1 Position du problème et vocabulaire

Du nombre d'arêtes au poids : deux notions de distance

Avec le parcours en largeur (BFS), nous avons mesuré la distance entre deux sommets en nombre d'arêtes : c'est le nombre minimal de « sauts » pour passer d'un sommet à l'autre. Mais dans la réalité, toutes les arêtes ne se valent pas : entre deux villes, il peut y avoir 30 km de bitume ou 120 km de piste dégradée. Le graphe doit donc être **pondéré**, et la bonne notion de distance devient le poids total du chemin.

DÉFINITION Poids d'un chemin, distance pondérée

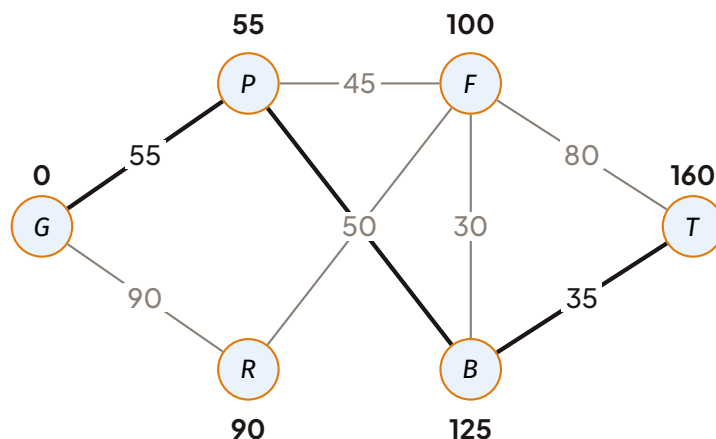
Soit G un graphe (orienté ou non) pondéré, dont chaque arête (ou arc) porte un poids réel. Le **poids d'un chemin** (ou d'une chaîne) est la somme des poids de ses arêtes. La **distance pondérée** entre deux sommets A et B , notée $d(A, B)$, est le poids minimal d'un chemin reliant A à B , s'il en existe un. Un chemin réalisant ce minimum est appelé **plus court chemin** de A à B .

ATTENTION Distance BFS $\neq$ distance pondérée

Le plus court chemin en nombre d'arêtes n'est pas forcément le plus court en poids. Si l'on peut aller de Garoua à Touboro en 2 étapes par une route de 250 km, ou en 3 étapes par des routes totalisant 160 km, le BFS préfère la première option, Dijkstra la seconde. Les deux algorithmes ne répondent pas à la même question.

6.2 Le fil rouge : le réseau routier autour de Garoua

Nous reprenons notre situation fil rouge. Un transporteur de la région du Nord dessert six localités : Garoua (G), Pitoa (P), Rey-Bouba (R), Figuil (F), Bibemi (B) et Touboro (T). Le graphe ci-dessous modélise les liaisons routières directes, pondérées par les distances en kilomètres.



Le réseau routier fil rouge. En rouge : le plus court chemin de Garoua à Touboro ($G \rightarrow P \rightarrow B \rightarrow T$, 160 km). En rouge à côté de chaque sommet : son étiquette définitive, c'est-à-dire sa distance minimale depuis Garoua.

La question est simple à énoncer : quel est l'itinéraire de Garoua à Touboro dont la distance totale est minimale ? On pourrait énumérer tous les chemins, mais dès que le graphe compte quelques dizaines de sommets, cette méthode devient inhumaine — et impensable pour un ordinateur de navigation qui traite des millions de carrefours. Il faut une méthode systématique : c'est Dijkstra.

6.3 L'idée de l'algorithme : des étiquettes qui deviennent définitives

L'intuition est lumineuse. Imagine une tache d'encre qui se diffuse sur le réseau à vitesse constante depuis Garoua, en progressant le long des routes. La tache atteint d'abord la ville la plus proche : à cet instant, on est certain de la distance minimale vers cette ville, car tout autre itinéraire devrait d'abord atteindre une ville déjà plus lointaine. On « fige » alors cette distance, on la déclare **définitive**, et la tache repart de cette nouvelle ville. De proche en proche, chaque ville reçoit une étiquette provisoire (la meilleure distance connue à ce stade) qui finit par devenir définitive.

MÉTHODE Algorithme de Dijkstra

Pour trouver le plus court chemin d'un sommet source S à un sommet arrivée A dans un graphe pondéré à poids positifs ou nuls :

1. Initialisation. Écrire l'étiquette 0 sur le sommet source S et l'étiquette $+\infty$ sur tous les autres sommets. Aucune étiquette n'est définitive.
2. Choix. Parmi les sommets dont l'étiquette est encore provisoire, choisir celui de plus petite étiquette, soit X . Son étiquette devient définitive (on la souligne, on la barre ou on la met en gras dans le tableau).
3. Mise à jour. Pour chaque voisin Y de X dont l'étiquette n'est pas définitive, calculer

étiquette(X) + $p(XY)$. Si cette somme est strictement inférieure à l'étiquette actuelle de Y , remplacer l'étiquette de Y par cette somme et noter X comme **prédécesseur** de Y : on écrit l'étiquette sous la forme « distance (prédécesseur) ».

4. Itération. Revenir à l'étape 2 tant que l'arrivée A n'a pas d'étiquette définitive (ou continuer jusqu'à ce que tous les sommets soient fixés pour obtenir toutes les distances depuis S).
5. Lecture. La distance minimale $d(S,A)$ est l'étiquette définitive de A ; le chemin optimal se reconstitue en remontant les prédécesseurs de A vers S .

PROPRIÉTÉ**Correction de l'algorithme de Dijkstra (admise)**

Dans un graphe dont tous les poids sont positifs ou nuls, à chaque étape de l'algorithme, l'étiquette du sommet que l'on fixe est égale à la distance pondérée minimale depuis la source. Par conséquent, lorsque l'arrivée est fixée, son étiquette est la longueur du plus court chemin, et la remontée des prédécesseurs fournit un plus court chemin.

Cette propriété est admise au programme de Terminale ; sa démonstration n'est pas exigible au Baccalauréat.

EXPÉRIENCE**Pourquoi l'étiquette fixée est-elle la bonne ? (justification intuitive)**

Raisonnons au moment où l'on fixe le sommet X , de plus petite étiquette provisoire ℓ . Supposons qu'il existe un chemin de S à X strictement plus court que ℓ . Ce chemin, partant de la zone déjà fixée, doit à un moment « sortir » par une arête joignant un sommet fixé U à un sommet non fixé V . Alors l'étiquette provisoire de V vaut au plus $d(S,U) + p(UV)$, quantité inférieure ou égale à la longueur totale de ce chemin (car les poids sont positifs), donc strictement inférieure à ℓ . Contradiction : X n'aurait pas la plus petite étiquette. L'hypothèse est donc absurde et $\ell = d(S,X)$. Observe où l'hypothèse « poids positifs » intervient : sans elle, un détour ultérieur pourrait « rembourser » du poids, et l'argument s'effondre.

ATTENTION**La condition « poids positifs ou nuls » est indispensable**

L'algorithme de Dijkstra est faux si le graphe contient un poids négatif. Contre-exemple (complément culturel) : trois sommets S, U, V avec $p(SU) = 2$, $p(SV) = 5$, $p(UV) = -4$. Dijkstra fixe U à 2 dès la première étape, alors que le chemin $S \rightarrow V \rightarrow U$ vaut $5 + (-4) = 1 < 2$. L'étiquette fixée trop tôt est définitivement fautive. En pratique (distances, durées, coûts), les poids sont naturellement positifs, donc Dijkstra s'applique ; pour les poids quelconques, il existe l'algorithme de Bellman-Ford (hors programme).

6.4 Exécution complète : plus court itinéraire Garoua – Touboro

Appliquons l'algorithme pas à pas sur notre fil rouge, avec la présentation en tableau exigée au Baccalauréat : une ligne par étape, une colonne par sommet, et dans chaque case l'étiquette sous la forme « distance (prédécesseur) ». L'étiquette fixée à chaque étape est écrite en gras.

Initialisation. G reçoit 0, tous les autres sommets $+\infty$.

Étape 1. Plus petite étiquette provisoire : $G(0)$. On fixe G . Voisins : $P(0+55 = 55)$ et $R(0+90 = 90)$. Étiquettes : $P : 55(G)$, $R : 90(G)$.

Étape 2. Plus petite étiquette provisoire : $P(55 < 90)$. On fixe P . Voisins non fixés : $F(55 + 45 = 100)$ et $B(55 + 70 = 125)$. Étiquettes : $F : 100(P)$, $B : 125(P)$.

Étape 3. Plus petite étiquette provisoire : $R(90 < 100 < 125)$. On fixe R . Voisin non fixé : F via $R : 90 + 50 = 140 > 100$: pas de mise à jour.

Étape 4. Plus petite étiquette provisoire : $F(100)$. On fixe F . Voisins non fixés : $B(100 + 30 = 130 > 125$: pas de mise à jour) et $T(100 + 80 = 180)$. Étiquette : $T : 180(F)$.

Étape 5. Plus petite étiquette provisoire : $B(125 < 180)$. On fixe B . Voisin non fixé : T via $B : 125 + 35 = 160 < 180$: mise à jour ! Étiquette : $T : 160(B)$.

Étape 6. On fixe $T(160)$: c'est l'arrivée, l'algorithme s'arrête.

Étape	G	P	R	F	B	T
Init.	0	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$
1 (fixe G)	0	55 (G)	90 (G)	$+\infty$	$+\infty$	$+\infty$
2 (fixe P)	0	55 (G)	90 (G)	100 (P)	125 (P)	$+\infty$
3 (fixe R)	0	55 (G)	90 (G)	100 (P)	125 (P)	$+\infty$
4 (fixe F)	0	55 (G)	90 (G)	100 (P)	125 (P)	180 (F)
5 (fixe B)	0	55 (G)	90 (G)	100 (P)	125 (P)	160 (B)
6 (fixe T)	0	55 (G)	90 (G)	100 (P)	125 (P)	160 (B)

MÉTHODE

Reconstituer le chemin optimal par remontée des prédécesseurs

1. Partir du sommet d'arrivée et lire son prédécesseur dans son étiquette définitive.
2. Remonter de prédécesseur en prédécesseur jusqu'à atteindre la source.
3. Écrire le chemin dans le sens source $\rightarrow$ arrivée (la remontée le donne à l'envers).
4. Vérifier : la somme des poids des arêtes du chemin doit égaler l'étiquette définitive de l'arrivée.

Ici : prédécesseur de T est B , prédécesseur de B est P , prédécesseur de P est G . Le chemin optimal est donc $G \rightarrow P \rightarrow B \rightarrow T$, et l'on vérifie : $55 + 70 + 35 = 160$.

EXEMPLE Interprétation concrète du résultat

L'itinéraire le plus court de Garoua à Touboro passe par Pitoa puis Bibemi : Garoua – Pitoa (55 km), Pitoa – Bibemi (70 km), Bibemi – Touboro (35 km), soit 160 km au total. Remarque le piège de l'intuition naïve : passer par Figuil semble « tout droit » sur la carte, mais coûte $55 + 45 + 80 = 180$ km, soit 20 km de plus. Si le carburant et l'usure reviennent à environ 350 FCFA par kilomètre, le choix de l'itinéraire optimal économise $20 \times 350 = 7\,000$ FCFA par voyage aller — une somme non négligeable pour un transporteur qui effectue ce trajet chaque semaine.

ATTENTION Les deux erreurs qui coûtent des points au Bac

- Fixer un sommet qui n'a pas la plus petite étiquette provisoire. À l'étape 3, on doit fixer $R(90)$ et non $F(100)$ ni $B(125)$. Choisir « au hasard » ou par ordre alphabétique invalide toute la suite : c'est la garantie de l'algorithme qui saute.
- Oublier le prédécesseur dans l'étiquette. Sans la mention « (prédécesseur) », impossible de reconstituer le chemin : tu n'as que la distance, pas l'itinéraire, et la question « donner le plus court chemin » reste sans réponse complète. Écris systématiquement chaque étiquette sous la forme « distance (prédécesseur) ».

EXERCICE RÉSOLU 6 Livraison de matériel scolaire

Une librairie de Yaoundé doit livrer des manuels à un lycée d'Ebolowa. Le réseau simplifié est le suivant (distances en km) : Yaoundé (Y) – Mbalmayo (M) : 45 ; Yaoundé – Obala (O) : 50 ; Mbalmayo – Ebolowa (E) : 115 ; Obala – Ebolowa (E) : 140 ; Mbalmayo – Obala : 60. Déterminer l'itinéraire le plus court de Yaoundé à Ebolowa.

SOLUTION

Solution. On applique Dijkstra depuis Y .

- Init. : $Y : 0, M, O, E : +\infty$.
- Fixe $Y(0)$: $M : 45(Y), O : 50(Y)$.
- Fixe $M(45)$: $E : 45 + 115 = 160(M)$; O via $M : 45 + 60 = 105 > 50$, pas de mise à jour.
- Fixe $O(50)$: E via $O : 50 + 140 = 190 > 160$, pas de mise à jour.
- Fixe $E(160)$: arrivée atteinte.

Distance minimale : 160 km. Remontée des prédécesseurs : $E \leftarrow M \leftarrow Y$. L'itinéraire optimal est Yaoundé – Mbalmayo – Ebolowa (160 km), et non l'itinéraire par Obala (190 km).

EXERCICE 4 À toi de jouer

Sur le graphe fil rouge, détermine par l'algorithme de Dijkstra le plus court chemin de Rey-Bouba (R) à Touboro (T), en rédigeant le tableau complet d'exécution. Quelle est la distance minimale ? Compare avec l'itinéraire passant par Garoua.

CORRIGÉ Exercice 1

Init. : R : 0, autres $+\infty$. Fixe R : G : 90 (R), F : 50 (R). Fixe F (50) : P via F : $50 + 45 = 95$; B : $50 + 30 = 80$ (F); T : $50 + 80 = 130$ (F). Fixe B (80) : T via B : $80 + 35 = 115 < 130$, mise à jour : T : 115 (B); P via B : $80 + 70 = 150 > 95$. Fixe G (90) : P via G : $90 + 55 = 145 > 95$. Fixe P (95), puis fixe T (115). Distance minimale : 115 km, chemin $R \rightarrow F \rightarrow B \rightarrow T$ (vérification : $50 + 30 + 35 = 115$). Par Garoua, il faudrait $90 + 160 = 250$ km : bien plus long.

LES AVAIS-TU ? Vingt minutes qui ont changé le monde

L'informaticien néerlandais Edsger Dijkstra a conçu son algorithme en 1956, en une vingtaine de minutes, assis en terrasse d'un café d'Amsterdam avec sa fiancée — simplement pour se demander quel était le plus court chemin entre Rotterdam et Groningue ! Il ne le publia qu'en 1959, dans un article de trois pages devenu l'un des plus cités de l'histoire de l'informatique. Aujourd'hui, cet algorithme (et ses variantes comme A*) bat au cœur de chaque GPS, des applications de navigation utilisées par les motos-taxis de Douala, et du protocole de routage OSPF qui achemine tes données sur Internet, y compris via les réseaux MTN et Orange. Chaque fois qu'un téléphone calcule un itinéraire, c'est l'idée de Dijkstra qui travaille.

À RETENIR L'essentiel de la section

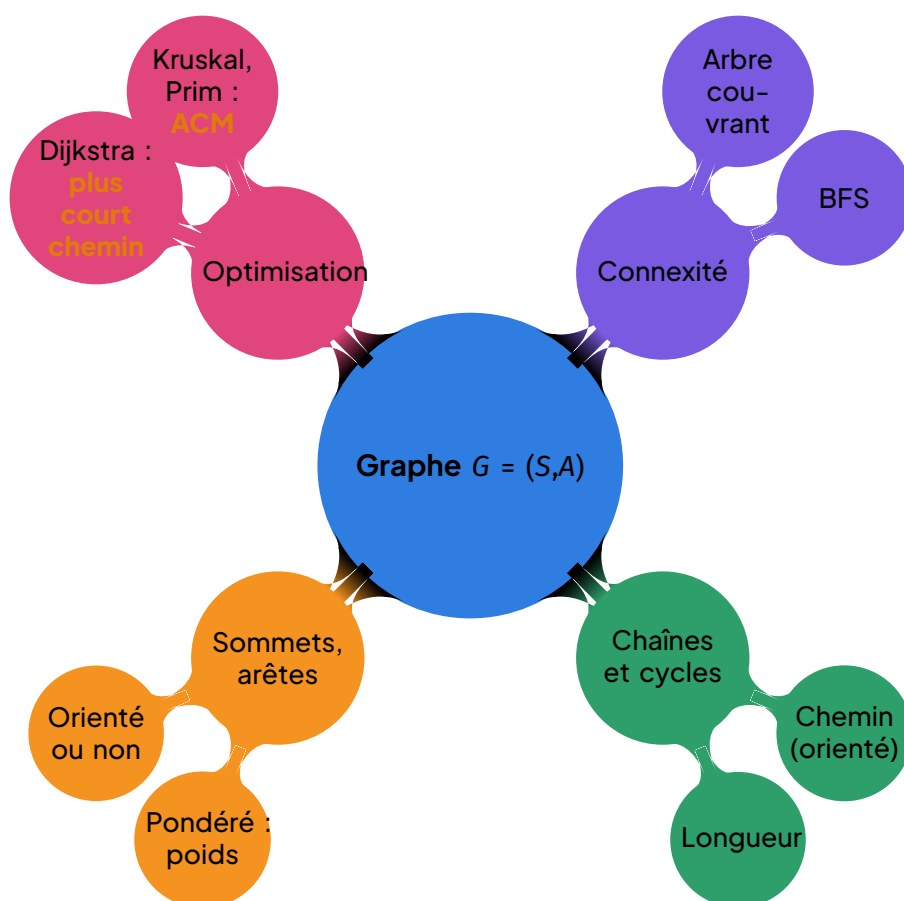
- Le **poids d'un chemin** est la somme des poids de ses arêtes ; le **plus court chemin** minimise ce poids (à ne pas confondre avec le nombre minimal d'arêtes du BFS).
- L'**algorithme de Dijkstra** (poids positifs ou nuls) : source à 0, autres à $+\infty$; on fixe le sommet de plus petite étiquette provisoire ; on met à jour ses voisins en notant le prédécesseur ; on répète jusqu'à l'arrivée.
- L'étiquette définitive de l'arrivée donne la **distance minimale** ; la **remontée des prédécesseurs** donne le chemin optimal.
- Au Bac : tableau à une ligne par étape, étiquettes « distance (prédécesseur) », étiquette fixée clairement marquée, et vérification finale de la somme des poids.

7 SYNTHÈSE ET MODÉLISATION DE SITUATIONS

Dans la section précédente, tu as appris à exécuter l'algorithme de Dijkstra pour déterminer un plus court chemin entre deux sommets d'un graphe pondéré. Tu disposes désormais de toute la boîte à outils de la leçon : les objets (graphes, chaînes, arbres), les propriétés (connexité, existence d'un arbre couvrant) et les trois grandes familles d'algorithmes (BFS, Kruskal-Prim, Dijkstra). Cette dernière section a un objectif précis : t'apprendre à **choisir** le bon outil face à un énoncé de type Baccalauréat, à **rédigier** proprement l'exécution d'un algorithme, et à **interpréter** le résultat dans le contexte concret. C'est exactement ce que le correcteur attend de toi.

7.1 Carte mentale de la leçon

Avant tout, organisons les notions. Toute la théorie des graphes vue en Terminale tient en une phrase : on modélise un réseau par un **graphe**, on s'assure qu'il est **connexe**, et selon le problème posé on en extrait un **arbre couvrant** de poids minimal ou un **plus court chemin**.



Carte mentale de la leçon : les quatre branches de la théorie des graphes au programme de Terminale.

À RETENIR Bilan des objets

- Un **graphe** $G = (S, A)$ est la donnée d'un ensemble de **sommets** et d'un ensemble d'**arêtes** (ou d'**arcs** si le graphe est orienté).
- Une **chaîne** est une suite de sommets reliés par des arêtes ; sa **longueur** est le nombre d'arêtes. Un **cycle** est une chaîne fermée.
- Un graphe est **connexe** si deux sommets quelconques sont toujours reliés par une chaîne.
- Un **arbre** est un graphe connexe sans cycle ; un **arbre couvrant** d'un graphe connexe G est un sous-graphe de G qui est un arbre contenant tous les sommets. Tout graphe connexe admet un arbre couvrant.
- Dans un graphe **pondéré**, le **poids** d'un sous-graphe est la somme des poids de ses arêtes.

7.2 Quel algorithme pour quel problème ?

Voici le cœur de la compétence exigible. Face à un énoncé, trois problèmes types peuvent se cacher, et chacun appelle un algorithme précis.

Problème concret	Objet mathématique	Algorithme adapté	Résultat à interpréter
Vérifier qu'un réseau est en un seul morceau ; distance minimale en nombre de liaisons	Connexité ; distance en nombre d'arêtes	Parcours en largeur (BFS)	Graphe connexe ou non ; plus courte chaîne en nombre d'arêtes
Câbler, canaliser ou connecter tous les points au moindre coût total	Arbre couvrant de poids minimal (ACM)	Kruskal ou Prim	Coût total minimal et liste des liaisons à construire
Trouver l'itinéraire le moins coûteux entre deux lieux précis	Plus court chemin entre deux sommets	Dijkstra	Coût minimal du trajet et itinéraire à suivre

MÉTHODE Fiche de décision : quel algorithme choisir face à un énoncé ?

Pose-toi trois questions, dans l'ordre :

1. Le graphe est-il pondéré ? Si non, seul le BFS est pertinent (connexité, distance en nombre d'arêtes). Si oui, continue.
2. Le problème concerne-t-il TOUS les sommets ou seulement DEUX sommets ? Relier

tout le monde au moindre coût $\Rightarrow$ **arbre couvrant minimal** (Kruskal ou Prim). Aller d'un point A à un point B $\Rightarrow$ **plus court chemin** (Dijkstra).

3. Que demande exactement la question finale ? Le coût total d'un réseau (somme des poids des arêtes retenues) ou le coût d'un trajet (somme des poids le long d'un chemin) ? Cette question tranche définitivement.

ATTENTION Ne confonds jamais ACM et plus court chemin

C'est l'erreur la plus fréquente au Baccalauréat. Kruskal et Prim construisent un arbre qui relie tous les sommets à coût total minimal ; Dijkstra construit un chemin minimal entre deux sommets. Ce sont deux problèmes distincts : le plus court chemin de A vers B n'est en général pas contenu dans l'arbre couvrant minimal, et l'arbre couvrant minimal ne donne pas les plus courts chemins. Appliquer Dijkstra à un problème de câblage minimal (ou Kruskal à un problème d'itinéraire) est une erreur de modélisation sanctionnée, même si les calculs sont justes. Autres pièges : appliquer Dijkstra à un graphe comportant des **poids négatifs** (l'algorithme n'est alors plus valide), et traiter un graphe **orienté** comme non orienté (un arc $A \rightarrow B$ ne permet pas de revenir de B vers A).

7.3 Méthodologie complète d'un problème type Bac

MÉTHODE Résoudre un problème de graphes en quatre étapes

1. Modéliser : identifier les sommets (lieux, machines, personnes), les arêtes ou arcs (liaisons possibles), le sens éventuel, et les poids (coûts, distances, durées). Dessiner le graphe ou donner sa liste d'arêtes pondérées.
2. Choisir l'algorithme grâce à la fiche de décision ci-dessus, et le justifier en une phrase : « On cherche à relier tous les villages à coût minimal, donc on détermine un arbre couvrant de poids minimal par l'algorithme de Kruskal. »
3. Exécuter avec un tableau : tableau d'arêtes triées et décisions (Kruskal), tableau de sommets atteints et arêtes retenues (Prim), ou tableau des étiquettes provisoires/définitives (Dijkstra). Le tableau est obligatoire : il rend ton raisonnement lisible et vérifiable.
4. Interpréter dans le contexte : conclure par une phrase en français courant, avec l'unité : « Le coût minimal de l'électrification est de 37 millions de FCFA ». Une réponse sans phrase d'interprétation perd des points.

7.4 Étude de cas guidée : électrification rurale

EXERCICE RÉSOLU 7 Électrification de cinq villages

Une commune de la région de l'Est veut électrifier cinq villages A, B, C, D, E à partir de la centrale située en A. Les liaisons électriques possibles et leurs coûts de construction (en millions de FCFA) sont : $AB : 4$; $AC : 2$; $BC : 1$; $BD : 5$; $CD : 8$; $CE : 10$; $DE : 2$.

1. Modéliser la situation par un graphe.
2. Déterminer le réseau de coût total minimal permettant d'alimenter tous les villages, et donner ce coût.

SOLUTION

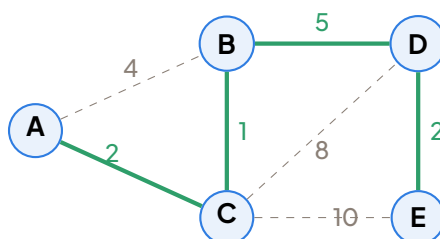
1. Modélisation. Les sommets sont les cinq villages ; les arêtes sont les liaisons possibles, pondérées par leur coût. Le graphe est non orienté (le courant circule dans les deux sens) et pondéré.
2. Choix et exécution. On veut relier tous les villages à coût total minimal : c'est un problème d'**arbre couvrant de poids minimal**. On applique l'algorithme de Kruskal : on trie les arêtes par coût croissant et on retient chaque arête sauf si elle crée un cycle.

Arête	Coût	Décision	Justification
BC	1	Retenue	ne crée pas de cycle
AC	2	Retenue	ne crée pas de cycle
DE	2	Retenue	ne crée pas de cycle
AB	4	Rejetée	crée le cycle A-B-C-A
BD	5	Retenue	relie {D,E} au reste

On s'arrête dès que l'on a retenu $5 - 1 = 4$ arêtes : $\{BC, AC, DE, BD\}$. Le poids total est

$$1 + 2 + 2 + 5 = 10.$$

Interprétation. La commune doit construire les lignes B-C, A-C, D-E et B-D ; le coût minimal de l'électrification des cinq villages est de 10 millions de FCFA, et tous les villages sont alors alimentés depuis la centrale A.



Le réseau d'électrification : en vert plein, l'arbre couvrant de poids minimal (poids 10); en pointillés gris, les liaisons non retenues.

7.5 Étude de cas guidée : distribution d'eau potable et plus court chemin

Compliquons la situation. Une commune veut (a) poser des canalisations pour desservir tous ses quartiers au moindre coût, puis (b) déterminer le trajet le plus court pour un camion de maintenance allant du château d'eau S au quartier T.

Pour la question (a), c'est un **ACM** : on applique Kruskal ou Prim sur le graphe pondéré par les coûts de pose. Pour la question (b), c'est un **plus court chemin** : on applique Dijkstra depuis S. Remarque bien que les deux réponses sont en général différentes : le camion n'a aucune raison de suivre uniquement les canalisations de l'arbre minimal, puisqu'il emprunte les routes, pondérées par des distances. La même carte de la commune donne ainsi naissance à deux graphes différents (coûts de pose d'un côté, distances routières de l'autre) et à deux algorithmes différents. C'est exactement le type de discernement que l'épreuve du Baccalauréat teste.

EXERCICE 5 Réseau d'eau potable

Le réseau routier d'une commune relie le château d'eau S aux quartiers A, B, C, T par les routes suivantes (distances en km) : SA : 1 ; SB : 4 ; AB : 2 ; AC : 6 ; BC : 3 ; BT : 8 ; CT : 2.

1. Appliquer l'algorithme de Dijkstra pour déterminer la distance minimale de S à T, en présentant le tableau des étiquettes.
2. Donner l'itinéraire optimal du camion de maintenance et sa longueur.

CORRIGÉ Exercice : réseau d'eau potable

1. Tableau des étiquettes (on note ∞ une étiquette provisoire infinie ; en gras le sommet sélectionné à chaque étape, étiquette définitive soulignée) :

Étape	S	A	B	C	T
Initialisation	<u>0</u>	∞	∞	∞	∞
Après S	-	1 (S)	4 (S)	∞	∞
Après A	-	<u>1</u>	3 (A)	7 (A)	∞
Après B	-	<u>1</u>	<u>3</u>	6 (B)	11 (B)
Après C	-	<u>1</u>	<u>3</u>	<u>6</u>	8 (C)

2. L'étiquette définitive de T est 8, obtenue depuis C, lui-même atteint depuis B, atteint depuis A, atteint depuis S. L'itinéraire optimal est donc $S \rightarrow A \rightarrow B \rightarrow C \rightarrow T$, de longueur $1 + 2 + 3 + 2 = 8$ km. Interprétation : le camion parcourt au minimum 8 km en passant successivement par les quartiers A, B et C.

7.6 Pièges fréquents aux examens et entraînement à la rédaction

ATTENTION Les cinq pièges classiques

1. Orienté ou non orienté ? Lis l'énoncé : une « route à sens unique » impose un arc, pas une arête. Dans un graphe orienté, une chaîne peut emprunter un arc à contre-sens, un **chemin** non.
2. Poids négatifs : Dijkstra est valide uniquement si tous les poids sont positifs ou nuls. Si un énoncé comporte des gains (poids négatifs), signale que Dijkstra ne s'applique pas.
3. Oublier le critère d'arrêt : Kruskal s'arrête à $n-1$ arêtes retenues ; Dijkstra peut s'arrêter dès que le sommet cible reçoit son étiquette définitive.
4. Oublier l'interprétation finale : tout résultat numérique doit être traduit en une phrase avec l'unité et le contexte (FCFA, km, minutes).
5. Tableau brouillon : au Bac, le tableau d'exécution de l'algorithme est un élément de notation. Présente-le avec une ligne par étape, les sommets en colonnes, et indique clairement les étiquettes définitives (surlignées ou encadrées) ainsi que le sommet d'origine entre parenthèses.

MÉTHODE Présentation soignée exigée au Bac

1. Annoncer l'algorithme utilisé et le justifier en une phrase.
2. Pour Kruskal : tableau à trois colonnes (arête, poids, décision retenue/rejetée avec

justification « crée un cycle » ou « relie deux composantes »).

3. Pour Prim : indiquer à chaque étape l'ensemble des sommets atteints et l'arête de poids minimal ajoutée.
4. Pour Dijkstra : tableau des étiquettes avec, pour chaque sommet, la distance provisoire suivie du sommet d'origine entre parenthèses, par exemple 6 (B); barrer proprement les étiquettes remplacées plutôt que de les effacer.
5. Conclure par la réponse en français, avec l'unité, en reprenant le vocabulaire de l'énoncé (villages, câbles, quartiers).

LESAVAIS-TU ? Les sept ponts de Königsberg (1736)

La théorie des graphes est née d'une promenade ! À Königsberg (aujourd'hui Kaliningrad), sept ponts enjambaient la rivière Pregel, et les habitants se demandaient s'il était possible de traverser chaque pont exactement une fois et de revenir à son point de départ. En 1736, le mathématicien suisse **Leonhard Euler** modélisa la ville par un graphe — les rives et îles comme sommets, les ponts comme arêtes — et démontra que la promenade était impossible, car tous les sommets avaient un degré impair. Ce simple dessin de points et de traits fondait une branche entière des mathématiques, aujourd'hui indispensable : c'est elle qui fait fonctionner les GPS, l'acheminement des appels chez MTN et Orange, la planification des réseaux d'eau et d'électricité, et même les recommandations des réseaux sociaux.

À RETENIR L'essentiel de la leçon

- Un problème de réseau se résout en quatre temps : **modéliser**, **choisir** l'algorithme, **exécuter** avec un tableau, **interpréter**.
- Connexité ou distance en nombre d'arêtes $\Rightarrow$ BFS.
- Relier tous les sommets à coût total minimal $\Rightarrow$ Kruskal ou Prim (arbre couvrant de poids minimal).
- Trajet minimal entre deux sommets d'un graphe à poids positifs $\Rightarrow$ Dijkstra.
- Tout graphe connexe admet un arbre couvrant : c'est la propriété qui garantit que les problèmes de câblage minimal ont toujours une solution.

Tu es maintenant au bout de la leçon. La théorie des graphes est l'un des chapitres les plus concrets du programme : chaque algorithme que tu as appris répond à une vraie question d'ingénieur, d'économiste ou d'urbaniste. Entraîne-toi sur les exercices qui suivent en rédigeant systématiquement tes tableaux et tes phrases d'interprétation : c'est cette rigueur qui fera la différence le jour du Baccalauréat.

8 ACTIVITÉ D'INTÉGRATION

9 THÉORIE DES GRAPHS : ACTIVITÉ D'INTÉGRATION — ÉLECTRIFICATION RURALE À PITOA

9.1 Objectifs de l'activité

À l'issue de cette activité d'intégration, tu seras capable de :

- **modéliser** une situation concrète d'aménagement du territoire par un **graphe pondéré** (double pondération coût / distance) ;
- vérifier la **connexité** d'un réseau à l'aide d'un **parcours en largeur** (BFS) et en extraire un **arbre couvrant** ;
- déterminer un **arbre couvrant de poids minimal** par les algorithmes de **Kruskal** et de **Prim**, puis comparer les résultats ;
- déterminer un **plus court chemin** par l'algorithme de **Dijkstra** ;
- rédiger une note de synthèse argumentée, à destination d'un décideur, en interprétant les résultats mathématiques dans le contexte de la situation.

9.2 Situation

La commune de **Pitoa**, dans le département du Bénoué (région du Nord), compte sept localités : le chef-lieu **Pitoa** (*P*) et les villages **Bibemi** (*B*), **Galdi** (*G*), **Tcheboa** (*T*), **Mbé** (*M*), **Ngaoui** (*N*) et **Dourbeye** (*D*). Dans le cadre du programme d'électrification rurale, l'Agence de l'énergie rurale, en partenariat avec ENEO, envisage de raccorder tous les villages au réseau électrique en suivant les pistes rurales existantes : les lignes électriques ne peuvent passer que le long de ces pistes.

Le bureau d'études a relevé, pour chaque tronçon de piste reliant directement deux localités, le **coût d'électrification** (en millions de FCFA, incluant poteaux, câbles et transformateurs) et la **longueur** de la piste (en km) :

Tronçon	Localités reliées	Coût (millions FCFA)	Distance (km)
1	Pitoa – Bibemi ($P-B$)	8	6
2	Pitoa – Galdi ($P-G$)	12	9
3	Bibemi – Galdi ($B-G$)	6	5
4	Bibemi – Tcheboa ($B-T$)	10	8
5	Galdi – Mbé ($G-M$)	7	7
6	Tcheboa – Mbé ($T-M$)	5	4
7	Tcheboa – Ngaoui ($T-N$)	9	7
8	Mbé – Ngaoui ($M-N$)	4	6
9	Mbé – Dourbeye ($M-D$)	11	9
10	Ngaoui – Dourbeye ($N-D$)	6	5
11	Galdi – Ngaoui ($G-N$)	14	12

Le maire de Pitoa dispose d'un budget limité. Il pose à son équipe technique (c'est toi et ton groupe!) deux questions précises :

1. **Question budgétaire** : quel ensemble de tronçons faut-il électrifier pour que tous les villages soient raccordés au réseau (directement ou par transit), au coût total minimal? Quel est ce coût?
2. **Question logistique** : le village de Dourbeye, le plus isolé, doit être approvisionné en matériel depuis Pitoa par camion. Quel itinéraire routier (en km) faut-il recommander au transporteur?

9.3 Consignes

Le travail se fait en groupes de 3 ou 4 élèves. Chaque groupe rédigera ses réponses sur une feuille, puis une mise en commun aura lieu en fin de séance.

1. **Modélisation**. Représente la situation par un graphe pondéré G : précise l'ensemble des sommets, l'ensemble des arêtes, et la fonction de poids. Le graphe est-il orienté? Justifie. (On pourra faire figurer sur chaque arête les deux nombres : coût et distance.)
2. **Connexité**. Exécute un parcours en largeur (BFS) depuis le sommet P (chef-lieu), en choisissant les voisins par ordre alphabétique. Donne l'ordre de visite des sommets et la liste des arêtes de l'arbre de parcours. Le graphe est-il connexe? Justifie en t'appuyant sur le résultat du parcours. Quel arbre couvrant obtient-on ainsi? Calcule son poids (en coût) : est-il nécessairement minimal?
3. **Coût minimal par Kruskal**. Applique l'algorithme de Kruskal au graphe pondéré par les coûts : range les arêtes par coût croissant, puis construis l'arbre couvrant minimal en détaillant chaque étape (arête retenue ou rejetée, avec la raison du rejet). Donne la liste des tronçons retenus et le coût total.
4. **Coût minimal par Prim**. Applique l'algorithme de Prim en partant du sommet P : à chaque étape, indique l'ensemble des sommets déjà atteints et l'arête choisie. Compare le résultat avec celui de Kruskal (tronçons retenus, coût total). Ce résultat était-il prévisible? Justifie.

5. **Plus court itinéraire par Dijkstra.** Applique l'algorithme de Dijkstra au graphe pondéré par les distances pour déterminer le plus court chemin de P à D . Présente le tableau des distances provisoires et définitives, puis donne l'itinéraire optimal et sa longueur totale.
6. **Note de synthèse.** Rédige une note d'une demi-page à l'intention du maire : coût total du réseau minimal, liste des tronçons à électrifier, itinéraire recommandé pour Dourbeye, et une phrase expliquant pourquoi le réseau proposé ne contient aucun « bouclage » (cycle) inutile.

9.4 Ressources à mobiliser

À RETENIR Boîte à outils de la leçon

- Un **graphe** $G = (S, A)$ est **connexe** si deux sommets quelconques sont reliés par une chaîne ; le **parcours en largeur** depuis un sommet s atteint tous les sommets si et seulement si le graphe est connexe, et les arêtes empruntées forment alors un **arbre couvrant**.
- Un **arbre** est un graphe connexe sans cycle ; un arbre à n sommets possède exactement $n - 1$ arêtes. Tout graphe connexe admet un **arbre couvrant** (propriété exigible au Bac).
- **Kruskal** : trier les arêtes par poids croissant, retenir chaque arête qui ne crée pas de cycle avec les arêtes déjà choisies, jusqu'à obtenir $n - 1$ arêtes.
- **Prim** : partir d'un sommet, ajouter à chaque étape l'arête de poids minimal reliant un sommet atteint à un sommet non atteint, jusqu'à atteindre tous les sommets.
- **Dijkstra** (poids strictement positifs) : affecter 0 au sommet de départ et $+\infty$ aux autres ; à chaque étape, fixer le sommet de plus petite distance provisoire et mettre à jour ses voisins ; lire le chemin optimal en remontant les prédécesseurs.

9.5 Démarche et corrigé commenté

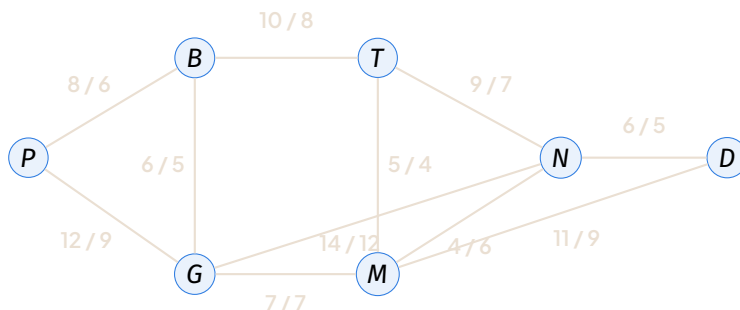
EXERCICE RÉSOLU 8 Électrification de la commune de Pitoa : corrigé complet

Énoncé : répondre aux six consignes de l'activité à partir des données du bureau d'études.

SOLUTION

Étape 1 : modélisation. On modélise la commune par un graphe non orienté $G = (S, A)$ où $S = \{P, B, G, T, M, N, D\}$ (les sept localités) et A l'ensemble des onze tronçons. Le graphe est non orienté car une piste se parcourt dans les deux sens, et une ligne électrique alimente dans les deux sens. On utilise deux pondérations sur les mêmes arêtes : le coût c (en millions de

FCFA) pour les questions budgétaires, et la distance d (en km) pour la question logistique.



Graphe des tronçons : sur chaque arête, coût (millions FCFA) / distance (km).

Étape 2 : BFS depuis P (voisins par ordre alphabétique). On maintient une file (premier entré, premier sorti). On marque les sommets visités.

Sommet traité	Voisins non visités (ordre alphab.)	File (après ajout)
P	B, G	B, G
B	T (P, G déjà vus)	G, T
G	M, N (P, B déjà vus)	T, M, N
T	– (B, M, N déjà vus)	M, N
M	D (G, T, N déjà vus)	N, D
N	– (T, M, D, G déjà vus)	D
D	– (M, N déjà vus)	–

Ordre de visite (défilement) : P, B, G, T, M, N, D . Les 7 sommets sont atteints, donc le graphe est connexe. L'arbre de parcours (arêtes empruntées lors de la découverte) : $\{PB, PG, BT, GM, GN, MD\}$. C'est un arbre couvrant (6 arêtes, 7 sommets, connexe, sans cycle). Son poids en coût : $c(PB) + c(PG) + c(BT) + c(GM) + c(GN) + c(MD) = 8 + 12 + 10 + 7 + 14 + 11 = 62$ millions de FCFA. Il couvre bien tous les villages mais n'est pas minimal (on verra qu'on peut faire 36 millions), d'où l'intérêt des algorithmes d'optimisation.

Étape 3 : Kruskal (pondération par les coûts). On trie les arêtes par coût croissant. On construit l'arbre en ajoutant les arêtes qui ne forment pas de cycle.

Arête	Coût	Action	Justification / Composantes connexes
<i>MN</i>	4	Retenue	$\{M, N\}$
<i>TM</i>	5	Retenue	$\{T, M, N\}$
<i>BG</i>	6	Retenue	$\{B, G\}$ et $\{T, M, N\}$
<i>ND</i>	6	Retenue	$\{T, M, N, D\}$ et $\{B, G\}$
<i>GM</i>	7	Retenue	Relie $\{B, G\}$ à $\{T, M, N, D\} \rightarrow \{B, G, T, M, N, D\}$
<i>PB</i>	8	Retenue	Relie <i>P</i> à la grande composante $\rightarrow$ Tous les sommets atteints (6 arêtes)
<i>TN</i>	9	Rejetée	Crée un cycle dans $\{T, M, N, D\}$
<i>BT</i>	10	Rejetée	Crée un cycle $(B - G - M - T - B)$
<i>MD</i>	11	Rejetée	Crée un cycle $(M - N - D - M)$
<i>PG</i>	12	Rejetée	Crée un cycle $(P - B - G - P)$
<i>GN</i>	14	Rejetée	Crée un cycle

Arbre couvrant minimal (ACM) : $\{MN, TM, BG, ND, GM, PB\}$. Coût total : $4 + 5 + 6 + 6 + 7 + 8 = 36$ millions de FCFA.

Étape 4 : Prim depuis *P* (pondération par les coûts). On part de $\{P\}$, on ajoute à chaque étape l'arête de coût minimal reliant l'ensemble atteint à un sommet extérieur.

Sommets atteints	Arêtes candidates (sortantes)	Arête choisie
$\{P\}$	<i>PB</i> (8), <i>PG</i> (12)	<i>PB</i> (8)
$\{P, B\}$	<i>PG</i> (12), <i>BG</i> (6), <i>BT</i> (10)	<i>BG</i> (6)
$\{P, B, G\}$	<i>PG</i> (12), <i>BT</i> (10), <i>GM</i> (7), <i>GN</i> (14)	<i>GM</i> (7)
$\{P, B, G, M\}$	<i>PG</i> (12), <i>BT</i> (10), <i>GN</i> (14), <i>MN</i> (4), <i>MD</i> (11)	<i>MN</i> (4)
$\{P, B, G, M, N\}$	<i>PG</i> (12), <i>BT</i> (10), <i>GN</i> (14), <i>MD</i> (11), <i>TM</i> (5), <i>ND</i> (6)	<i>TM</i> (5)
$\{P, B, G, M, N, T\}$	<i>PG</i> (12), <i>BT</i> (10), <i>GN</i> (14), <i>MD</i> (11), <i>ND</i> (6)	<i>ND</i> (6)

Arêtes retenues : $\{PB, BG, GM, MN, TM, ND\}$. Coût total : $8 + 6 + 7 + 4 + 5 + 6 = 36$ millions de FCFA. Conclusion : Kruskal et Prim donnent le même arbre et le même coût minimal. L'arbre couvrant minimal est ici unique (les poids sont tous distincts sauf $BG = ND = 6$, mais l'ordre de traitement n'a pas créé d'ambiguïté), ce qui conforte le résultat.

Étape 5 : Dijkstra de *P* à *D* (pondération par les distances en km). Initialisation : $d(P) = 0$ (définitive), $d(X) = +\infty$ pour $X \neq P$. Prédécesseurs $\pi(X) = \emptyset$.

Étape	Sommet fixé	$d(B)$	$d(G)$	$d(T)$	$d(M)$	$d(N)$	$d(D)$
Init	–	∞	∞	∞	∞	∞	∞
1	$P(0)$	$6 (\pi = P)$	$9 (\pi = P)$	∞	∞	∞	∞
2	$B(6)$	6	9	$14 (\pi = B)$	∞	∞	∞
3	$G(9)$	6	9	14	$16 (\pi = G)$	$21 (\pi = G)$	∞
4	$T(14)$	6	9	14	$16 (\min(16,18))$	$21 (\min(21,21))$	∞
5	$M(16)$	6	9	14	16	$21 (\min(21,22))$	$25 (\pi = M)$
6	$N(21)$	6	9	14	16	21	$25 (\min(25,26))$
7	$D(25)$	6	9	14	16	21	25

Légende : les cellules vertes indiquent les distances devenues définitives. En remontant les prédécesseurs depuis D : $D \xleftarrow{\pi} M \xleftarrow{\pi} G \xleftarrow{\pi} P$. Le plus court chemin est $P \rightarrow G \rightarrow M \rightarrow D$. Longueur totale : $d(P,G) + d(G,M) + d(M,D) = 9 + 7 + 9 = 25$ km.

Étape 6 : note de synthèse (modèle attendu).

Note à l'attention de Monsieur le Maire de Pitoa

Objet : Réseau d'électrification rurale optimal et itinéraire d'approvisionnement de Dourbeye.

Monsieur le Maire,

L'étude du réseau de pistes de la commune montre qu'il est possible de raccorder les sept localités au réseau électrique pour un coût minimal de 36 millions de FCFA, en électrifiant les six tronçons suivants : Pitoa–Bibemi, Bibemi–Galdi, Galdi–Mbé, Mbé–Ngaoui, Mbé–Tcheboa, Ngaoui–Dourbeye.

Ce réseau ne comporte aucun « bouclage » (cycle) : tout tronçon supplémentaire créerait un circuit fermé inutile, donc un surcoût sans aucun village raccordé de plus (un arbre couvrant à 7 sommets a exactement 6 arêtes).

Pour l'approvisionnement en matériel du village de Dourbeye depuis Pitoa, l'itinéraire routier le plus court est : Pitoa $\rightarrow$ Galdi $\rightarrow$ Mbé $\rightarrow$ Dourbeye, soit une distance de 25 km.

Ces choix garantissent l'efficacité budgétaire et logistique du projet.

ATTENTION

Pièges fréquents dans ce type de problème

- Ne pas confondre les deux pondérations : Kruskal et Prim s'appliquent ici aux coûts, Dijkstra aux distances. Mélanger les deux fausse tout le raisonnement.
- Dans Kruskal, une arête n'est rejetée que si ses deux extrémités sont déjà dans la même composante connexe : vérifier soigneusement avant de rejeter (utiliser un coloriage ou une notation d'ensemble).

- Dans Dijkstra, on ne fixe un sommet que lorsqu'il porte la plus petite distance provisoire parmi les non fixés ; et l'algorithme exige des poids strictement positifs, ce qui est le cas ici (distances > 0).
- Un arbre couvrant d'un graphe à 7 sommets possède exactement 6 arêtes : si tu en comptes 5 (graphe non connexe) ou 7 (cycle), il y a une erreur.
- Pour le BFS, l'ordre alphabétique des voisins est une contrainte de l'énoncé pour avoir un corrigé unique ; en situation libre, l'ordre n'importe pas pour la connexité mais change l'arbre de parcours.

9.6 Bilan de l'activité

Cette activité a permis de mobiliser, sur une situation authentique unique, l'ensemble des savoir-faire de la leçon. Tu as d'abord constaté qu'une même réalité (le réseau de pistes) se traduit par un seul graphe, mais que le choix de la **pondération** dépend de la question posée : les coûts pour optimiser le budget (problème global : relier tout le monde), les distances pour optimiser le trajet (problème local : aller d'un point à un autre). Le parcours en largeur a fourni un outil simple pour prouver la **connexité** — condition indispensable, car sans elle aucun réseau ne peut alimenter tous les villages — tout en montrant qu'un arbre couvrant « quelconque » (ici 62 millions) peut être très loin de l'optimum. Les algorithmes de **Kruskal** et de **Prim**, menés en parallèle, ont abouti au même réseau optimal de 36 millions, illustrant le fait que deux méthodes différentes peuvent certifier un même optimum ; la comparaison des résultats entre groupes joue d'ailleurs le rôle de vérification croisée. Enfin, l'algorithme de **Dijkstra** a répondu à une question d'une nature différente. La rédaction de la note au maire t'a enfin entraîné à interpréter les résultats mathématiques : c'est cette capacité à traduire un graphe en décision concrète qui fait toute la valeur de la théorie des graphes, au Baccalauréat comme dans la vie professionnelle (aménagement du territoire, télécommunications MTN/Orange, logistique, transport).

10 MÉTHODES ET EXERCICES RÉSOLUS

11 THÉORIE DES GRAPHS

11.1 Définitions fondamentales

DÉFINITION Graphe, sommets, arêtes

Un **graphe** G est un couple (S, A) où :

- S est un ensemble fini non vide dont les éléments sont appelés **sommets** (ou nœuds) ;
- A est un ensemble de paires de sommets distincts, appelées **arêtes** (ou liens).

Si les arêtes sont des paires non ordonnées $\{x, y\}$, le graphe est **non orienté**. Si ce sont des paires ordonnées (x, y) , le graphe est **orienté** (on parle d'**arcs**).

DÉFINITION Graphe pondéré, sous-graphe

Un **graphe pondéré** est un graphe muni d'une fonction $p : A \rightarrow \mathbb{R}_+^*$ (ou $\mathbb{R}_+$) qui associe un **poids** (coût, distance, durée...) à chaque arête. Un **sous-graphe** de $G = (S, A)$ est un graphe $G' = (S', A')$ tel que $S' \subseteq S$ et $A' \subseteq A$ (avec les extrémités dans S').

DÉFINITION Chaîne, chemin, cycle, longueur

Dans un graphe non orienté :

- Une **chaîne** est une suite de sommets $v_0, v_1, \dots, v_k$ telle que $\{v_{i-1}, v_i\} \in A$ pour tout $i = 1..k$. Sa **longueur** est k (nombre d'arêtes).
- Un **chemin** est une chaîne dont les sommets sont deux à deux distincts (sauf éventuellement $v_0 = v_k$).
- Un **cycle** est un chemin de longueur ≥ 3 tel que $v_0 = v_k$ (premier et dernier sommet identiques, les autres distincts).

Dans un graphe orienté, on parle de **chemin orienté** et de **cycle orienté** en suivant le sens des arcs.

DÉFINITION Graphe connexe, composante connexe

Un graphe non orienté est **connexe** si pour tout couple de sommets (x, y) , il existe au moins une chaîne reliant x à y . Une **composante connexe** est un sous-graphe connexe maximal (non contenu dans un sous-graphe connexe plus grand).

DÉFINITION Arbre, arbre couvrant

Un **arbre** est un graphe connexe sans cycle.

PROPRIÉTÉ Caractérisation de l'arbre

Pour un graphe connexe à n sommets, les propriétés suivantes sont équivalentes :

1. C'est un arbre (connexe et sans cycle).
2. Il a exactement $n - 1$ arêtes.
3. Il est connexe et la suppression de toute arête le rend non connexe.
4. Entre deux sommets quelconques, il existe un unique chemin.

Soit G un graphe connexe. Un **arbre couvrant** de G est un sous-graphe de G qui est un arbre et qui contient tous les sommets de G . Il a donc $n - 1$ arêtes.

PROPRIÉTÉ Existence

Tout graphe connexe possède au moins un arbre couvrant.

11.2 Modéliser une situation concrète par un graphe

MÉTHODE Traduire une situation en graphe

Pour modéliser une situation concrète (réseau routier, réseau téléphonique, tournoi, planning, câblage...) :

1. Identifier les sommets : ce sont les objets de la situation (villes, antennes, équipes, personnes, tâches, quartiers...). Les nommer par des lettres (majuscules) et lister explicitement l'ensemble S .
2. Identifier les arêtes : deux sommets sont reliés s'il existe la relation étudiée entre

eux (route directe, liaison possible, match joué, canalisation possible...). Lister l'ensemble A.

3. Choisir orienté ou non orienté : si la relation est symétrique (route à double sens, liaison bidirectionnelle, « A connaît B » réciproque), le graphe est **non orienté** ; si elle ne l'est pas (sens unique, « A appelle B », « A bat B », précédence de tâches), le graphe est **orienté** et on dessine des flèches.
4. Choisir pondéré ou non : si chaque liaison porte une quantité numérique (distance en km, coût en FCFA, durée en min, capacité), le graphe est **pondéré** : on inscrit le poids sur chaque arête.
5. Rédiger la légende : Indispensable. Préciser dans la copie : « Sommets = ... ; Arêtes = ... ; Poids = ... (unité) ; Graphe orienté / non orienté ». C'est ce qui donne son sens à la modélisation et qui est noté au Bac.

EXERCICE RÉSOLU 9 Réseau de distribution d'eau – CAMWATER

La société CAMWATER veut relier cinq quartiers A, B, C, D, E d'une ville par des canalisations. Les liaisons techniquement possibles et leurs coûts (en millions de FCFA) sont : AB : 4 ; AC : 2 ; BC : 5 ; BD : 3 ; BE : 7 ; CD : 8 ; DE : 1.

1. Modéliser cette situation par un graphe que l'on précisera (orienté ou non, pondéré ou non).
2. Donner la liste des sommets et la liste des arêtes avec leurs poids.

SOLUTION

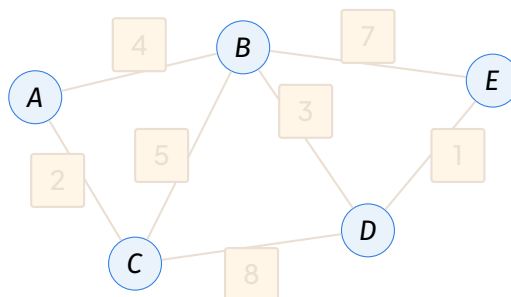
1. Choix du modèle.

- Sommets : les cinq quartiers → graphe à 5 sommets.
- Arêtes : une canalisation entre deux quartiers peut servir dans les deux sens (relation symétrique) → graphe **non orienté**.
- Poids : chaque liaison porte un coût de construction → graphe **pondéré**.

2. Description formelle.

- Ensemble des sommets : $S = \{A, B, C, D, E\}$;
- Ensemble des arêtes pondérées (poids en millions de FCFA) :

$$\{A,B\} : 4 ; \{A,C\} : 2 ; \{B,C\} : 5 ; \{B,D\} : 3 ; \{B,E\} : 7 ; \{C,D\} : 8 ; \{D,E\} : 1.$$



Réseau CAMWATER : graphe non orienté pondéré (coûts en millions FCFA).

Conclusion. La situation est modélisée par un graphe non orienté pondéré à 5 sommets et 7 arêtes, où chaque poids représente le coût de construction de la canalisation correspondante.

11.3 Chaînes, chemins, cycles et connexité

MÉTHODE Étudier la connexité d'un graphe

1. Parcours depuis un sommet : choisir un sommet x_0 , le marquer. Propager le marquage : tant qu'il existe une arête entre un sommet marqué et un sommet non marqué, marquer ce dernier.
2. Conclusion :
 - Si tous les sommets sont marqués → le graphe est **connexe**.
 - Sinon, les sommets marqués forment une composante connexe. Exhiber un sommet non marqué : il n'est relié à x_0 par aucune chaîne → le graphe n'est pas connexe.
3. Preuve de non-connexité : il suffit d'exhiber deux sommets tels qu'aucune chaîne ne les relie (ou de partitionner S en deux parties sans arête entre elles).

EXERCICE RÉSOLU 10 Graphe connexe ou non ?

On considère le graphe G de sommets $\{A, B, C, D, E, F\}$ et d'arêtes :

$$\{A, B\}, \{A, C\}, \{B, C\}, \{B, D\}, \{C, D\}, \{D, E\}, \{E, F\}.$$

1. Le graphe G est-il connexe ? Justifier.

2. G est-il un arbre ? Justifier.

3. Extraire de G un arbre couvrant et donner sa liste d'arêtes.

SOLUTION

1. Connexité. Partons de A et propageons le marquage : A atteint B et C ; B atteint D ; D atteint E ; E atteint F . Les six sommets sont atteints. Par exemple, A et F sont reliés par la chaîne $A - B - D - E - F$ (longueur 4). Deux sommets quelconques étant reliés par une chaîne, le graphe G est connexe.

2. Est-ce un arbre ? G possède $n = 6$ sommets et 7 arêtes. Or un arbre à 6 sommets possède exactement $6 - 1 = 5$ arêtes (propriété équivalente). Comme $7 \neq 5$, G n'est pas un arbre. (On peut aussi exhiber le cycle $A - B - C - A$ de longueur 3, ce qui prouve directement la présence d'un cycle.)

3. Extraction d'un arbre couvrant. On supprime des arêtes dans les cycles jusqu'à n'en plus avoir :

- Cycle $A - B - C - A$: on supprime $\{A, C\}$ (choix arbitraire).
- Cycle $B - C - D - B$: on supprime $\{C, D\}$ (choix arbitraire).

Il reste les arêtes $\{A, B\}, \{B, C\}, \{B, D\}, \{D, E\}, \{E, F\}$, soit $5 = n - 1$ arêtes. Le sous-graphe obtenu contient les 6 sommets, reste connexe (tout sommet est relié à B ou via B) et ne contient plus de cycle. Conclusion. Un arbre couvrant de G est donné par les arêtes $\{A, B\}, \{B, C\}, \{B, D\}, \{D, E\}, \{E, F\}$. (Ce n'est pas le seul : toute suppression d'arêtes cassant tous les cycles convient.)

11.4 Arbres et arbres couvrants

MÉTHODE**Extraire un arbre couvrant d'un graphe connexe**

1. Vérifier que le graphe est connexe (sinon, pas d'arbre couvrant).
2. Tant qu'il existe un cycle dans le graphe courant : supprimer une arête de ce cycle (n'importe laquelle).
3. S'arrêter quand il n'y a plus de cycle. Le sous-graphe obtenu contient tous les sommets, est connexe et sans cycle : c'est un arbre couvrant (il a exactement $n - 1$ arêtes).
4. Vérification finale : compter les arêtes restantes ($= n - 1$) et vérifier que tous les sommets sont encore reliés.

À RETENIR**Propriétés clés pour le Bac**

- $\text{Arbre} \iff \text{Connexe} + \text{Sans cycle} \iff \text{Connexe} + n - 1 \text{ arêtes.}$
- Tout graphe connexe admet un arbre couvrant (démonstration constructive par suppression d'arêtes dans les cycles).
- Un arbre couvrant relie tous les sommets avec le minimum d'arêtes possible ($n - 1$).

11.5 Parcours en largeur (BFS)

MÉTHODE Exécuter un parcours en largeur (BFS)

Donné un graphe non orienté et un **sommet de départ** S :

1. Initialiser une **file** (FIFO : premier entré, premier sorti). Marquer S , l'enfiler. Noter son niveau 0 et son prédécesseur « - ».
2. Tant que la file n'est pas vide :
 - (a) Défiler le sommet X en tête de file.
 - (b) Examiner tous les voisins Y de X non encore marqués (par ordre alphabétique si aucun ordre n'est imposé).
 - (c) Pour chaque tel voisin Y : le marquer, noter son niveau = niveau(X)+1, noter son prédécesseur = X , et enfile Y .
3. L'ordre de visite est l'ordre dans lequel les sommets sont défilés (équivalent à l'ordre de marquage pour un BFS standard).
4. L'arbre de parcours (ou arbre BFS) est formé des arêtes $\{Y, \text{prédécesseur}(Y)\}$ pour tout $Y \neq S$. C'est un arbre couvrant de la composante connexe de S .
5. Si à la fin des sommets ne sont pas marqués, le graphe n'est pas connexe ; on peut relancer le BFS depuis un sommet non marqué pour explorer une autre composante.

EXERCICE RÉSOLU 11 BFS sur un réseau d'antennes – MTN/Orange

Un opérateur téléphonique teste son réseau : le graphe des antennes a pour sommets $\{S, A, B, C, D, E\}$ et pour arêtes :

$$\{S, A\}, \{S, B\}, \{A, C\}, \{B, C\}, \{B, D\}, \{C, E\}, \{D, E\}.$$

Effectuer un parcours en largeur depuis S (voisins traités par ordre alphabétique) : donner l'ordre de visite, le niveau de chaque sommet et l'arbre de parcours.

SOLUTION

On gère une file ; on note entre parenthèses le sommet qui provoque le marquage (pré-décesseur).

Étape	Action	File (tête à gauche)
Départ	Marquer S (niv. 0, préd. -), enfiler S	S
1	Défiler S ; marquer A (niv.1, préd. S), B (niv.1, préd. S) ; enfiler A, B	A, B
2	Défiler A ; voisin C non marqué $\rightarrow$ marquer C (niv.2, préd. A) ; enfiler C	B, C
3	Défiler B ; C déjà marqué ; voisin D non marqué $\rightarrow$ marquer D (niv.2, préd. B) ; enfiler D	C, D
4	Défiler C ; voisin E non marqué $\rightarrow$ marquer E (niv.3, préd. C) ; enfiler E	D, E
5	Défiler D ; E déjà marqué	E
6	Défiler E ; rien à faire	vide

Ordre de visite (défilement) : S, A, B, C, D, E .

Niveaux (distance en nombre d'arêtes à S) : Niveau 0 : S ; Niveau 1 : A, B ; Niveau 2 : C, D ; Niveau 3 : E .

Arbre de parcours (BFS) : arêtes $\{S, A\}, \{S, B\}, \{A, C\}, \{B, D\}, \{C, E\}$. (5 arêtes pour 6 sommets, connexe, sans cycle : c'est bien un arbre couvrant de la composante de S .)

Conclusion. Tous les sommets sont atteints : le réseau est connexe. L'antenne E est la plus éloignée de S , à 3 liaisons (ex : $S - A - C - E$ ou $S - B - D - E$).

11.6 Arbre couvrant de poids minimal : algorithme de Kruskal

MÉTHODE Algorithme de Kruskal (ACPM)

Donné un graphe connexe pondéré à n sommets (poids positifs) :

1. Trier toutes les arêtes par poids croissant (les écrire dans un tableau).
2. Partir d'un ensemble vide d'arêtes retenues. Parcourir la liste triée :
 - Si l'arête courante ne crée pas de cycle avec les arêtes déjà retenues $\rightarrow$ la retenir.
 - Sinon $\rightarrow$ la rejeter.
3. S'arrêter dès que $n - 1$ arêtes sont retenues.
4. Le poids minimal est la somme des poids des arêtes retenues. L'ensemble des arêtes

retenues forme un ACPM.

Astuce pour détecter le cycle : maintenir les composantes connexes formées par les arêtes retenues. Une arête crée un cycle si et seulement si ses deux extrémités appartiennent déjà à la même composante.

EXERCICE RÉSOLU 12 Kruskal : câblage à moindre coût – CAMWATER

On reprend le réseau CAMWATER : sommets $\{A, B, C, D, E\}$, arêtes pondérées (millions de FCFA) : $AB : 4, AC : 2, BC : 5, BD : 3, BE : 7, CD : 8, DE : 1$. Déterminer, par l'algorithme de Kruskal, un arbre couvrant de poids minimal et son coût total.

SOLUTION

Étape 1 : tri des arêtes par poids croissant.

$DE : 1 ; AC : 2 ; BD : 3 ; AB : 4 ; BC : 5 ; BE : 7 ; CD : 8$.

Étape 2 : sélection. Le graphe a $n = 5$ sommets, il faut retenir $n - 1 = 4$ arêtes. On suit les composantes connexes (CC) formées par les arêtes retenues.

Arête	Poids	Décision & Composantes
DE	1	Retenue. CC : $\{D, E\}$
AC	2	Retenue. CC : $\{D, E\}, \{A, C\}$
BD	3	Retenue (B isolé, D dans $\{D, E\}$). CC : $\{B, D, E\}, \{A, C\}$
AB	4	Retenue (relie $\{A, C\}$ à $\{B, D, E\}$, pas de cycle). CC : $\{A, B, C, D, E\}$

On a retenu 4 arêtes : on s'arrête. Les arêtes suivantes (BC, BE, CD) seraient rejetées (créent un cycle).

Étape 3 : coût total.

$$1 + 2 + 3 + 4 = 10 \text{ millions de FCFA.}$$

Vérification : L'arbre obtenu (arêtes DE, AC, BD, AB) contient les 5 sommets, est connexe et possède $5 - 1 = 4$ arêtes : c'est bien un arbre couvrant.

Conclusion. Le câblage de coût minimal relie les quartiers par les canalisations DE, AC, BD et AB , pour un coût total de 10 millions de FCFA.

11.7 Arbre couvrant de poids minimal : algorithme de Prim

MÉTHODE Algorithme de Prim (ACPM)

Donné un graphe connexe pondéré à n sommets (poids positifs) :

1. Choisir un sommet de départ quelconque ; l'ensemble X des sommets « connectés » ne contient que lui. L'arbre T est vide.
2. À chaque étape, parmi toutes les arêtes ayant une extrémité dans X et l'autre hors de X (arêtes **frontière**), choisir celle de poids minimal ; l'ajouter à T et ajouter son extrémité extérieure à X .
3. Répéter l'étape 2 jusqu'à ce que X contienne tous les sommets (soit $n - 1$ arêtes choisies dans T).
4. Additionner les poids des arêtes de T .

Différence avec Kruskal : Kruskal choisit globalement l'arête la plus légère sans cycle (forêt qui grandit) ; Prim fait grandir un seul arbre depuis un sommet. Les deux donnent un poids minimal identique (l'arbre peut différer).

EXERCICE RÉSOLU 13

Prim : électrification rurale – AES–SONEL/ENEO

Un projet d'électrification rurale doit relier 5 villages $\{A, B, C, D, E\}$. Les coûts des lignes possibles (millions de FCFA) sont : $AB : 4, AC : 2, BC : 5, BD : 3, BE : 7, CD : 8, DE : 1$. En partant du village A, appliquer l'algorithme de Prim et donner le coût minimal.

SOLUTION

On note X l'ensemble des villages déjà connectés. Départ : $X = \{A\}, T = \emptyset$.

Étape	Arêtes frontière (poids)	Choix & Mise à jour
1	$AB : 4, AC : 2$	Min = $AC : 2$. $T = \{AC\}, X = \{A, C\}$
2	$AB : 4, BC : 5, CD : 8$	Min = $AB : 4$. $T = \{AC, AB\}, X = \{A, B, C\}$
3	$BD : 3, BE : 7, CD : 8$ (BC interne)	Min = $BD : 3$. $T = \{AC, AB, BD\}, X = \{A, B, C, D\}$
4	$BE : 7, DE : 1$	Min = $DE : 1$. $T = \{AC, AB, BD, DE\}, X = \{A, B, C, D, E\}$

Coût total : $2 + 4 + 3 + 1 = 10$ millions de FCFA.

Conclusion. L'arbre couvrant de poids minimal est formé des lignes AC, AB, BD, DE , pour un coût de 10 millions de FCFA. On retrouve le même poids minimal qu'avec Kruskal (l'arbre lui-même est ici identique, mais en général il peut différer tout en ayant le même poids optimal).

11.8 Plus court chemin : algorithme de Dijkstra

MÉTHODE Algorithme de Dijkstra (Plus court chemin)

Donné un graphe pondéré connexe à poids positifs (ou nuls), un sommet de départ S et un sommet d'arrivée T :

1. Construire un tableau : une ligne par sommet, une colonne par étape. Initialiser : S reçoit la marque 0 (fixée), tous les autres sommets $+\infty$ (non fixés).
2. Tant que T n'est pas fixé :
 - (a) Parmi les sommets non fixés, choisir celui de plus petite marque ; le fixer (souligner, encadrer...). Sa marque devient définitive = distance minimale depuis S .
 - (b) Depuis ce sommet X (marque m), examiner ses voisins Y non fixés : si $m + p(X,Y) <$ marque actuelle de Y , remplacer la marque de Y par $m + p(X,Y)$ et noter la provenance : « $m + p(X,Y)$ (X) ».
3. Quand T est fixé, sa marque est la longueur du plus court chemin.
4. Remonter les provenances depuis T jusqu'à S pour obtenir le chemin inverse, puis l'écrire dans le bon ordre $S \rightarrow \dots \rightarrow T$.

ATTENTION Conditions d'application de Dijkstra

- Graphe connexe (au moins entre S et T).
- Poids positifs ou nuls (pas de poids négatif, sinon l'algorithme échoue).
- Ne jamais modifier la marque d'un sommet déjà fixé.
- Noter systématiquement la provenance pour pouvoir remonter le chemin.

EXERCICE RÉSOLU 14 Dijkstra : itinéraire Douala–Kribi

Un transporteur veut aller de D (Douala) à K (Kribi) en minimisant la distance. Le graphe des distances (km) est : $DA : 60, DB : 90, AB : 20, AE : 50, BE : 40, BK : 130, EK : 70, DK : 170$. Appliquer l'algorithme de Dijkstra et conclure.

SOLUTION

Vérification préalable : Graphe connexe, poids positifs $\rightarrow$ Dijkstra applicable.
Tableau des marques (le sommet fixé à chaque étape est souligné) :

D	A	B	E	K	Sommet fixé
<u>0</u>	$+\infty$	$+\infty$	$+\infty$	$+\infty$	D
	<u>60</u> (D)	90 (D)	$+\infty$	170 (D)	A
		<u>80</u> (A)	110 (A)	170 (D)	B
			<u>110</u> (A)	170 (D)	E
				<u>170</u> (D)	K

Détail des mises à jour (traçabilité) :

- Depuis D (marque 0, fixé) : $A \leftarrow 0+60 = 60(D)$; $B \leftarrow 0+90 = 90(D)$; $K \leftarrow 0+170 = 170(D)$.
- Plus petite marque non fixée : A (60). On fixe A . Depuis A (marque 60) : $B : 60+20 = 80 < 90 \rightarrow B \leftarrow 80(A)$; $E \leftarrow 60+50 = 110(A)$.
- Plus petite marque non fixée : B (80). On fixe B . Depuis B (marque 80) : $E : 80+40 = 120 > 110$ (on garde 110); $K : 80+130 = 210 > 170$ (on garde 170).
- Plus petite marque non fixée : E (110). On fixe E . Depuis E (marque 110) : $K : 110+70 = 180 > 170$ (on garde 170).
- Plus petite marque non fixée : K (170). On fixe K . Algorithme terminé.

Remontée du chemin : K a pour provenance (D) (marque 170 venant de D). Le chemin est donc $D \rightarrow K$ directement.

Conclusion. Le plus court chemin de Douala à Kribi est la route directe $D - K$, de longueur 170 km. Tout itinéraire passant par A , B ou E est plus long (le meilleur détour, $D - A - E - K$, vaut 180 km; $D - A - B - K$ vaut 210 km).

11.9 Synthèse : modélisation complète et résolution

EXERCICE RÉSOLU 15

Situation complète : Réseau fibre optique – CAMTEL

CAMTEL veut déployer la fibre optique entre 6 localités P, Q, R, S, T, U . Les tronçons possibles et leurs coûts (millions FCFA) sont : $PQ : 5$, $PR : 8$, $PS : 10$, $QR : 6$, $QT : 7$, $RS : 4$, $RT : 3$, $RU : 9$, $ST : 2$, $TU : 5$.

1. Modéliser par un graphe (préciser type, sommets, arêtes, poids).
2. Vérifier la connexité.
3. Déterminer un arbre couvrant de poids minimal (ACPM) par Kruskal et par Prim (départ P). Comparer.
4. Trouver le plus court chemin (coût minimal) de P à U par Dijkstra.

SOLUTION

1. Modélisation. Graphe non orienté pondéré (liaisons bidirectionnelles, coûts). Sommets : $S = \{P, Q, R, S, T, U\}$ ($n = 6$). Arêtes pondérées : $\{P, Q\} : 5, \{P, R\} : 8, \{P, S\} : 10, \{Q, R\} : 6, \{Q, T\} : 7, \{R, S\} : 4, \{R, T\} : 3, \{R, U\} : 9, \{S, T\} : 2, \{T, U\} : 5$.

2. Connexité. Depuis $P : P \rightarrow Q, R, S; Q \rightarrow T; R \rightarrow U; S \rightarrow T$ (déjà vu); $T \rightarrow U$ (déjà vu). Tous les 6 sommets atteints. Graphe connexe.

3. ACPM par Kruskal. Tri croissant : $ST : 2, RT : 3, RS : 4, PQ : 5, TU : 5, QR : 6, QT : 7, PR : 8, RU : 9, PS : 10$. Besoin de $n - 1 = 5$ arêtes.

- $ST(2)$: retenue. CC : $\{S, T\}$
- $RT(3)$: retenue. CC : $\{R, S, T\}$
- $RS(4)$: rejetée (crée cycle $R - S - T - R$).
- $PQ(5)$: retenue. CC : $\{R, S, T\}, \{P, Q\}$
- $TU(5)$: retenue (U isolé, T dans $\{R, S, T\}$). CC : $\{R, S, T, U\}, \{P, Q\}$
- $QR(6)$: retenue (relie $\{P, Q\}$ à $\{R, S, T, U\}$). CC : $\{P, Q, R, S, T, U\}$.

5 arêtes retenues : ST, RT, PQ, TU, QR . Poids total = $2 + 3 + 5 + 5 + 6 = 21$.

ACPM par Prim (départ P). $X = \{P\}$. Frontière : $PQ : 5, PR : 8, PS : 10$. Min $PQ : 5$. $X = \{P, Q\}$, $T = \{PQ\}$. $X = \{P, Q\}$. Frontière : $PR : 8, PS : 10, QR : 6, QT : 7$. Min $QR : 6$. $X = \{P, Q, R\}$, $T = \{PQ, QR\}$. $X = \{P, Q, R\}$. Frontière : $PR : 8, PS : 10, RT : 3, RU : 9, QT : 7$. Min $RT : 3$. $X = \{P, Q, R, T\}$, $T = \{PQ, QR, RT\}$. $X = \{P, Q, R, T\}$. Frontière : $PS : 10, RU : 9, ST : 2, TU : 5, QT : 7$. Min $ST : 2$. $X = \{P, Q, R, S, T\}$, $T = \{PQ, QR, RT, ST\}$. $X = \{P, Q, R, S, T\}$. Frontière : $RU : 9, TU : 5$. Min $TU : 5$. $X = \{P, Q, R, S, T, U\}$, $T = \{PQ, QR, RT, ST, TU\}$. Poids total = $5 + 6 + 3 + 2 + 5 = 21$. Comparaison : Même poids minimal (21). Arbres différents : Kruskal a ST, RT, PQ, TU, QR ; Prim a PQ, QR, RT, ST, TU . Ici ils coïncident (mêmes arêtes). En général, poids identiques, arbres possiblement différents.

4. Plus court chemin $P \rightarrow U$ (Dijkstra).

P	Q	R	S	T	U	Fixé
<u>0</u>	∞	∞	∞	∞	∞	P
	<u>5(P)</u>	8(P)	10(P)	∞	∞	Q
		<u>8(P)</u>	10(P)	12(Q)	∞	R
			<u>10(P)</u>	11(R)	17(R)	S
				<u>11(R)</u>	16(T)	T
					<u>16(T)</u>	U

Traçabilité : $P(0) \rightarrow Q(5) \rightarrow R(8) \rightarrow S(10) \rightarrow T(11 \text{ via } R) \rightarrow U(16 \text{ via } T)$. Chemin : $P \rightarrow Q \rightarrow R \rightarrow T \rightarrow U$ (ou $P \rightarrow R \rightarrow T \rightarrow U$ même coût 16? Vérif : $P - R(8) + R - T(3) + T - U(5) = 16$. Oui, deux plus courts chemins). Coût minimal : 16 millions FCFA.

ATTENTION Les 5 erreurs qui coûtent des points au Bac

1. Oublier de justifier la connexité avant Kruskal, Prim ou Dijkstra. Ces algorithmes supposent un graphe connexe (et des poids positifs pour Dijkstra). Annoncer et vérifier cette hypothèse fait partie de la rédaction attendue.
2. Chez Kruskal, retenir une arête qui crée un cycle. Erreur classique : ne regarder que le poids. Avant de retenir une arête, toujours contrôler que ses deux extrémités ne sont pas déjà dans la même composante des arêtes retenues. Et s'arrêter exactement à $n - 1$ arêtes.
3. Chez Dijkstra, modifier la marque d'un sommet déjà fixé. Une marque fixée est définitive : on ne met à jour que les sommets non fixés. Autre piège : oublier de noter la provenance, ce qui rend impossible la remontée du chemin à la fin.
4. Confondre la longueur du plus court chemin et le chemin lui-même. La question « déterminer le plus court chemin » exige les deux : la distance totale et la liste ordonnée des sommets, obtenue en remontant les provenances depuis l'arrivée puis en réécrivant le chemin dans le sens départ $\rightarrow$ arrivée.
5. Modélisation bâclée. Ne pas préciser ce que représentent sommets, arêtes et poids, ou choisir un graphe orienté quand la relation est symétrique (et inversement), fausse toute la suite. Une légende claire du graphe est une compétence évaluée à part entière.

12 EXERCICES

13 THÉORIE DES GRAPHS : ORGANISATION ET GESTION DES DONNÉES

13.1 Graphes : définitions et modélisation

DÉFINITION Graphe, sommets, arêtes

Un **graphe** non orienté G est un couple (V, E) où :

- V est un ensemble fini non vide, dont les éléments sont appelés **sommets** (ou nœuds) ;
- E est un ensemble de paires non ordonnées de sommets distincts, appelées **arêtes** (ou liens). Une arête reliant u et v se note $\{u, v\}$ ou uv .

Si les arêtes sont des paires ordonnées (u, v) , on parle de **graphe orienté** (ou **digraphe**) ; les arêtes sont alors des **arcs** notés $u \rightarrow v$. Un graphe est **simple** s'il ne contient ni boucle (arête $\{v, v\}$), ni arêtes multiples entre le même couple de sommets. Dans tout le cours, les graphes sont supposés simples sauf mention contraire.

DÉFINITION Graphe pondéré

Un **graphe pondéré** est un graphe muni d'une fonction $w : E \rightarrow \mathbb{R}_+$ (ou $\mathbb{R}$) qui associe un **poids** (ou **coût**, **longueur**) à chaque arête. Le poids d'un sous-graphe est la somme des poids de ses arêtes.

EXEMPLE Modélisation : réseau routier interurbain

On souhaite modéliser les liaisons routières entre cinq villes du Cameroun : Yaoundé (Y), Douala (D), Bafoussam (B), Garoua (G), Maroua (M).

- Sommets : $\{Y, D, B, G, M\}$.
- Arêtes : $\{Y, D\}, \{Y, B\}, \{B, G\}, \{G, M\}, \{D, B\}$ (liaisons bidirectionnelles).
- Poids : distance en km ou coût du trajet en FCFA. Exemple : $w(Y, D) = 300$, $w(Y, B) = 280$, etc.

C'est un graphe non orienté, pondéré, connexe.

EXEMPLE Modélisation : sens uniques dans un quartier

Un quartier de Douala avec des rues à sens unique.

- Sommets : carrefours A, B, C, D .
- Arcs : $A \rightarrow B, B \rightarrow C, C \rightarrow A, C \rightarrow D, D \rightarrow B$.

C'est un graphe orienté non pondéré.

DÉFINITION Sous-graphe

Soit $G = (V, E)$ un graphe. Un **sous-graphe** de G est un graphe $G' = (V', E')$ tel que $V' \subseteq V$ et $E' \subseteq E \cap \{\{u, v\} \mid u, v \in V'\}$. Un sous-graphe **induit** par $V' \subseteq V$ a pour arêtes toutes les arêtes de G reliant deux sommets de V' .

À RETENIR Vocabulaire de base

- Ordre d'un graphe : nombre de sommets $|V|$.
- Taille : nombre d'arêtes $|E|$.
- Degré d'un sommet v (graphe non orienté) : nombre d'arêtes incidentes à v , noté $\deg(v)$ ou $d(v)$.
- Degré entrant/sortant (graphe orienté) : nombre d'arcs arrivant à / partant de v .

13.2 Chaînes, chemins, cycles et connexité**DÉFINITION** Chaîne, chemin, cycle

Dans un graphe non orienté :

- Une **chaîne** de longueur k est une suite de sommets $v_0, v_1, \dots, v_k$ telle que $\{v_{i-1}, v_i\} \in E$ pour tout $i = 1..k$. Sa **longueur** est k (nombre d'arêtes).
- Un **chemin** est une chaîne sans répétition de sommet (sauf éventuellement $v_0 = v_k$).
- Un **cycle** est un chemin de longueur ≥ 3 tel que $v_0 = v_k$.

Dans un graphe orienté, on parle de **chaîne orientée**, **chemin orienté**, **cycle orienté** en suivant le sens des arcs.

PROPRIÉTÉ Lemme des poignées de main

Dans tout graphe non orienté, la somme des degrés de tous les sommets est égale au double du nombre d'arêtes :

$$\sum_{v \in V} \deg(v) = 2|E|.$$

Conséquence : Le nombre de sommets de degré impair est toujours pair.

DÉFINITION Graphe connexe, composantes connexes

Un graphe non orienté est **connexe** s'il existe une chaîne entre n'importe quels deux sommets. Une **composante connexe** est un sous-graphe connexe maximal (non contenu dans un sous-graphe connexe plus grand).

ATTENTION Pièges fréquents

- Ne pas confondre chaîne (sommets peuvent se répéter) et chemin (sommets distincts).
- Un graphe orienté peut être « faiblement connexe » (connexe si on ignore les orientations) sans être « fortement connexe » (chemin orienté dans les deux sens entre tout couple).
- Le lemme des poignées de main ne s'applique qu'aux graphes non orientés.

13.3 Arbres et arbres couvrants

DÉFINITION Arbre

Un **arbre** est un graphe non orienté connexe et sans cycle.

PROPRIÉTÉ Caractérisations de l'arbre

Pour un graphe non orienté G à n sommets, les propriétés suivantes sont équivalentes :

1. G est un arbre.
2. G est connexe et a exactement $n - 1$ arêtes.
3. G est sans cycle et a exactement $n - 1$ arêtes.

4. Il existe un unique chemin entre n'importe quels deux sommets de G .
5. G est connexe, mais la suppression de n'importe quelle arête le rend non connexe (arête **isthme** ou **pont**).

EXPÉRIENCE Démonstration : tout arbre à n sommets a $n - 1$ arêtes

Par récurrence sur n .

- Initialisation ($n = 1$) : L'arbre réduit à un sommet isolé a $0 = 1 - 1$ arête. Vrai.
- Hérédité : Soit T un arbre à $k + 1$ sommets ($k \geq 1$). Tout arbre fini non trivial a au moins deux **feuilles** (sommets de degré 1). Supprimons une feuille f et son arête incidente. Le graphe obtenu T' est connexe (le chemin entre deux sommets restants ne passait pas par f) et sans cycle (sous-graphe d'un acyclique), donc c'est un arbre à k sommets. Par hypothèse de récurrence, T' a $k - 1$ arêtes. T a donc $(k - 1) + 1 = k = (k + 1) - 1$ arêtes.

Conclusion : La propriété est vraie pour tout $n \geq 1$.

DÉFINITION Arbre couvrant

Soit $G = (V, E)$ un graphe connexe. Un **arbre couvrant** (ou **arbre générateur**) de G est un sous-graphe $T = (V, E')$ qui est un arbre (donc $E' \subseteq E$, $|E'| = |V| - 1$, T connexe et acyclique).

PROPRIÉTÉ Existence d'un arbre couvrant

Tout graphe connexe possède au moins un arbre couvrant. Preuve (constructive) : Tant que le graphe contient un cycle, supprimer une arête de ce cycle. La connexité est préservée (l'autre chemin du cycle reste), le nombre d'arêtes diminue. Le processus s'arrête sur un arbre couvrant.

13.4 Parcours en largeur (BFS)

MÉTHODE Algorithme de parcours en largeur (BFS) depuis un sommet s

Données : Graphe non orienté $G = (V, E)$, sommet de départ $s \in V$. Résultats : Pour chaque sommet v accessible depuis s : $\text{dist}(v)$ (distance = longueur plus court chemin en nombre d'arêtes) et $\text{pred}(v)$ (prédécesseur dans l'arbre BFS).

1. Initialiser une file $F \leftarrow [s]$.

2. Marquer s comme visité ; $\text{dist}(s) \leftarrow 0$; $\text{pred}(s) \leftarrow \text{Nil}$.

3. Tant que F n'est pas vide :

(a) Extraire $u \leftarrow \text{défiler}(F)$.

(b) Pour chaque voisin v de u non visité :

- Marquer v visité.
- $\text{dist}(v) \leftarrow \text{dist}(u) + 1$.
- $\text{pred}(v) \leftarrow u$.
- Enfiler v dans F .

Complexité : $O(|V| + |E|)$ avec listes d'adjacence.

PROPRIÉTÉ Propriétés du BFS

- L'ensemble des arêtes $\{\text{pred}(v), v \mid v \neq s\}$ forme un arbre couvrant de la composante connexe de s (appelé **arbre BFS**).
- Pour tout sommet v atteint, $\text{dist}(v)$ est la distance (longueur minimale en arêtes) de s à v .
- Le chemin obtenu en remontant les prédécesseurs depuis v jusqu'à s est un plus court chemin (en nombre d'arêtes) de s à v .

EXEMPLE BFS sur un graphe exemple

Graphe : $V = \{A, B, C, D, E, F\}$, $E = \{AB, AC, BD, BE, CF, DE, EF\}$. Départ A .

Sommet défilé	File (après)	Sommet visité (dist, pred)	Arête arbre
A	B, C	A(0, -), B(1, A), C(1, A)	AB, AC
B	C, D, E	D(2, B), E(2, B)	BD, BE
C	D, E, F	F(2, C)	CF
D	E, F	-	-
E	F	-	-
F	∅	-	-

Arbre BFS : $\{AB, AC, BD, BE, CF\}$. Distances : $d(A, F) = 2$ (chemin $A - C - F$).

13.5 Arbre couvrant de poids minimal : Kruskal et Prim

DÉFINITION Arbre couvrant de poids minimal (ACPM)

Soit $G = (V, E, w)$ un graphe connexe pondéré (poids positifs). Un **arbre couvrant de poids minimal** (ACPM ou MST en anglais) est un arbre couvrant T tel que $\sum_{e \in T} w(e)$ soit minimal parmi tous les arbres couvrants de G .

MÉTHODE Algorithme de Kruskal

Idée : Construire l'ACPM en ajoutant les arêtes les plus légères une à une, sans créer de cycle.

1. Trier toutes les arêtes par poids croissant : $e_1, e_2, \dots, e_m$.
2. Initialiser la forêt $F \leftarrow \emptyset$ (chaque sommet est un arbre à part).
3. Pour $i = 1$ à m :
 - Si l'ajout de e_i à F ne crée pas de cycle (i.e. ses extrémités sont dans deux composantes distinctes de F), alors ajouter e_i à F .
4. Arrêter quand F a $|V| - 1$ arêtes (c'est un arbre couvrant).

Note : La détection de cycle se fait efficacement avec une structure Union-Find (disjoint sets).

MÉTHODE Algorithme de Prim

Idée : Faire grandir un unique arbre à partir d'un sommet de départ, en ajoutant à chaque étape l'arête de poids minimal reliant l'arbre à un sommet extérieur.

1. Choisir un sommet de départ s . $A \leftarrow \{s\}$ (sommet atteints), $T \leftarrow \emptyset$ (arêtes de l'arbre).
2. Tant que $|A| < |V|$:
 - Trouver l'arête $e = \{u, v\}$ de poids minimal telle que $u \in A$ et $v \notin A$.
 - Ajouter v à A et e à T .

Complexité : $O(|E| \log |V|)$ avec file de priorité (tas min).

PROPRIÉTÉ Optimalité et unicité

- Kruskal et Prim produisent toujours un ACPM (preuve par échange / coupe).

- Si tous les poids sont deux à deux distincts, l'ACPM est unique.
- S'il y a des poids égaux, il peut y avoir plusieurs ACPM (même poids total, structure différente). L'ordre de tri dans Kruskal ou le sommet de départ dans Prim peut en sélectionner un différent.

ATTENTION Conditions d'application

- Le graphe doit être connexe (sinon on obtient une forêt couvrante de poids minimal, un ACPM par composante).
- Les poids doivent être comparables (réels). Pas de condition de positivité stricte pour Kruskal/Prim (fonctionne avec poids négatifs), mais les poids sont souvent des coûts/longueurs ≥ 0 .

13.6 Plus court chemin : algorithme de Dijkstra

MÉTHODE Algorithme de Dijkstra (plus courts chemins depuis une source s)

Condition : Graphe pondéré, poids positifs ou nuls ($w(e) \geq 0$). Fonctionne pour graphes orientés ou non. Principe : Étiquettes provisoires / définitives (méthode gloutonne).

1. Initialisation : $\text{dist}(s) = 0$ (définitive, notée $\bullet$). $\forall v \neq s, \text{dist}(v) = +\infty$ (provisoire). $\text{pred}(v) = \text{Nil}$.
2. Tant qu'il existe des étiquettes provisoires :
 - Choisir le sommet u non traité dont l'étiquette $\text{dist}(u)$ est minimale. La rendre définitive ($\bullet$).
 - Relâchement (relaxation) : Pour chaque voisin v de u non définitif :

$$\text{si } \text{dist}(u) + w(u,v) < \text{dist}(v) \quad \text{alors} \quad \text{dist}(v) \leftarrow \text{dist}(u) + w(u,v), \text{pred}(v) \leftarrow u.$$

À la fin, $\text{dist}(v)$ est la distance minimale $d(s,v)$ et le chemin se reconstruit par les prédécesseurs.

ATTENTION Pièges Dijkstra

- Ne fonctionne PAS si il y a des poids négatifs (risque de boucler ou de figer une étiquette trop grande). Pour poids négatifs : Bellman-Ford.

- L'étiquette définitive est jamais modifiée par la suite (propriété de l'algorithme glouton valide car poids ≥ 0).
- Bien distinguer provisoire (peut encore baisser) et définitive (valeur finale).

LES AVAIS-TU ? Applications réelles au Cameroun

- Transport : Optimisation des tournées de camions (SODECAO, CDC), planification vols Camair-Co (Dijkstra pour temps/coût minimal).
- Télécoms : Réseau fibre optique CAMTEL / MTN / Orange (Kruskal/Prim pour câblage minimal), routage paquets IP (Dijkstra/OSPF).
- Énergie : Réseau électrique ENEO / électrification rurale (arbres couvrants).
- Logistique : Livraison dernier kilomètre (Jumia, Glovo) → Problème du voyageur de commerce (TSP), approximation par arbres.

13.7 Synthèse et modélisation de situations

13.8 Je vérifie mes connaissances

EXERCICE 6 QCM : vocabulaire des graphes

Pour chaque question, une seule réponse est exacte. Recopie la bonne réponse sur ta copie.

1. Un graphe non orienté possède 5 sommets de degrés respectifs 3, 3, 2, 2, 4. Le nombre d'arêtes de ce graphe est :
a) 7
b) 14
c) 28
2. Un arbre à 10 sommets possède :
a) 10 arêtes
b) 9 arêtes
c) 11 arêtes
3. L'algorithme de Dijkstra s'applique à un graphe pondéré dont les poids sont :
a) quelconques
b) tous positifs ou nuls
c) tous entiers
4. Un graphe connexe à n sommets possède au minimum :
a) n arêtes
b) $n - 1$ arêtes
c) $\frac{n(n-1)}{2}$ arêtes

EXERCICE 7 Vrai ou faux, en justifiant

Pour chaque affirmation, dire si elle est vraie ou fausse et justifier la réponse par une démonstration courte ou un contre-exemple.

1. Tout graphe connexe contient au moins un arbre couvrant.
2. Tout graphe sans cycle est un arbre.
3. Dans un graphe non orienté, la somme des degrés des sommets est toujours un nombre pair.
4. L'algorithme de Kruskal fournit toujours le même arbre couvrant, quel que soit l'ordre de tri des arêtes de même poids.

EXERCICE 8 Définitions et exemples

1. Définir les termes suivants : **chaîne**, **cycle**, **graphe connexe**, **arbre couvrant**.
2. Donner un exemple de graphe à 4 sommets qui est connexe mais qui n'est pas un arbre (on donnera la liste des arêtes).
3. Donner un exemple de graphe à 5 sommets non connexe, et préciser ses composantes connexes.

EXERCICE 9 Degrés et lemme des poignées de main

Un graphe non orienté G est donné par sa liste d'arêtes :

$$\{A,B\}, \{A,C\}, \{A,D\}, \{B,C\}, \{B,E\}, \{C,E\}, \{D,E\}.$$

1. Déterminer le degré de chaque sommet de G .
2. Calculer la somme des degrés et vérifier le lemme des poignées de main.
3. Le graphe G admet-il une chaîne passant par toutes les arêtes une et une seule fois ? Justifier.

13.9 Je m'entraîne**EXERCICE 10** Connexité et parcours en largeur

Soit G le graphe non orienté de sommets $\{A, B, C, D, E, F, G\}$ et d'arêtes :

$$\{A,B\}, \{A,D\}, \{B,C\}, \{B,E\}, \{C,F\}, \{D,E\}, \{E,F\}, \{F,G\}.$$

1. Exécuter un parcours en largeur (BFS) depuis le sommet A et rédiger le tableau de suivi (sommet visité, file d'attente, prédécesseur).
2. En déduire que G est connexe.
3. Donner la distance (nombre minimal d'arêtes) de A à chaque sommet, puis un chemin de longueur minimale de A à G .
4. Extraire de ce parcours un arbre couvrant de G .

EXERCICE 11 Kruskal pas à pas

Un graphe pondéré G a pour sommets $\{S_1, S_2, S_3, S_4, S_5, S_6\}$ et pour arêtes pondérées :

Arête	S_1S_2	S_1S_3	S_2S_3	S_2S_4	S_3S_5	S_4S_5	S_4S_6	S_5S_6	S_1S_6
Poids	4	2	5	3	7	6	1	8	9

1. Trier les arêtes par poids croissant dans un tableau.
2. Appliquer l'algorithme de Kruskal : pour chaque arête examinée, indiquer si elle est retenue ou rejetée (cycle détecté).
3. Donner la liste des arêtes de l'arbre couvrant minimal et calculer son poids total.

EXERCICE 12 Prim depuis deux sommets différents

On reprend le graphe pondéré de l'exercice précédent.

1. Appliquer l'algorithme de Prim en partant du sommet S_1 : rédiger le tableau de suivi (sommets atteints, arête choisie à chaque étape).
2. Appliquer l'algorithme de Prim en partant du sommet S_4 .
3. Comparer les arbres obtenus et leurs poids totaux. Le résultat était-il prévisible ? Justifier.

EXERCICE 13 Sens uniques d'un quartier

Un quartier de Yaoundé est modélisé par le graphe orienté suivant : les sommets sont les

carrefours A, B, C, D, E, F et les arcs (sens uniques) sont :

$$A \rightarrow B, A \rightarrow C, B \rightarrow D, C \rightarrow D, C \rightarrow E, D \rightarrow F, E \rightarrow F, F \rightarrow A.$$

1. Représenter ce graphe orienté.
2. Exécuter un parcours en largeur depuis le sommet A et déterminer l'ensemble des sommets atteignables depuis A .
3. Déterminer un chemin de longueur minimale de A à F et préciser sa longueur.
4. Un conducteur se trouve en D . Peut-il atteindre tous les carrefours du quartier ? Justifier.

13.10 Je me prépare au Bac

EXERCICE 14 Type Bac : liaisons aériennes en Afrique

Une compagnie aérienne dessert six villes africaines : Douala (D), Yaoundé (Y), Libreville (L), Kinshasa (K), Lagos (G) et Nairobi (N). Le tableau ci-dessous donne les prix des billets (en milliers de FCFA) pour les liaisons directes existantes ; une case vide signifie qu'il n'existe pas de liaison directe.

	D	Y	L	K	G	N
D	-	25	40	60	55	
Y	25	-	35		70	
L	40	35	-	30		120
K	60		30	-	45	80
G	55	70		45	-	90
N			120	80	90	-

1. Représenter cette situation par un graphe pondéré dont on précisera les sommets et les arêtes.
2. Ce graphe est-il connexe ? Justifier.
3. Un homme d'affaires veut voyager de Douala à Nairobi au moindre coût.
 - (a) Appliquer l'algorithme de Dijkstra depuis le sommet D : rédiger le tableau complet de suivi (sommets traités, étiquettes provisoires et définitives, prédécesseurs).
 - (b) En déduire le prix minimal du voyage et l'itinéraire correspondant.
4. La compagnie supprime la liaison directe Kinshasa–Nairobi. Déterminer le nouvel itinéraire le moins cher de Douala à Nairobi et le surcoût éventuel.

EXERCICE 15 Type Bac : réseau de fibres optiques entre lycées

Dans le cadre du programme d'électrification numérique, CAMTEL souhaite raccorder huit lycées d'une ville, notés $L_1, L_2, \dots, L_8$, par un réseau de fibres optiques. Les tronçons possibles et leurs coûts de pose (en millions de FCFA) sont donnés ci-dessous :

Tronçon	L_1L_2	L_1L_4	L_2L_3	L_2L_5	L_3L_6	L_4L_5	L_4L_7	L_5L_6	L_5L_8	L_6L_8	L_7L_8
Coût	3	6	2	4	5	1	7	8	3	4	2

Tous les lycées doivent pouvoir communiquer entre eux, directement ou par l'intermédiaire d'autres lycées, et le coût total de pose doit être minimal.

1. Modéliser la situation par un graphe pondéré dont on précisera la nature (sommets, arêtes, poids).
2. Expliquer pourquoi le réseau cherché est un arbre couvrant de poids minimal de ce graphe.
3. Appliquer l'algorithme de Kruskal : dresser le tableau des arêtes triées par coût croissant, en indiquant pour chacune si elle est retenue ou rejetée (cycle détecté).
4. Donner la liste des tronçons à construire et calculer le coût minimal de raccordement.
5. Reprendre la question avec l'algorithme de Prim à partir du lycée L_1 et vérifier que le coût minimal est le même.
6. Le tronçon L_4L_5 devient indisponible pour des raisons techniques. Déterminer le nouveau coût minimal.

EXERCICE 16 Type Bac : synthèse autour des arbres et tableau de Dijkstra

Partie A : démonstrations.

1. Démontrer par récurrence que tout arbre à n sommets ($n \geq 1$) possède exactement $n - 1$ arêtes.
2. En déduire que tout graphe connexe à n sommets possède au moins $n - 1$ arêtes.
3. Un graphe connexe possède 12 sommets et 11 arêtes. Montrer que c'est un arbre.

Partie B : tableau de Dijkstra à compléter. Sur un graphe pondéré de sommets $\{A, B, C, D, E\}$, on a exécuté l'algorithme de Dijkstra depuis le sommet A. Le tableau de suivi ci-dessous est incomplet (les étiquettes sont de la forme « distance (prédécesseur) » ; le symbole • désigne une étiquette devenue définitive) :

Étape	A	B	C	D	E
Initialisation	0 •	∞	∞	∞	∞
1		4 (A) •	2 (A)	∞	7 (A)
2			2 (A) •	9 (C)	5 (C)
3				?	5 (C) •
4				8 (E) •	

1. Expliquer pourquoi, à l'étape 2, c'est le sommet C dont l'étiquette devient définitive.
2. Justifier la valeur 9 (C) de l'étiquette de D à l'étape 2, puis compléter la case manquante de l'étape 3.
3. Déterminer la distance minimale de A à D et reconstituer, à l'aide des prédécesseurs, le chemin optimal de A à D.
4. Reconstituer la liste des arêtes pondérées du graphe (on admettra que chaque amélioration d'étiquette correspond à une arête du graphe).

14 CORRIGÉS DES EXERCICES

CORRIGÉ Exercice 1 — QCM : vocabulaire des graphes

1. Réponse a) : 7 arêtes.

D'après le **lemme des poignées de main**, la somme des degrés vaut le double du nombre d'arêtes :

$$\sum_{v \in V} \deg(v) = 3 + 3 + 2 + 2 + 4 = 14 = 2|E| \quad \Rightarrow \quad |E| = \frac{14}{2} = 7.$$

2. Réponse b) : 9 arêtes.

Propriété fondamentale des arbres : tout arbre à n sommets possède exactement $n - 1$ arêtes. Ici $n = 10$, donc $10 - 1 = 9$ arêtes.

3. Réponse b) : tous positifs ou nuls.

L'algorithme de Dijkstra exige $w(e) \geq 0$ pour toute arête : c'est cette hypothèse qui garantit qu'une étiquette devenue définitive ne pourra plus être améliorée. Ni des poids quelconques (a), ni la seule intégrité des poids (c) ne suffisent.

4. Réponse b) : $n - 1$ arêtes.

Tout graphe connexe contient un arbre couvrant, lequel possède exactement $n - 1$ arêtes. Un graphe connexe a donc au moins $n - 1$ arêtes, et ce minimum est atteint exactement lorsque le graphe est lui-même un arbre.

CORRIGÉ Exercice 2 — Vrai ou faux, en justifiant

1. Vrai. C'est une propriété du cours, démontrée de façon constructive : si G est connexe et contient un cycle, on supprime une arête de ce cycle ; la connexité est préservée car les deux extrémités de l'arête supprimée restent reliées par le reste du cycle. On recommence tant qu'il reste un cycle. Le processus s'arrête (le nombre d'arêtes diminue strictement à chaque étape) sur un graphe connexe, sans cycle, couvrant tous les sommets : c'est un arbre couvrant.
2. Faux. Il manque l'hypothèse de connexité. Un graphe sans cycle est une forêt (réunion disjointe d'arbres) ; il n'est un arbre que s'il est en outre connexe. Contre-exemple : le graphe $V = \{1, 2\}$, $E = \emptyset$ (deux sommets isolés) est sans cycle mais n'est pas connexe, donc n'est pas un arbre.
3. Vrai. Par le lemme des poignées de main : $\sum_{v \in V} \deg(v) = 2|E|$, qui est un nombre pair (multiple de 2).
4. Faux. En cas d'ex æquo sur les poids, l'ordre de départage influence l'arbre obtenu.

Contre-exemple : le triangle ABC dont les trois arêtes ont poids 1. Kruskal retient deux arêtes quelconques parmi les trois : trois arbres couvrants distincts sont possibles ($\{AB, BC\}$, $\{AB, AC\}$ ou $\{AC, BC\}$), tous de poids total 2. En revanche, le poids total minimal est toujours le même.

CORRIGÉ Exercice 3 — Définitions et exemples

1. Définitions :

- Chaîne : suite de sommets $v_0, v_1, \dots, v_k$ telle que deux sommets consécutifs sont reliés par une arête ; sa longueur est k (nombre d'arêtes). Les sommets peuvent se répéter.
 - Cycle : chaîne fermée ($v_0 = v_k$) de longueur ≥ 3 dont les sommets intermédiaires sont tous distincts (autrement dit, un chemin dont les extrémités coïncident).
 - Graphe connexe : graphe non orienté dans lequel il existe une chaîne entre n'importe quels deux sommets.
 - Arbre couvrant : sous-graphe d'un graphe connexe $G = (V, E)$ qui contient tous les sommets de G et qui est un arbre (connexe, sans cycle, $|V| - 1$ arêtes).
2. Exemple : le cycle $C_4 : V = \{A, B, C, D\}, E = \{\{A, B\}, \{B, C\}, \{C, D\}, \{D, A\}\}$. Ce graphe est connexe (on peut aller de tout sommet à tout autre) mais possède 4 arêtes $> 4 - 1 = 3$ et contient le cycle $A - B - C - D - A$: ce n'est pas un arbre.
3. Exemple : $V = \{1, 2, 3, 4, 5\}, E = \{\{1, 2\}, \{2, 3\}, \{4, 5\}\}$.
Composantes connexes : $C_1 = \{1, 2, 3\}$ (chaîne $1 - 2 - 3$) et $C_2 = \{4, 5\}$ (arête isolée). Le sommet 1 et le sommet 4, par exemple, ne sont reliés par aucune chaîne : le graphe est non connexe, à deux composantes connexes.

CORRIGÉ Exercice 4 — Degrés et lemme des poignées de main

1. On compte, pour chaque sommet, le nombre d'arêtes incidentes :

Sommet	Arêtes incidentes	Degré
A	AB, AC, AD	3
B	AB, BC, BE	3
C	AC, BC, CE	3
D	AD, DE	2
E	BE, CE, DE	3

2. Somme des degrés : $3 + 3 + 3 + 2 + 3 = 14$. Le graphe possède $|E| = 7$ arêtes, et $2|E| = 14$.

On a bien $\sum_v \deg(v) = 2|E|$: le lemme des poignées de main est vérifié.

3. Une chaîne passant par chaque arête exactement une fois (chaîne eulérienne) existe si et seulement si le graphe est connexe et possède 0 ou 2 sommets de degré impair. Ici le graphe est connexe, mais les sommets de degré impair sont A, B, C et E : il y en a quatre. Conclusion : non, G n'admet pas de chaîne eulérienne.

CORRIGÉ Exercice 5 — Connexité et parcours en largeur

1. Tableau de suivi du BFS depuis A (voisins examinés par ordre alphabétique) :

Sommet défilé	File après action	Nouveaux sommets visités (dist, pred)	Arêtes de l'arbre BFS
A	$[B, D]$	$A(0, -), B(1, A), D(1, A)$	AB, AD
B	$[D, C, E]$	$C(2, B), E(2, B)$	BC, BE
D	$[C, E]$	aucun (A, E déjà visités)	—
C	$[E, F]$	$F(3, C)$	CF
E	$[F]$	aucun (B, D, F déjà visités)	—
F	$[G]$	$G(4, F)$	FG
G	$[]$	aucun	—

2. Les 7 sommets du graphe ont tous été atteints par le parcours (chacun a reçu une distance finie). Il existe donc une chaîne de A vers tout sommet, et par suite entre deux sommets quelconques (en passant par A). Le graphe G est connexe.
3. Distances depuis A :

$$d(A, A) = 0, \quad d(A, B) = d(A, D) = 1, \quad d(A, C) = d(A, E) = 2, \quad d(A, F) = 3, \quad d(A, G) = 4.$$

Chemin minimal de A à G : on remonte les prédécesseurs : $G \leftarrow F \leftarrow C \leftarrow B \leftarrow A$, soit

$$A - B - C - F - G \quad (\text{longueur } 4).$$

4. Arbre couvrant extrait (arbre BFS) : arêtes $\{AB, AD, BC, BE, CF, FG\}$. Vérification : $6 = 7 - 1$ arêtes, connexe (tous les sommets atteints), sans cycle (propriété du BFS : chaque sommet n'est enfilé qu'une seule fois, donc aucune arête retenue ne peut refermer un cycle) : c'est bien un arbre couvrant de G .

CORRIGÉ Exercice 6 — Kruskal pas à pas

1. Tri des arêtes par poids croissant :

Rang	1	2	3	4	5	6	7	8	9
Arête	S_4S_6	S_1S_3	S_2S_4	S_1S_2	S_2S_3	S_4S_5	S_3S_5	S_5S_6	S_1S_6
Poids	1	2	3	4	5	6	7	8	9

2. Exécution de Kruskal (on suit les composantes connexes de la forêt en construction) :

Arête	Poids	Composantes avant examen	Décision
S_4S_6	1	$\{S_4\}, \{S_6\}$ distinctes	Retenue
S_1S_3	2	$\{S_1\}, \{S_3\}$ distinctes	Retenue
S_2S_4	3	$\{S_2\}, \{S_4, S_6\}$ distinctes	Retenue
S_1S_2	4	$\{S_1, S_3\}, \{S_2, S_4, S_6\}$ distinctes	Retenue
S_2S_3	5	S_2 et S_3 dans la même composante	Rejetée (cycle $S_1S_2S_3$)
S_4S_5	6	$\{S_1, S_2, S_3, S_4, S_6\}, \{S_5\}$ distinctes	Retenue
S_3S_5	7	même composante	Rejetée (cycle)
S_5S_6	8	même composante	Rejetée (cycle)
S_1S_6	9	même composante	Rejetée (cycle)

On a retenu $5 = 6 - 1$ arêtes : l'algorithme s'arrête.

3. Arbre couvrant de poids minimal :

$$T = \{S_4S_6, S_1S_3, S_2S_4, S_1S_2, S_4S_5\}, \quad w(T) = 1 + 2 + 3 + 4 + 6 = 16.$$

CORRIGÉ Exercice 7 — Prim depuis deux sommets différents

1. Prim depuis S_1 (à chaque étape, on choisit l'arête de poids minimal quittant l'ensemble A des sommets atteints) :

Étape	Sommets atteints A	Arêtes candidates (poids)	Choix
1	$\{S_1\}$	$S_1S_3(2), S_1S_2(4), S_1S_6(9)$	$S_1S_3(2)$
2	$\{S_1, S_3\}$	$S_1S_2(4), S_1S_6(9), S_2S_3(5), S_3S_5(7)$	$S_1S_2(4)$
3	$\{S_1, S_2, S_3\}$	$S_1S_6(9), S_3S_5(7), S_2S_4(3)$	$S_2S_4(3)$
4	$\{S_1, S_2, S_3, S_4\}$	$S_1S_6(9), S_3S_5(7), S_4S_5(6), S_4S_6(1)$	$S_4S_6(1)$
5	$\{S_1, S_2, S_3, S_4, S_6\}$	$S_3S_5(7), S_4S_5(6), S_5S_6(8)$	$S_4S_5(6)$

Arbre obtenu : $\{S_1S_3, S_1S_2, S_2S_4, S_4S_6, S_4S_5\}$, de poids $2 + 4 + 3 + 1 + 6 = 16$.

2. Prim depuis S_4 :

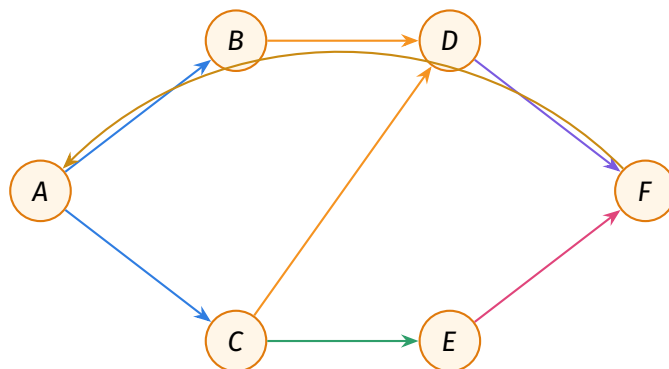
Étape	Sommets atteints A	Arêtes candidates (poids)	Choix
1	$\{S_4\}$	$S_4S_6(1), S_4S_2(3), S_4S_5(6)$	$S_4S_6(1)$
2	$\{S_4, S_6\}$	$S_4S_2(3), S_4S_5(6), S_5S_6(8), S_1S_6(9)$	$S_4S_2(3)$
3	$\{S_2, S_4, S_6\}$	$S_4S_5(6), S_5S_6(8), S_1S_6(9), S_1S_2(4), S_2S_3(5)$	$S_1S_2(4)$
4	$\{S_1, S_2, S_4, S_6\}$	$S_4S_5(6), S_5S_6(8), S_2S_3(5), S_1S_3(2)$	$S_1S_3(2)$
5	$\{S_1, S_2, S_3, S_4, S_6\}$	$S_4S_5(6), S_5S_6(8), S_3S_5(7)$	$S_4S_5(6)$

Arbre obtenu : $\{S_4S_6, S_2S_4, S_1S_2, S_1S_3, S_4S_5\}$, de poids $1 + 3 + 4 + 2 + 6 = 16$.

3. Comparaison : les deux exécutions de Prim, ainsi que Kruskal (exercice 6), fournissent exactement le même ensemble d'arêtes, donc le même poids total 16. Ce résultat était prévisible : dans ce graphe, tous les poids sont deux à deux distincts, donc l'arbre couvrant de poids minimal est unique. Ni l'algorithme choisi, ni le sommet de départ de Prim ne peuvent changer le résultat.

CORRIGÉ Exercice 8 — Sens uniques d'un quartier

1. Représentation du graphe orienté :



Graphe orienté des sens uniques du quartier.

2. BFS orienté depuis A (on ne suit que les arcs sortants) :

Sommet défilé	File après action	Nouveaux sommets visités (dist, pred)
A	[B, C]	A(0, -), B(1, A), C(1, A)
B	[C, D]	D(2, B)
C	[D, E]	E(2, C) (D déjà visité)
D	[E, F]	F(3, D)
E	[F]	aucun (F déjà visité)
F	[]	aucun (A déjà visité)

Sommets atteignables depuis A : {A, B, C, D, E, F} : tous les carrefours.

- Les chemins orientés de A à F sont : $A \rightarrow B \rightarrow D \rightarrow F$, $A \rightarrow C \rightarrow D \rightarrow F$ et $A \rightarrow C \rightarrow E \rightarrow F$, tous de longueur 3. Le BFS confirme $d(A, F) = 3$. Longueur minimale : 3 ; un chemin optimal est par exemple $A \rightarrow B \rightarrow D \rightarrow F$.
- Depuis D, on peut suivre : $D \rightarrow F \rightarrow A$, puis $A \rightarrow B$, $A \rightarrow C$, et $C \rightarrow E$. Les sommets A, B, C, E, F sont donc tous atteignables depuis D. Oui, le conducteur en D peut atteindre tous les carrefours du quartier. (On peut d'ailleurs vérifier que le graphe est fortement connexe : tout sommet est atteignable depuis tout sommet.)

CORRIGÉ Exercice 9 — Type Bac : liaisons aériennes en Afrique

Barème indicatif (sur 5 points) : modélisation 0,75 ; connexité 0,5 ; tableau de Dijkstra 1,5 ; distance et itinéraire 0,75 ; nouvel itinéraire et surcoût 1 ; rédaction et interprétation 0,5.

- Modélisation : graphe non orienté pondéré $G = (V, E, w)$ avec $V = \{D, Y, L, K, G, N\}$ (les villes) et

$$E = \{DY(25), DL(40), DK(60), DG(55), YL(35), YG(70), LK(30), LN(120), KG(45), KN(80), GN(90)\},$$

les poids étant les prix en milliers de FCFA. Le graphe est non orienté car une liaison aérienne s'emprunte dans les deux sens au même prix.

- Connexité : oui. Par exemple, depuis D on atteint directement Y, L, K, G ; depuis L on atteint N (arête LN). Toute ville est donc reliée à toute autre (au plus tard en passant par D ou L) : le graphe est connexe.
- a) Tableau de Dijkstra depuis D (étiquette = distance(provenance) ; • = définitive) :

Étape	D	Y	L	K	G	N
Init	0 •	∞	∞	∞	∞	∞
1 : D		25(D)	40(D)	60(D)	55(D)	∞
2 : Y		25 •	40(D)	60(D)	55(D)	∞
3 : L			40 •	60(D)	55(D)	160(L)
4 : G				60(D)	55 •	145(G)
5 : K				60 •		140(K)
6 : N						140 •

Détail des relâchements :

- Étape 1 (D traité) : $Y \leftarrow 25, L \leftarrow 40, K \leftarrow 60, G \leftarrow 55$.
- Étape 2 (minimum provisoire : $Y = 25$) : via $Y : L$ vaudrait $25 + 35 = 60 > 40$ (rejeté); G vaudrait $25 + 70 = 95 > 55$ (rejeté).
- Étape 3 (minimum : $L = 40$) : via $L : K$ vaudrait $40 + 30 = 70 > 60$ (rejeté); $N \leftarrow 40 + 120 = 160$.
- Étape 4 (minimum : $G = 55$) : via $G : K$ vaudrait $55 + 45 = 100 > 60$ (rejeté); $N \leftarrow 55 + 90 = 145 < 160$ (amélioré).
- Étape 5 (minimum : $K = 60$) : via $K : N \leftarrow 60 + 80 = 140 < 145$ (amélioré).
- Étape 6 : $N = 140$ devient définitive.

b) Prix minimal : 140 milliers de FCFA, soit 140 000 FCFA.

Itinéraire : on remonte les prédécesseurs : $N \leftarrow K \leftarrow D$, soit Douala $\rightarrow$ Kinshasa $\rightarrow$ Nairobi.

4. Suppression de la liaison $K-N$: les seuls accès restants à N sont $LN(120)$ et $GN(90)$. Les distances depuis D vers L et G sont inchangées (les plus courts chemins vers L et G ne passaient pas par N) : $d(D, L) = 40$ et $d(D, G) = 55$. Donc :

$$\text{via } L : 40 + 120 = 160; \quad \text{via } G : 55 + 90 = 145.$$

Le nouvel itinéraire optimal est Douala $\rightarrow$ Lagos $\rightarrow$ Nairobi pour 145 milliers de FCFA.

Surcoût : $145 - 140 = 5$ milliers de FCFA, soit 5 000 FCFA.

Points de vigilance : citer l'hypothèse « poids positifs » qui autorise Dijkstra ; ne figer qu'un seul sommet par étape (le minimum provisoire) ; justifier chaque refus d'amélioration par la comparaison explicite ; conclure par une phrase d'interprétation dans le contexte (prix en FCFA, itinéraire).

CORRIGÉ Exercice 10 — Type Bac : réseau de fibres optiques entre lycées

Barème indicatif (sur 6 points) : modélisation 0,5 ; justification « arbre couvrant minimal » 1 ; Kruskal (tri + tableau) 1,5 ; coût minimal 0,5 ; Prim 1,5 ; tronçon indisponible 1.

1. Modélisation : graphe non orienté, simple, connexe et pondéré $G = (V, E, w) : V = \{L_1, \dots, L_8\}$ (les lycées), E l'ensemble des 11 tronçons possibles, et $w(e)$ le coût de pose en millions de FCFA.
2. Pourquoi un ACPM ? « Tous les lycées doivent communiquer entre eux » signifie que le sous-graphe des tronçons construits doit être connexe et couvrir tous les sommets. Parmi tous les sous-graphes connexes couvrants, on veut le coût total minimal. Or un tel sous-graphe minimal est nécessairement sans cycle : s'il contenait un cycle, on pourrait supprimer une arête de ce cycle en conservant la connexité, ce qui ferait strictement baisser le coût (poids strictement positifs) — contradiction avec la minimalité. Un sous-graphe connexe, couvrant et sans cycle est un arbre couvrant ; le réseau cherché est donc un arbre couvrant de poids minimal.
3. Kruskal — arêtes triées par coût croissant :

Rang	1	2	3	4	5	6	7	8	9	10	11
Arête	L_4L_5	L_2L_3	L_7L_8	L_1L_2	L_5L_8	L_2L_5	L_6L_8	L_3L_6	L_1L_4	L_4L_7	L_5L_6
Coût	1	2	2	3	3	4	4	5	6	7	8

Exécution :

Arête	Coût	Composantes avant examen	Décision
L_4L_5	1	$\{L_4\}, \{L_5\}$	Retenue
L_2L_3	2	$\{L_2\}, \{L_3\}$	Retenue
L_7L_8	2	$\{L_7\}, \{L_8\}$	Retenue
L_1L_2	3	$\{L_1\}, \{L_2, L_3\}$	Retenue
L_5L_8	3	$\{L_4, L_5\}, \{L_7, L_8\}$	Retenue
L_2L_5	4	$\{L_1, L_2, L_3\}, \{L_4, L_5, L_7, L_8\}$	Retenue
L_6L_8	4	$\{L_6\}, \{L_1, L_2, L_3, L_4, L_5, L_7, L_8\}$	Retenue
L_3L_6	5	même composante	Rejetée (cycle)
L_1L_4	6	même composante	Rejetée (cycle)
L_4L_7	7	même composante	Rejetée (cycle)
L_5L_6	8	même composante	Rejetée (cycle)

On a retenu $7 = 8 - 1$ arêtes : arrêt.

4. Tronçons à construire : $\{L_4L_5, L_2L_3, L_7L_8, L_1L_2, L_5L_8, L_2L_5, L_6L_8\}$.

Coût minimal : $1 + 2 + 2 + 3 + 3 + 4 + 4 = 19$ millions de FCFA.

5. Prim depuis L_1 :

Étape	Sommets atteints	Arêtes candidates (coût)	Choix
1	$\{L_1\}$	$L_1L_2(3), L_1L_4(6)$	$L_1L_2(3)$
2	$\{L_1, L_2\}$	$L_1L_4(6), L_2L_3(2), L_2L_5(4)$	$L_2L_3(2)$
3	$\{L_1, L_2, L_3\}$	$L_1L_4(6), L_2L_5(4), L_3L_6(5)$	$L_2L_5(4)$
4	$\{L_1, L_2, L_3, L_5\}$	$L_1L_4(6), L_3L_6(5), L_5L_6(8), L_5L_8(3), L_4L_5(1)$	$L_4L_5(1)$
5	$\{L_1, L_2, L_3, L_4, L_5\}$	$L_3L_6(5), L_5L_6(8), L_5L_8(3), L_4L_7(7)$	$L_5L_8(3)$
6	$\{L_1, \dots, L_5, L_8\}$	$L_3L_6(5), L_5L_6(8), L_4L_7(7), L_6L_8(4), L_7L_8(2)$	$L_7L_8(2)$
7	$\{L_1, \dots, L_5, L_7, L_8\}$	$L_3L_6(5), L_5L_6(8), L_6L_8(4)$	$L_6L_8(4)$

Arbre obtenu : $\{L_1L_2, L_2L_3, L_2L_5, L_4L_5, L_5L_8, L_7L_8, L_6L_8\}$ — exactement le même qu'avec Kruskal. Coût : $3 + 2 + 4 + 1 + 3 + 2 + 4 = 19$ millions de FCFA. Les deux algorithmes confirment le même coût minimal.

6. Sans le tronçon L_4L_5 : on réapplique Kruskal sur les 10 arêtes restantes. Les arêtes $L_2L_3(2)$, $L_7L_8(2)$, $L_1L_2(3)$, $L_5L_8(3)$, $L_2L_5(4)$, $L_6L_8(4)$ sont retenues comme précédemment (composante $\{L_1, L_2, L_3, L_5, L_6, L_7, L_8\}$ formée). $L_3L_6(5)$ est rejetée (cycle). Reste à connecter L_4 , dont les seules arêtes disponibles sont $L_1L_4(6)$ et $L_4L_7(7)$: on retient $L_1L_4(6)$. Nouvel ACPM : $\{L_2L_3, L_7L_8, L_1L_2, L_5L_8, L_2L_5, L_6L_8, L_1L_4\}$.

Nouveau coût minimal : $2+2+3+3+4+4+6 = 24$ millions de FCFA, soit un surcoût de 5 millions.

Points de vigilance : justifier explicitement chaque rejet par « l'arête crée un cycle » ; vérifier que l'arbre final compte exactement $n - 1 = 7$ arêtes ; dans Prim, ne considérer que les arêtes sortant de l'ensemble des sommets atteints ; exprimer le coût avec son unité (millions de FCFA).

CORRIGÉ Exercice 11 — Type Bac : synthèse autour des arbres et tableau de Dijkstra

Barème indicatif (sur 6 points) : Partie A : récurrence 1,5, conséquence 0,5, question 3 0,5 ; Partie B : justification du choix de C 0,5, case manquante 1, distance et chemin 0,75, reconstruction du graphe 1,25.

Partie A : démonstrations.

- Par récurrence sur $n \geq 1$: soit $P(n)$: « tout arbre à n sommets possède exactement $n - 1$ arêtes ».
 - Initialisation ($n = 1$) : un arbre à un sommet n'a aucune arête, et $0 = 1 - 1$: $P(1)$ est vraie.
 - Hérédité : soit $n \geq 1$ tel que $P(n)$ soit vraie. Soit T un arbre à $n+1$ sommets. Comme $n+1 \geq 2$, T possède au moins une **feuille** f (sommet de degré 1) ; soit e son unique arête

incidente. Le graphe T' obtenu en supprimant f et e a n sommets ; il est connexe (un chemin entre deux sommets restants ne pouvait pas passer par la feuille f , qui est de degré 1) et sans cycle (sous-graphe d'un graphe sans cycle) : c'est un arbre à n sommets. Par $P(n)$, T' a $n - 1$ arêtes, donc T en a $(n - 1) + 1 = n = (n + 1) - 1$: $P(n + 1)$ est vraie.

- Conclusion : $P(n)$ est vraie pour tout $n \geq 1$.
2. Soit G un graphe connexe à n sommets. G contient un arbre couvrant T (propriété du cours), qui a les mêmes n sommets et, d'après la question 1, exactement $n - 1$ arêtes. Comme $E(T) \subseteq E(G)$, on obtient $|E(G)| \geq n - 1$.
 3. Soit G connexe avec $n = 12$ sommets et $11 = n - 1$ arêtes. Supposons que G contienne un cycle : en supprimant une arête de ce cycle, on obtiendrait un graphe connexe à 12 sommets et 10 arêtes, ce qui contredit la question 2 ($10 < 12 - 1$). Donc G est sans cycle. Étant connexe et sans cycle, G est un arbre.

Partie B : tableau de Dijkstra.

1. À l'étape 2, les sommets non définitifs ont pour étiquettes : $C : 2, D : 9, E : 5$ (après mise à jour depuis C ... mais avant le traitement de l'étape 2, les étiquettes provisoires sont $B : 4, C : 2, E : 7, D : \infty$). La règle de Dijkstra impose de rendre définitive l'étiquette provisoire minimale : c'est celle de C (valeur 2). De plus, $2 \leq$ toute autre étiquette et les poids sont positifs, donc aucune amélioration ultérieure de C n'est possible : tout détour par un autre sommet coûterait au moins $2 + w(e) \geq 2$.
2. À l'étape 2, on traite C ($\text{dist}(C) = 2$) et on relâche l'arête CD : l'étiquette de D devient $2 + w(C, D) = 9$, d'où $w(C, D) = 7$. À l'étape 3, le minimum provisoire est E (valeur $5 < 9$) : E devient définitive. On relâche alors l'arête ED : nouvelle valeur candidate $5 + w(E, D)$. Le tableau de l'étape 4 montrant $D : 8(E)$, on a $5 + w(E, D) = 8$, soit $w(E, D) = 3$. Cette amélioration intervient à l'étape 3 : la case manquante est **8 (E)** (étiquette provisoire).
3. Distance minimale de A à D : **8** (étiquette définitive à l'étape 4). Chemin optimal : on remonte les prédécesseurs : $D \leftarrow E \leftarrow C \leftarrow A$, soit

$$A \rightarrow C \rightarrow E \rightarrow D, \quad \text{vérification : } 2 + 3 + 3 = 8.$$
4. Chaque création ou amélioration d'étiquette provient d'une arête :
 - Étape 1 (traitement de A) : $B : 4(A) \Rightarrow AB(4)$; $C : 2(A) \Rightarrow AC(2)$; $E : 7(A) \Rightarrow AE(7)$.
 - Étape 2 (traitement de C) : $D : 9(C) \Rightarrow CD(7)$; E passe de $7(A)$ à $5(C)$ avec $2 + 3 = 5 \Rightarrow CE(3)$.
 - Étape 3 (traitement de E) : D passe de $9(C)$ à $8(E)$ avec $5 + 3 = 8 \Rightarrow ED(3)$.

Liste des arêtes du graphe : $AB(4), AC(2), AE(7), CD(7), CE(3), ED(3)$.

Points de vigilance : dans une récurrence, rédiger explicitement initialisation, hérédité (avec l'hypothèse de récurrence citée) et conclusion ; pour le tableau, justifier le choix du sommet figé par « étiquette provisoire minimale » ; pour reconstruire le graphe, exploiter toutes les modifications d'étiquettes, y compris les améliorations (ici $E : 7 \rightarrow 5$ et $D : 9 \rightarrow 8$), pas seulement les créations.

15 FICHE BILAN

16 GRAPHES : DÉFINITIONS ET MODÉLISATION

16.1 Notions fondamentales

DÉFINITION Graphe, sommets, arêtes

Un **graphe** G est un couple ordonné $G = (S, A)$ où :

- S est un ensemble fini non vide dont les éléments sont appelés **sommets** (ou nœuds) ;
- A est un ensemble de paires de sommets distincts (pour un graphe simple) appelés **arêtes**.

Une arête reliant les sommets u et v se note $\{u, v\}$ ou simplement uv . On dit que u et v sont les **extrémités** de l'arête et qu'ils sont **adjacents** (ou voisins).

DÉFINITION Graphe orienté, graphe pondéré

- Un **graphe orienté** (ou **digraphe**) est un graphe dont les arêtes, appelées **arcs**, sont des couples ordonnés (u, v) . L'arc (u, v) part de u (origine) et arrive en v (extrémité). On note $u \rightarrow v$.
- Un **graphe pondéré** est un graphe (orienté ou non) muni d'une fonction **poids** $w : A \rightarrow \mathbb{R}_+$ (souvent $\mathbb{N}$ ou $\mathbb{R}^+$) qui associe un nombre (distance, coût, durée, capacité) à chaque arête.

DÉFINITION Sous-graphe, degré d'un sommet

- Un **sous-graphe** $G' = (S', A')$ de $G = (S, A)$ est obtenu en supprimant certains sommets de S (et toutes les arêtes incidentes) et/ou certaines arêtes de A . On note $G' \subseteq G$.
- Dans un graphe non orienté, le **degré** d'un sommet v , noté $\deg(v)$, est le nombre d'arêtes incidentes à v . Dans un graphe orienté, on distingue le **degré entrant** $\deg^-(v)$ (arcs arrivant en v) et le **degré sortant** $\deg^+(v)$ (arcs partant de v).

16.2 Modélisation : situations concrètes au Cameroun

EXEMPLE Modélisation du réseau routier interurbain

On souhaite modéliser les liaisons routières entre cinq villes : Yaoundé (Y), Douala (D), Ba-foussam (B), Ngaoundéré (N), Garoua (G).

- Sommets $S = \{Y, D, B, N, G\}$.
- Arêtes : YD (route nationale 3), YB (route nationale 6), BN (route nationale 6), NG (route nationale 1). Graphe non orienté (routes à double sens).
- Poids : distances en km. $w(YD) = 300$, $w(YB) = 300$, $w(BN) = 450$, $w(NG) = 350$.

Ce graphe est un arbre (connexe, sans cycle). Il permet d'étudier la connectivité du réseau.

EXEMPLE Modélisation d'un réseau d'antennes relais (MTN/Orange)

On modélise la couverture 4G dans un quartier de Douala.

- Sommets : Antennes relais A_1, A_2, A_3, A_4 .
- Arcs orientés : $A_i \rightarrow A_j$ si un téléphone peut basculer (handover) de l'antenne i vers j sans coupure. L'orientation compte car la puissance d'émission diffère.
- Poids : Temps de latence (ms) ou coût de commutation.

C'est un graphe orienté pondéré. L'algorithme de Dijkstra trouvera le chemin de commutation le plus rapide.

ATTENTION Pièges de modélisation

- Ne pas confondre sommet (entité : ville, antenne, personne) et arête (relation : route, câble, amitié).
- Vérifier l'orientation : une route à sens unique $\rightarrow$ arc ; une amitié Facebook $\rightarrow$ arête non orientée ; un suivi Twitter $\rightarrow$ arc.
- Les poids doivent être homogènes (tous en km, ou tous en FCFA, ou tous en minutes). On ne somme pas des km et des FCFA.

17 CHÂÎNES, CHEMINS, CYCLES ET CONNEXITÉ

DÉFINITION Chaîne, chemin, cycle, longueur

Soit $G = (S, A)$ un graphe.

- Une **chaîne** de G est une suite finie de sommets $v_0, v_1, \dots, v_k$ telle que pour tout $i \in \{0, \dots, k-1\}$, $\{v_i, v_{i+1}\} \in A$ (ou $(v_i, v_{i+1}) \in A$ si orienté, on parle alors de **chemin**).
- Les sommets v_0 et v_k sont les **extrémités** de la chaîne. Les sommets $v_1, \dots, v_{k-1}$ sont des **sommets intermédiaires**.
- La **longueur** d'une chaîne est le nombre k d'arêtes qui la composent.
- Une chaîne est **simple** si tous ses sommets sont distincts (sauf éventuellement les extrémités).
- Un **cycle** est une chaîne simple fermée ($v_0 = v_k$) de longueur ≥ 3 (ou ≥ 1 si boucles autorisées, hors programme simple).

PROPRIÉTÉ Graphe connexe

Un graphe non orienté G est **connexe** si pour tout couple de sommets distincts $(u, v) \in S^2$, il existe au moins une chaîne reliant u à v . Un graphe orienté est **fortement connexe** si pour tout couple (u, v) , il existe un chemin de u vers v et un chemin de v vers u . Il est **faiblement connexe** si le graphe non orienté associé est connexe.

EXEMPLE Chaînes, cycles et connexité

Soit le graphe $G : S = \{A, B, C, D, E\}$, $A = \{AB, BC, CD, DA, AC, BE\}$.

- $A \rightarrow B \rightarrow E$ est une chaîne (longueur 2), c'est un chemin simple.
- $A \rightarrow B \rightarrow C \rightarrow A$ est un cycle (longueur 3).
- $A \rightarrow C \rightarrow D \rightarrow A$ est un cycle (longueur 3).
- G est connexe (ex : E rejoint D par $E - B - A - D$ ou $E - B - C - D$).
- Si on retire l'arête BE , E devient isolé : le graphe n'est plus connexe.

MÉTHODE Vérifier la connexité d'un graphe

1. Choisir un sommet de départ s .
2. Effectuer un parcours en largeur (BFS) ou en profondeur depuis s (voir Section 4).
3. Si tous les sommets ont été visités, le graphe est connexe. Sinon, il est non connexe (il

a au moins deux composantes connexes).

18 ARBRES ET ARBRES COUVRANTS

DÉFINITION Arbre

Un **arbre** est un graphe non orienté connexe et sans cycle. Un **arbre raciné** est un arbre muni d'un sommet distingué appelé **racine**. Cela induit une orientation naturelle des arêtes (du parent vers les enfants) et une structure hiérarchique (niveaux, feuilles, père, fils, frères).

PROPRIÉTÉ Caractérisations d'un arbre – Démonstration exigible au Bac

Soit G un graphe non orienté à n sommets ($n \geq 1$). Les propriétés suivantes sont équivalentes :

1. G est un arbre (connexe et sans cycle).
2. G est connexe et possède exactement $n - 1$ arêtes.
3. G est sans cycle et possède exactement $n - 1$ arêtes.
4. Entre deux sommets quelconques de G , il existe exactement une chaîne simple.
5. G est sans cycle, mais l'ajout de n'importe quelle arête entre deux sommets non adjacents crée exactement un cycle.

EXPÉRIENCE Démonstration guidée : n sommets $\Rightarrow n - 1$ arêtes

Objectif : Prouver par récurrence sur n que tout arbre à n sommets a $n - 1$ arêtes.

1. Initialisation ($n = 1$) : Un arbre à 1 sommet a 0 arête. $0 = 1 - 1$. Vrai.
2. Hérédité : Soit T un arbre à $k + 1$ sommets ($k \geq 1$).
3. Lemme : Tout arbre à au moins 2 sommets possède au moins une **feuille** (sommet de degré 1). Preuve : Prendre un plus long chemin simple ; ses extrémités ont degré 1.
4. Supprimer une feuille f et son arête incidente. On obtient un graphe T' à k sommets.
5. T' est connexe (supprimer une feuille ne déconnecte pas) et sans cycle (sous-graphe d'un arbre). Donc T' est un arbre.

6. Par hypothèse de récurrence, T' a $k - 1$ arêtes.
7. T a une arête de plus que T' : $(k - 1) + 1 = k = (k + 1) - 1$ arêtes.
8. Conclusion : La propriété est vraie pour tout $n \in \mathbb{N}^*$.

DÉFINITION **Arbre couvrant**

Soit $G = (S, A)$ un graphe connexe. Un **arbre couvrant** de G est un sous-graphe $T = (S, A')$ tel que :

- T est un arbre (connexe, sans cycle) ;
- T contient tous les sommets de G (S identique) ;
- $A' \subseteq A$ et $|A'| = |S| - 1 = n - 1$.

PROPRIÉTÉ **Existence d'un arbre couvrant – Démonstration exigible au Bac**

Théorème. Tout graphe connexe admet au moins un arbre couvrant. Preuve (par suppression de cycles) : Soit G connexe. Si G est sans cycle, c'est un arbre, c'est son propre arbre couvrant. Sinon, G contient un cycle C . Retirons une arête e de C . Le graphe $G' = G \setminus \{e\}$ reste connexe (les sommets de C restent reliés par le reste du cycle). On répète l'opération tant qu'il existe un cycle. Le processus s'arrête car le nombre d'arêtes diminue. Le graphe final est connexe, sans cycle, et a les mêmes sommets : c'est un arbre couvrant.

MÉTHODE **Extraire un arbre couvrant d'un graphe connexe (méthode manuelle)**

1. Lister tous les cycles du graphe (visuellement ou par algorithme).
2. Tant qu'il reste un cycle, supprimer une arête quelconque de ce cycle.
3. Vérifier que le graphe obtenu a n sommets et $n - 1$ arêtes et est connexe.

Remarque : Il existe généralement plusieurs arbres couvrants différents pour un même graphe.

19 PARCOURS EN LARGEUR (BFS)

MÉTHODE Algorithme du parcours en largeur (BFS – Breadth-First Search)

Données : Graphe $G = (S, A)$ (non orienté ou orienté), sommet de départ $s \in S$. Structure : Une **file** (FIFO : First In, First Out) et un ensemble **Marqués** (ou tableau booléen).

1. Initialisation : $File \leftarrow [s]$; $Marqués \leftarrow \{s\}$; $Prédécesseur[s] \leftarrow Nil$; $Distance[s] \leftarrow 0$.
2. Tant que la file n'est pas vide :
 - (a) Défiler le sommet u en tête de file.
 - (b) Pour chaque voisin v de u (tel que $\{u, v\} \in A$ ou $(u, v) \in A$) faire :
 - Si $v \notin Marqués$ alors :
 - Marquer v ($Marqués \leftarrow Marqués \cup \{v\}$).
 - Enfiler v ($File \leftarrow File + [v]$).
 - $Prédécesseur[v] \leftarrow u$.
 - $Distance[v] \leftarrow Distance[u] + 1$.
3. Fin. L'ordre de visite est l'ordre de défilement. L'arbre des prédécesseurs donne les plus courts chemins (en nombre d'arêtes) depuis s .

PROPRIÉTÉ Propriétés fondamentales du BFS

- Le BFS visite les sommets par distance croissante (en nombre d'arêtes) au sommet de départ s .
- L'arbre de parcours (arêtes $(Prdcesseur[v], v)$) est un arbre couvrant de la composante connexe de s .
- Pour tout sommet v atteint, $Distance[v]$ est la longueur d'un plus court chemin (en nombre d'arêtes) de s à v .

EXEMPLE Exécution du BFS

Graphe : $S = \{A, B, C, D, E, F\}$, $A = \{AB, AC, BC, BD, CE, DF, EF\}$. Départ A .

Étape	File (tête → queue)	Sommet défilé	Marqués (ordre visite)
Init	[A]	–	A (dist 0)
1	[B, C]	A	B, C (dist 1)
2	[C, D]	B	D (dist 2)
3	[D, E]	C	E (dist 2)
4	[E, F]	D	F (dist 3)
5	[F]	E	–
6	[]	F	–

Ordre de visite : A, B, C, D, E, F. Arbre couvrant BFS : arêtes AB, AC, BD, CE, DF. Plus court chemin $A \rightarrow F$: A – B – D – F (longueur 3) ou A – C – E – F (longueur 3).

LESAVAIS-TU ? Application réelle : Propagation d'une alerte CAMTEL

CAMTEL détecte une panne sur un nœud central (Yaoundé). Le BFS modélise la propagation de l'alerte vers les nœuds voisins (centres régionaux), puis vers les suivants. La "distance" BFS donne le nombre de sauts (hops) pour que l'alerte atteigne chaque centre. Cela permet de dimensionner le temps de réaction du réseau.

20 ARBRE COUVRANT DE POIDS MINIMAL : ALGORITHMES DE KRUSKAL ET DE PRIM

DÉFINITION Graphe pondéré, poids d'un sous-graphe

Un **graphe pondéré** est un graphe $G = (S, A)$ muni d'une fonction poids $w : A \rightarrow \mathbb{R}_+$. Le **poids** d'un sous-graphe $H = (S_H, A_H)$ est la somme des poids de ses arêtes : $w(H) = \sum_{e \in A_H} w(e)$. Un **arbre couvrant de poids minimal** (ACM) d'un graphe connexe pondéré G est un arbre couvrant T tel que $w(T)$ soit minimal parmi tous les arbres couvrants de G .

20.1 Algorithme de Kruskal (approche globale par tri des arêtes)

MÉTHODE Algorithme de Kruskal

Données : Graphe connexe pondéré $G = (S, A)$, $|S| = n$.

1. Trier l'ensemble des arêtes A par poids croissant : $e_1, e_2, \dots, e_m$ avec $w(e_1) \leq w(e_2) \leq \dots \leq w(e_m)$.
2. Initialiser l'ensemble des arêtes retenues $A_T \leftarrow \emptyset$.
3. Pour i de 1 à m faire :
 - Si l'ajout de e_i à A_T ne crée pas de cycle dans le graphe (S, A_T) alors $A_T \leftarrow A_T \cup \{e_i\}$.
4. S'arrêter dès que $|A_T| = n - 1$. L'arbre couvrant minimal est $T = (S, A_T)$.

20.2 Algorithme de Prim (approche locale par croissance)

MÉTHODE Algorithme de Prim

Données : Graphe connexe pondéré $G = (S, A)$, $|S| = n$. Sommet de départ s quelconque.

1. Initialiser : $S_T \leftarrow \{s\}$ (sommets atteints), $A_T \leftarrow \emptyset$.
2. Tant que $|S_T| < n$ faire :
 - Parmi toutes les arêtes $\{u, v\} \in A$ avec $u \in S_T$ et $v \notin S_T$ (arêtes **sortantes** ou **frontière**), choisir celle de poids minimal $e^* = \{u^*, v^*\}$.
 - $S_T \leftarrow S_T \cup \{v^*\}$, $A_T \leftarrow A_T \cup \{e^*\}$.
3. Fin. $T = (S, A_T)$ est un arbre couvrant de poids minimal.

ATTENTION Kruskal vs Prim : points de vigilance

- Kruskal : on examine les arêtes par poids croissant et on rejette celles qui créeraient un cycle. Pratique quand la liste des arêtes est donnée.
- Prim : on fait grandir un arbre à partir d'un sommet en ajoutant à chaque étape l'arête de poids minimal qui le relie à un nouveau sommet.
- Unicité : Si tous les poids sont distincts, l'ACM est unique. Sinon, il peut y en avoir plusieurs de même poids minimal.
- Vérification : Toujours compter les arêtes retenues : exactement $n - 1$. Vérifier la connexité finale.

EXERCICE RÉSOLU 16 Câblage en fibre optique – Région du Centre

Énoncé. On veut relier 5 localités : Yaoundé (Y), Mbalmayo (M), Obala (O), Sa'a (S), Monaté (T). Coûts de câblage (en millions FCFA) : $YM = 10$, $YO = 15$, $YS = 20$, $YT = 25$, $MO = 12$, $MS = 18$, $OS = 8$, $OT = 15$, $ST = 10$. Déterminer le câblage de coût minimal reliant toutes les localités (ACM) par Kruskal et Prim. Solution détaillée.

SOLUTION

Kruskal : Tri des arêtes : $OS(8)$, $YM(10)$, $ST(10)$, $MO(12)$, $YO(15)$, $OT(15)$, $MS(18)$, $YS(20)$, $YT(25)$.

1. Prendre $OS(8)$. $A_T = \{OS\}$.
2. Prendre $YM(10)$. $A_T = \{OS, YM\}$.
3. Prendre $ST(10)$. $A_T = \{OS, YM, ST\}$. (Composantes : $\{O, S, T\}$, $\{Y, M\}$).
4. Prendre $MO(12)$. Relie M à O . $A_T = \{OS, YM, ST, MO\}$. (Composante : $\{Y, M, O, S, T\}$). Les 5 sommets sont reliés par $4 = n - 1$ arêtes : STOP.

Arêtes retenues : OS, YM, ST, MO . Poids total = $8 + 10 + 10 + 12 = 40$ millions FCFA.

Prim (départ Y) : $S_T = \{Y\}$. Sortantes : $YM(10)$, $YO(15)$, $YS(20)$, $YT(25)$. Min = $YM(10)$. $S_T = \{Y, M\}$, $A_T = \{YM\}$. Sortantes : $YO(15)$, $YS(20)$, $YT(25)$, $MO(12)$, $MS(18)$. Min = $MO(12)$. $S_T = \{Y, M, O\}$, $A_T = \{YM, MO\}$. Sortantes : $YO(15)$, $YS(20)$, $YT(25)$, $MS(18)$, $OS(8)$, $OT(15)$. Min = $OS(8)$. $S_T = \{Y, M, O, S\}$, $A_T = \{YM, MO, OS\}$. Sortantes : $YS(20)$, $YT(25)$, $MS(18)$, $ST(10)$, $OT(15)$. Min = $ST(10)$. $S_T = \{Y, M, O, S, T\}$, $A_T = \{YM, MO, OS, ST\}$. Poids total = $10 + 12 + 8 + 10 = 40$ millions FCFA. Même ACM. Interprétation : Le câblage optimal relie Obala-Sa'a (8), Yaoundé-Mbalmayo (10), Sa'a-Monaté (10), Mbalmayo-Obala (12). Coût minimal : 40 millions FCFA.

21 PLUS COURT CHEMIN : ALGORITHME DE DIJKSTRA

ATTENTION Condition d'application impérative – Piège fréquent au Bac

L'algorithme de Dijkstra ne fonctionne correctement que si tous les poids des arêtes sont positifs ou nuls ($w(e) \geq 0$ pour tout $e \in A$). Si le graphe contient des poids négatifs, l'algorithme peut donner un résultat faux (il ne "revient pas en arrière" pour corriger une marque). Hors programme : Bellman-Ford gère les poids négatifs (sans cycle absorbant).

MÉTHODE Algorithme de Dijkstra (tableau de marques – version Bac)

Données : Graphe orienté ou non pondéré $G = (S, A)$, poids $w \geq 0$, sommet source s . Tableau : Une ligne par sommet. Colonnes : Marque (distance estimée), Statut (Non traité / Traité), Prédécesseur.

1. Initialisation : $\text{Marque}[s] \leftarrow 0$; Pour tout $v \neq s$, $\text{Marque}[v] \leftarrow +\infty$. $\text{Statut}[\text{tous}] \leftarrow \text{Non traité}$. $\text{Prédécesseur}[\text{tous}] \leftarrow \text{Nil}$.
2. Répéter n fois (ou jusqu'à ce que la cible soit traitée) :
 - (a) Choisir le sommet u Non traité de marque minimale. (Si plusieurs, choisir l'un quelconque).
 - (b) Marquer u comme Traitée (Fixer u).
 - (c) Relâchement (Mise à jour) : Pour chaque voisin v de u Non traité :
 - Si $\text{Marque}[u] + w(u,v) < \text{Marque}[v]$ alors :
 - $\text{Marque}[v] \leftarrow \text{Marque}[u] + w(u,v)$.
 - $\text{Prédécesseur}[v] \leftarrow u$.
3. Fin. $\text{Marque}[v]$ = distance minimale (poids) de s à v . Chemin = remonter les prédécesseurs depuis v jusqu'à s .

EXPÉRIENCE Découverte guidée : Pourquoi le sommet de marque minimale est-il "fixé" ?

Considérons un graphe à poids positifs. Soit u le sommet non traité de marque minimale d .

1. Tout chemin de s vers u doit passer par un dernier sommet non traité x avant d'entrer dans u (ou venir directement de s).
 2. La longueur de ce chemin est $\geq \text{Marque}[x] + w(x,u)$.
 3. Comme x est non traité, $\text{Marque}[x] \geq \text{Marque}[u] = d$ (par choix de u).
 4. Comme $w(x,u) > 0$, la longueur du chemin est $> d$.
 5. Mais on a déjà un chemin de longueur d (celui qui a donné la marque d à u).
 6. Donc aucun chemin plus court n'existe. La marque d est optimale et définitive.
- C'est la justification de la correction de l'algorithme (invariant de boucle).

EXEMPLE Exécution de Dijkstra – Réseau routier simplifié

Sommets : A (Douala), B (Edea), C (Yaoundé), D (Bafia), E (Bafoussam). Arcs pondérés (km) : $A \rightarrow B(60)$, $A \rightarrow C(300)$, $B \rightarrow C(240)$, $B \rightarrow D(100)$, $C \rightarrow D(120)$, $C \rightarrow E(280)$, $D \rightarrow E(150)$. Source A . Cible E .

Sommet	Init	Après A	Après B	Après D	Après C
A	0 (T)	0 (T)	0 (T)	0 (T)	0 (T)
B	∞	60 (A)	60 (T)	60 (T)	60 (T)
C	∞	300 (A)	300 (A)	300 (A)	300 (T)
D	∞	∞	160 (B)	160 (T)	160 (T)
E	∞	∞	∞	310 (D)	310 (D)

Ordre de fixation : $A(0), B(60), D(160), C(300), E(310)$. Plus court chemin $A \rightarrow E$: Remonter Prédécesseurs : $E \leftarrow D \leftarrow B \leftarrow A$. Chemin : $A \rightarrow B \rightarrow D \rightarrow E$. Distance = $60 + 100 + 150 = 310$ km. (Vérification : $A \rightarrow C \rightarrow E = 580, A \rightarrow B \rightarrow C \rightarrow E = 580, A \rightarrow C \rightarrow D \rightarrow E = 570$).

MÉTHODE Reconstitution du plus court chemin

Une fois le tableau de Dijkstra terminé pour la cible t :

1. Si $\text{Marque}[t] = +\infty$: aucun chemin n'existe.
2. Sinon : Initialiser liste $L \leftarrow [t], u \leftarrow t$.
3. Tant que $\text{Prédécesseur}[u] \neq \text{Nil}$:
 - $u \leftarrow \text{Prédécesseur}[u]$.
 - Insérer u en tête de L .
4. L est le plus court chemin de s à t . Distance = $\text{Marque}[t]$.

EXERCICE RÉSOLU 17 Optimisation trajet Douala $\rightarrow$ Bafoussam via Bafia

Énoncé. Reprenons le graphe de l'exemple précédent. Un transporteur veut aller de Douala (A) à Bafoussam (E) en minimisant le temps de trajet. Les poids sont maintenant des durées (minutes) : $A \rightarrow B(90), A \rightarrow C(360), B \rightarrow C(300), B \rightarrow D(130), C \rightarrow D(150), C \rightarrow E(330), D \rightarrow E(180)$. Appliquer Dijkstra. Solution détaillée.

SOLUTION

Tableau Dijkstra (source A) :

Sommet	Init	Fixe A	Fixe B	Fixe D	Fixe C
A	0(T)	0(T)	0(T)	0(T)	0(T)
B	∞	90(A)	90(T)	90(T)	90(T)
C	∞	360(A)	360(A)	360(A)	360(T)
D	∞	∞	220(B)	220(T)	220(T)
E	∞	∞	∞	400(D)	400(D)

Chemin optimal : $A \rightarrow B \rightarrow D \rightarrow E$. Durée = $90 + 130 + 180 = 400$ min (6h 40min). Interprétation : Passer par Yaoundé (C) prend au minimum $360 + 330 = 690$ min ou $360 + 150 + 180 = 690$ min. La route "secondaire" via Edea (B) et Bafia (D) est plus rapide de près de 5 heures.

22 SYNTHÈSE ET MODÉLISATION DE SITUATIONS



Carte mentale de la leçon M12 : Théorie des graphes

À RETENIR L'essentiel de la leçon : définitions et résultats clés

Définitions à connaître par cœur.

- **Graphe** $G = (S, A)$: ensemble de **sommets** S reliés par des **arêtes** A . Graphe **orienté** : arcs (u, v) avec sens. Graphe **pondéré** : fonction poids $w : A \rightarrow \mathbb{R}_+$.
- **Sous-graphe** : obtenu par suppression de sommets/arêtes.

- **Chaîne** : suite de sommets consécutifs reliés par des arêtes ; **longueur** = nb arêtes.
Chemin : chaîne en graphe orienté (sens respecté). **Cycle** : chaîne fermée simple (extrémités identiques, arêtes distinctes).
- **Graphe connexe** : $\forall u, v \in S, \exists \text{ chaîne } u \rightsquigarrow v$.
- **Arbre** : graphe connexe sans cycle. **Arbre couvrant** : sous-graphe arbre contenant tous les sommets de G .
- **Poids d'un sous-graphe** : somme des poids de ses arêtes.

Théorèmes et propriétés (démonstration exigible pour ceux marqués *).

Résultat	Énoncé précis
Arbre*	Un arbre à n sommets possède exactement $n - 1$ arêtes. (Récurrence sur n en supprimant une feuille).
Connexité	Un graphe connexe à n sommets a au moins $n - 1$ arêtes.
Existence AC*	Tout graphe connexe admet un arbre couvrant. (Preuve : suppression d'arêtes de cycles).
BFS	Visite par distance croissante (en nb d'arêtes) depuis la source. Donne plus courts chemins en nombre d'arêtes.
Kruskal / Prim*	Sur graphe connexe pondéré, construisent un arbre couvrant de poids minimal (ACM).
Dijkstra*	Sur graphe à poids ≥ 0 , donne plus court chemin en poids depuis une source vers tous les sommets.

MÉTHODE

Méthodes express – Résumé pour l'examen

1. BFS (Parcours en largeur) : File d'attente (FIFO). Marquer départ. Répéter : défiler u , enfiler voisins non marqués. Ordre de visite = ordre de défilement. Donne distances en sauts.
2. Kruskal (ACM global) : Trier arêtes poids croissant. Parcourir liste : ajouter arête si pas de cycle créé. Stop à $n - 1$ arêtes.
3. Prim (ACM local) : Partir d'un sommet. Répéter : ajouter arête de poids minimal reliant ensemble atteint $\rightarrow$ ensemble non atteint. Stop quand tous sommets atteints.
4. Dijkstra (Plus court chemin pondéré) : Tableau marques (1 ligne/sommet). Init source, $+\infty$ autres. Répéter : fixer sommet non traité de marque min ; mettre à jour voisins non traités : $\text{marque}(v) \leftarrow \min(\text{marque}(v), \text{marque}(u) + w(u, v))$. Reconstituer chemin par prédécesseurs. Condition : poids ≥ 0 .
5. Modéliser : 1) Identifier sommets (objets). 2) Identifier arêtes/arcs (relations). 3) Définir poids (coût, distance, temps). 4) Choisir orienté/non orienté. 5) Appliquer algo.

6) Interpréter : "Le coût minimal de câblage est de 40 millions FCFA", "Le trajet le plus court fait 310 km via Bafia".

À RETENIR Checklist avant le Bac

- ☐ Je sais définir graphe, sommet, arête, arc, sous-graphe, graphe orienté, graphe pondéré, degré.
- ☐ Je sais modéliser une situation concrète (réseau routier, câblage fibre, antennes relais, relations sociales) par un graphe adapté.
- ☐ Je distingue chaîne, chemin, cycle, et je calcule la longueur d'une chaîne.
- ☐ Je sais justifier qu'un graphe est connexe (exhiber chaînes) ou non (exhiber composantes).
- ☐ Je sais qu'un graphe connexe admet un arbre couvrant (théorème) et qu'un arbre à n sommets a $n - 1$ arêtes (propriété équivalente).
- ☐ Je sais extraire un arbre couvrant manuellement (supprimer arêtes de cycles) ou via BFS/Prim/Kruskal.
- ☐ Je sais exécuter un BFS avec file, donner ordre de visite, arbre de parcours, distances en sauts.
- ☐ Je sais appliquer Kruskal : tri arêtes, test cycle, arrêt $n - 1$ arêtes.
- ☐ Je sais appliquer Prim : ensemble atteint, choix arête sortante min, maj ensemble, arrêt n sommets.
- ☐ Je sais remplir le tableau de Dijkstra (marques, statuts, prédécesseurs) et reconstituer le chemin.
- ☐ Je vérifie toujours : poids ≥ 0 pour Dijkstra ; $n - 1$ arêtes pour ACM ; connexité finale.
- ☐ Je conclus systématiquement par une phrase d'interprétation dans le contexte (coût en FCFA, distance en km, durée en min).

EXERCICE 17 Entraînement : Réseau d'antennes 5G – MTN Cameroun

On souhaite déployer la 5G dans 6 quartiers de Douala : Q_1 (Akwa), Q_2 (Bonanjo), Q_3 (Deido), Q_4 (Bessengue), Q_5 (New Bell), Q_6 (Logbaba). Les coûts de liaison fibre entre antennes (en millions FCFA) sont : $Q_1Q_2(5)$, $Q_1Q_3(8)$, $Q_2Q_3(6)$, $Q_2Q_4(10)$, $Q_3Q_4(7)$, $Q_3Q_5(12)$, $Q_4Q_5(9)$, $Q_4Q_6(11)$, $Q_5Q_6(4)$.

1. Modéliser par un graphe pondéré. Le graphe est-il connexe ?
2. Déterminer un arbre couvrant de poids minimal par l'algorithme de Kruskal. Donner le

coût total.

3. Déterminer un arbre couvrant de poids minimal par l'algorithme de Prim (départ Q_1). Vérifier le coût.
4. On ajoute une contrainte : la liaison Q_1Q_3 est interdite (traversée fleuve Wouri trop coûteuse). Recalculer l'ACM (Kruskal).
5. Interpréter le résultat final pour le directeur technique.

CORRIGÉ Exercice : Réseau d'antennes 5G – MTN Cameroun

1. Modélisation. Sommets $S = \{Q_1, \dots, Q_6\}$. Arêtes pondérées comme données. Graphe non orienté. Connexe ? Oui, par ex $Q_1 - Q_2 - Q_4 - Q_6 - Q_5 - Q_3$ relie tout. 2. Kruskal. Tri par poids croissant : $Q_5Q_6(4), Q_1Q_2(5), Q_2Q_3(6), Q_3Q_4(7), Q_1Q_3(8), Q_4Q_5(9), Q_2Q_4(10), Q_4Q_6(11), Q_3Q_5(12)$.
 1. $Q_5Q_6(4)$ OK. Comp : $\{Q_5, Q_6\}$. 2. $Q_1Q_2(5)$ OK. Comp : $\{Q_1, Q_2\}, \{Q_5, Q_6\}$. 3. $Q_2Q_3(6)$ OK. Comp : $\{Q_1, Q_2, Q_3\}, \{Q_5, Q_6\}$. 4. $Q_3Q_4(7)$ OK. Comp : $\{Q_1, Q_2, Q_3, Q_4\}, \{Q_5, Q_6\}$. 5. $Q_1Q_3(8)$: Crée cycle dans $\{Q_1, Q_2, Q_3\}$. Rejetée. 6. $Q_4Q_5(9)$: Relie les 2 composantes. OK. Comp : $\{Q_1..Q_6\}$. 5 arêtes ($n-1 = 5$). STOP. ACM : $\{Q_5Q_6, Q_1Q_2, Q_2Q_3, Q_3Q_4, Q_4Q_5\}$. Poids = $4 + 5 + 6 + 7 + 9 = 31$ M FCFA.
 3. Prim (départ Q_1). $S_T = \{Q_1\}$. Sortantes : $Q_1Q_2(5), Q_1Q_3(8)$. Min = $Q_1Q_2(5)$. $S_T = \{Q_1, Q_2\}$. Sortantes : $Q_1Q_3(8), Q_2Q_3(6), Q_2Q_4(10)$. Min = $Q_2Q_3(6)$. $S_T = \{Q_1, Q_2, Q_3\}$. Sortantes : $Q_1Q_3(8), Q_2Q_4(10), Q_3Q_4(7), Q_3Q_5(12)$. Min = $Q_3Q_4(7)$. $S_T = \{Q_1, Q_2, Q_3, Q_4\}$. Sortantes : $Q_1Q_3(8), Q_2Q_4(10), Q_3Q_5(12), Q_4Q_5(9), Q_4Q_6(11)$. Min = $Q_4Q_5(9)$. $S_T = \{Q_1..Q_5\}$. Sortantes : $Q_3Q_5(12), Q_4Q_6(11), Q_5Q_6(4)$. Min = $Q_5Q_6(4)$. $S_T = \{Q_1..Q_6\}$. Arêtes : $Q_1Q_2, Q_2Q_3, Q_3Q_4, Q_4Q_5, Q_5Q_6$. Poids = $5 + 6 + 7 + 9 + 4 = 31$ M FCFA. Identique.
 4. Sans Q_1Q_3 . Kruskal : même liste sans $Q_1Q_3(8)$. Sélection identique (cette arête était rejetée de toute façon). ACM inchangé, coût 31 M FCFA.
 5. Interprétation. Le déploiement optimal relie : New Bell-Logbaba (4), Akwa-Bonanjo (5), Bonanjo-Deido (6), Deido-Bessengue (7), Bessengue-New Bell (9). Coût minimal total : 31 millions FCFA. La liaison directe Akwa-Deido (8) est redondante et non retenue. L'interdiction de la traversée Wouri (Akwa-Deido) n'impacte pas le coût optimal.



Signé • OnBuch+

ÉQUIPE PÉDAGOGIQUE ONBUCH+
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+